

Chess A.I Game

Candidate Name: Ahmer Mahmood Alam

Candidate Number: 8919

Centre Name: Dr Challoner's Grammar School

Centre Number: 52205

Qualification Code: 7517

Date: Various (2021-2022)

Contents

Analysis	3
Problem Identification	3
Third-Parties Involved	3
Research: Building a Robust Chess A.I	4
The Evaluation Function	4
Existing Searching Algorithms: the Minimax Algorithm	9
Optimising Minimax using Alpha-Beta Pruning	11
Managing Game Difficulty Levels	13
Discussion with my Third-Parties	14
Questions for my Third-Parties	14
The Responses	15
My thoughts on the Responses	16
My Proposed Objectives	17
Documented Design	20
User-Interface Design	20
Main Menu Design	20
Chessboard and Chess Pieces Design	21
Creating the Game Functionalities	23
The OOP approach	23
Defining Movement Patterns of Chess Pieces	25
Identifying and Handling In-game States	32
Creating the A.I Opponent	36
Calculating Current Gamephases	37
Evaluation Function Design	38
Defining the Minimax Function	45
Adding Alpha-Beta Pruning to the Minimax Function	47
Further User Interface Design: Post Game Report Design	50
Technical Solution and Testing	52
Implementing the Main Menu (1.)	55
Implementing the A.I Difficulty UI	56
Implementing the Colour Selection UI	58
Implementing the Start Button Mechanism	60
Testing the Main Menu (1.)	61
Implementation of the Game Functionalities (2./3./4./5.)	62
Creating and Displaying the Chessboard and Pieces	62
Defining Piece Characteristics using Complex OOP	65
Updating and Maintaining the Game using the Chessboard, Gamedata and Move_engine Classes	74
Making a move	76
Undo a move	86
Testing the Functionality of the Game (2./3./4./5.)	89
Implementing the A.I (5./6./7./8./9.)	94
Implementing the Evaluation Function	96
Implementing the Minimax Algorithm (w/ alpha-beta pruning)	102

Hint move	106
Testing the A.I (5./6./7./8./9.)	107
Tuning the numerical evaluation values correctly	109
Redesigning the Move Generation Algorithm	115
Implementing the Renovated Move Generation Algorithms	119
Implementing the Post Game Report(10.)	125
Implementing the player move evaluation algorithm	126
Displaying the report	128
Testing the Post Game Report(10.)	129
Evaluation	130
Proposed Objectives Checklist	130
Appendices	136
Bibliography	136
Full Code	137
Constants.py	137
Main.py	138
Pieces.py	156
AI.py	163
Gamedata.py	169

Analysis

Problem Identification

Chess is a beautiful game. There are more possibilities of chess games than there are atoms of the universe, but each ending is as unique as the next. Benjamin Franklin once said: *'Life is a kind of Chess, in which we often have points to gain, and competitors or adversaries to contend with'*. With that being said, Chess can be intimidating for the beginner to get into unlike simpler games such as checkers and tic tac toe. It is estimated that 90% of the world population do not know how to play chess, my brother being one of them. He informs me that the complexity of chess is the reason for his reluctance to learn it, and I suspect this is the case for a large portion of chess novices.

Therefore, my solution is to create a chess game which is as beginner-friendly as possible in order for new players to comfortably understand the basic fundamentals of the game. The main functionality of my game will be that there is a built-in A.I opposition for the user to play against. The difficulty of the A.I opponent can be altered by the user depending on the skill set the user possesses. Ideally for a beginner chess player, he/she would apply the lowest difficulty of A.I, then gradually increase this to further develop their newly-found skills.

Third-Parties Involved

The clients and demographics my software aims for would primarily be anybody that wants to begin learning chess. However, the stakeholders of my project will appeal to those who are veterans at the game as well as a representative sample of the beginner demographic.

The clients for the beginner chess player demographic are Humza Alam, my brother who I mentioned earlier, and Sebastian Lawrence, a friend of mine. They both desire to one day know how to play chess and would both be some of the end users for my program. I think having direct third-party users such as Humza and Seb would be useful as I can acquire an accurate understanding of the general needs the end users of my game would likely want.

Whilst they won't necessarily be end users for the game themselves, I concluded that also involving third-parties with more experience in chess would also be beneficial, as for instance, they could provide insight as to how a good and bad chess player thinks (this would help me build the A.I opponent). The stakeholders in this area are Jake Barry and Jacob Gresty, both are experienced chess players that play on a regular basis at a high level.

Research: Building a Robust Chess A.I

The Evaluation Function

In chess, an evaluation function is implemented into chess programs to estimate how good (or bad) a state of the board is for a player. It is represented by a numerical score. There are 2 key properties a good evaluation function considers when assessing which player has the current advantage:

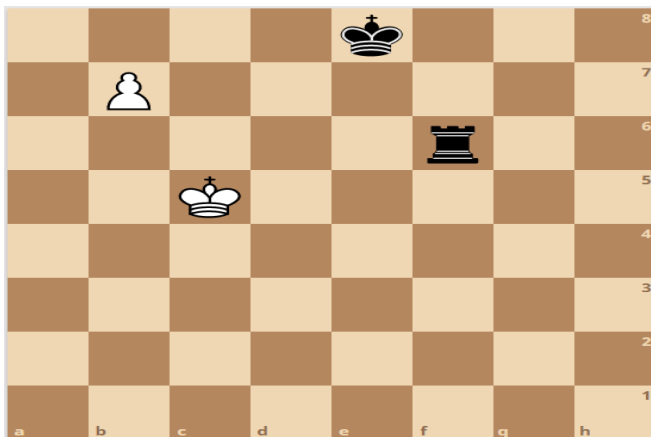
1. The total value of all of the pieces on the board (comparing materials)
2. The position of the pieces on the board

Determining the value of the pieces on the board (comparing materials)

To the right is a picture of the universally respected values of each of the 6 pieces in chess, in terms of white's perspective. The queen is the most powerful piece thus it has an extremely high value. However, the king has an almost infinite value, which makes sense as if you lose the king, you lose the game.

If we solely focus on the sum of the value of pieces on the board at its current state without looking ahead, there are 3 possible outcomes when :

1. Sum is positive = White is winning
2. Sum is negative = Black is winning
3. Sum is 0 = Game is tied



If we consider the scenario above, the net sum of the pieces equates to -40, suggesting that, if we only compare materials, black is winning.

	10		-10
	30		-30
	30		-30
	50		-50
	90		-90
	900		-900

Limitations of ONLY comparing materials: Whilst the A.I will be competent enough to undertake the measures to maximise the net material of the board towards its favour, its positional playing would be abysmal as the A.I has no conceptual knowledge of king safety, double pawn structures, centre control etc.

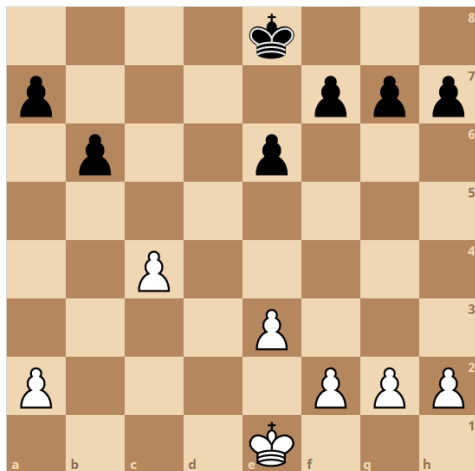
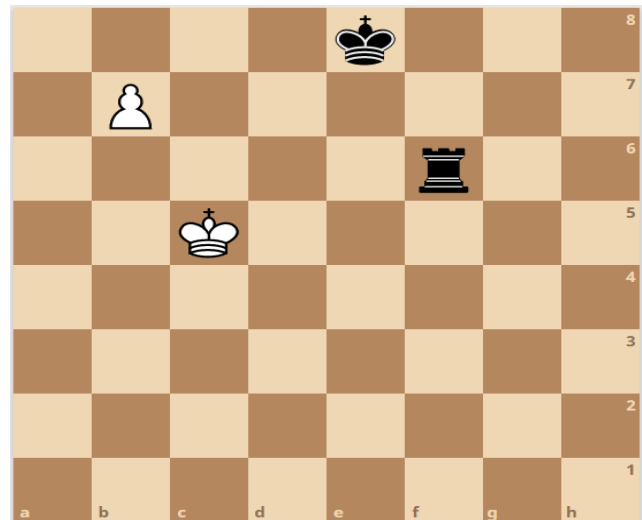
Evaluating the positions of the pieces on the board

I have divided the assessment of the positions of the pieces a robust chess evaluation function should consider into 3 further components:

1. Pawn structure
2. Centre control
3. King safety

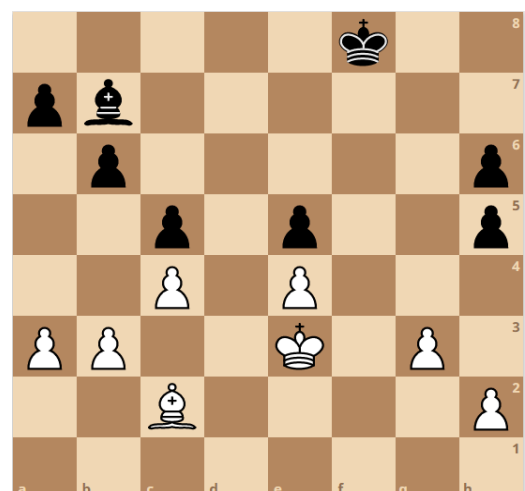
Pawn structure

Pawn promotion: If we observe the exact same scenario shown previously, a chess evaluation function with integrated knowledge of pawn promotion as well as material comparing would score this position heavily in favour of white instead, as the b7 pawn is perfectly positioned for advancing towards the 8th rank and promoting itself to a superior piece such as a queen without effective resistance. In general, pawns positioned near the 8th rank for white (or the 1st rank for black) should have its score considerably boosted by the evaluation function, given if there is no effective way for the opposition to remove the pawn.



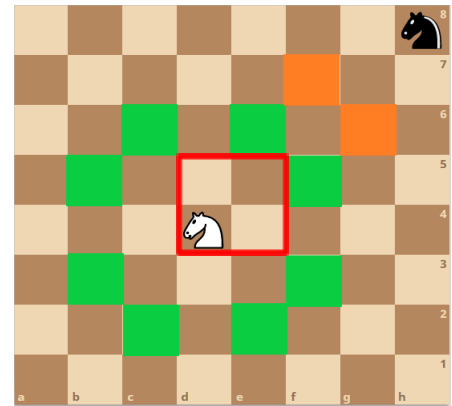
Pawn islands: In chess it is beneficial if pawns are connected to each other as they can offer protection for one another if they were to be attacked. A 'pawn island' is known when there are no pawns present in either of the adjacent files to a pawn on the board. In this game, white has 2 'pawn islands' on the 'a' and 'c' file, so both pawns are prone to attacks from the black king. Whereas both sets of black's pawns are connected, therefore the evaluation function should produce a negative score in favour of black if pawn positioning is also considered (as material is equal for both sides).

Doubled pawns: A doubled pawn structure is when there are 2 (or in rare cases more) pawns on the same rank, usually behind one another. Initially, I wasn't too sure why this should be a weakness that an evaluating system should pick up, so I asked Jake, one of the third-parties involved in my project. This was his response: *'Doubled pawns, by their nature, are weaker due to the fact that they can't defend each other and as such will become a liability that will require additional pieces to defend. The more advanced pawn will additionally lead to the other pawn being restricted as its path is impeded.'* The 2 black pawns on the 'h' files are 'doubled', thus are much harder to defend and will inevitably be captured. The evaluation function would score this board in favour of white.



Centre control

Centre control is very important in chess, particularly in the opening. Why? The main reason is that pieces placed at the centre are much more active, as they control more squares on the board than the piece can jump to within a single tempo. Furthermore, as a central piece is situated in the middle of the board, it can fulfil the roles of defence and offence simultaneously. These 2 aspects make a centralised piece very powerful in a game. There are 3 primary methods existing evaluating functions employ in order to measure which player has better control of the centre.



Example: a knight positioned at the centre has more moves available than one on a side/corner. This applies to all pieces.

Quantity of pawns controlling the centre: When I refer to the 'central squares', these are the 4 squares highlighted inside the red box on the image to the right ('d4', 'd5', 'e4', 'e5'). Evaluating functions in chess count up the number of pawns that directly influence the central squares (can jump **diagonally** to the central square within 1 move) for both sides, and whichever has more would have better control of the centre.



In this scenario, white has 2 pawns (shaded in aqua) controlling the central squares, whereas black only has one (shaded in turquoise).

Quantity of the remaining pieces controlling the centre: If we assess the same game, white boasts 4 non-pawn pieces (shaded in orange) influencing the central squares in relation to 3 for black (shaded in red). Although black's queen technically controls 2 of the central squares, the evaluation function will only count this as 1 point, as the queen (as well as any other piece) can only be present on one square at a time.

Quantity of pieces inside the centre: White has a pawn inside one of the central squares, whereas black does not. This benefits white.

Therefore, a good evaluation function will conclude that white has the advantage in terms of having superior control of the centre.

It is important to note that the amount of pawns controlling the centre/inside the centre should hold greater weight when a chess A.I evaluates centre control of the board. The reason for this is that ironically due to their low value, pawns are not ideal to exchange for as a result of giving away other more valuable pieces, therefore making pawns very effective at controlling squares.

King safety

Ensuring that your king is in a position away from harm as soon as possible is key in chess, especially during the middle of the game. A badly placed king near the centre is vulnerable to attacks from the opposition that must be addressed in the form of checks, which hinders the development of other pieces.

Castling: It is generally a very good idea to castle as soon as possible. This allows your king to be away from the centre to a position where it is very difficult for it to be attacked, as the pawns in front provide protection. Furthermore, the castled rook becomes much more active in which it can perform tasks such as controlling the centre etc. Existing evaluation functions almost always provide a bonus score to a player that has castled.

The pawn shield: This refers to the pawns in front of the king. Ideally, a safe king should have 3 pawns in front of it which can block off lines of sights to the king from attacking pieces, and should never be moved until the endgame. The evaluation function should check the presence of these 3 pawns in front of the kings when determining the overall score of the current position of the board.



The green squares shaded in this scenario is where the 'pawn shield' should be so that the respective kings are protected: white's pawn shield is perfect, whereas black is missing 2 pawns. Furthermore, black hasn't castled yet, meaning that its rook hasn't been developed as well as the fact that the king is still on the centre of the board. As the other aspects to evaluate (material, centre control etc.) are equal, the evaluation function should produce a positive score in favour of white as white has a safer king.

(All board scenarios produced using the Lichessboard editing software. Link: <https://lichess.org/editor>)

Determining the weight of the components of a chess evaluation function score

Now that I have identified the main components that most existing evaluation functions consider when producing its overall score of the current board, the question is what component should the A.I weigh up the most (as some components are clearly more important than others). The answer is not as black-and-white: the importance of these aspects varies throughout different stages of the game, though it is important to note that material should always be the most important factor at all stages. A chess game can be defined into 3 phases:

1. The opening
2. The middlegame
3. The endgame

The opening

- Begins after the first move has been made (of course!)
- Material comparison is a significant factor to consider
- Pawn structure initially is an insignificant factor to consider
- Centre control initially is a significant factor to consider
- King safety initially is a moderately significant factor to consider

The middlegame

- It can be difficult to pinpoint the exact transition from the opening to the middlegame, but the general rule of thumb is that the middlegame begins after each player has completed 10 moves, or if both players have castled
- Material comparison is a significant factor to consider
- Pawn structure becomes a moderately significant factor to consider
- Centre control becomes a moderately significant factor to consider
- King safety becomes a significant factor to consider

The endgame

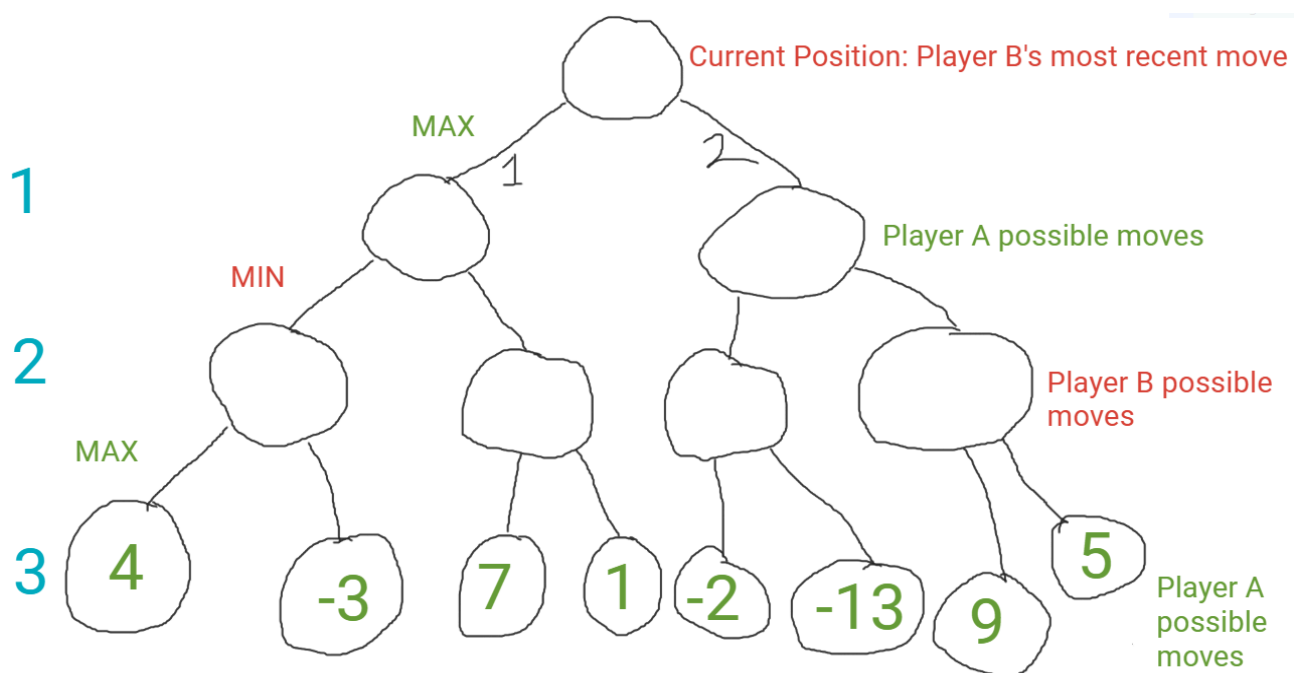
- The exact transition is also not clearly defined. The general rule is that the endgame begins when there are 3 or less pieces present on the board that are not pawns or kings.
- Material comparison is a critical factor to consider
- Pawn structure becomes a significant factor to consider
- Centre control becomes an insignificant to consider
- King safety becomes an insignificant factor to consider

Existing Searching Algorithms: the Minimax Algorithm

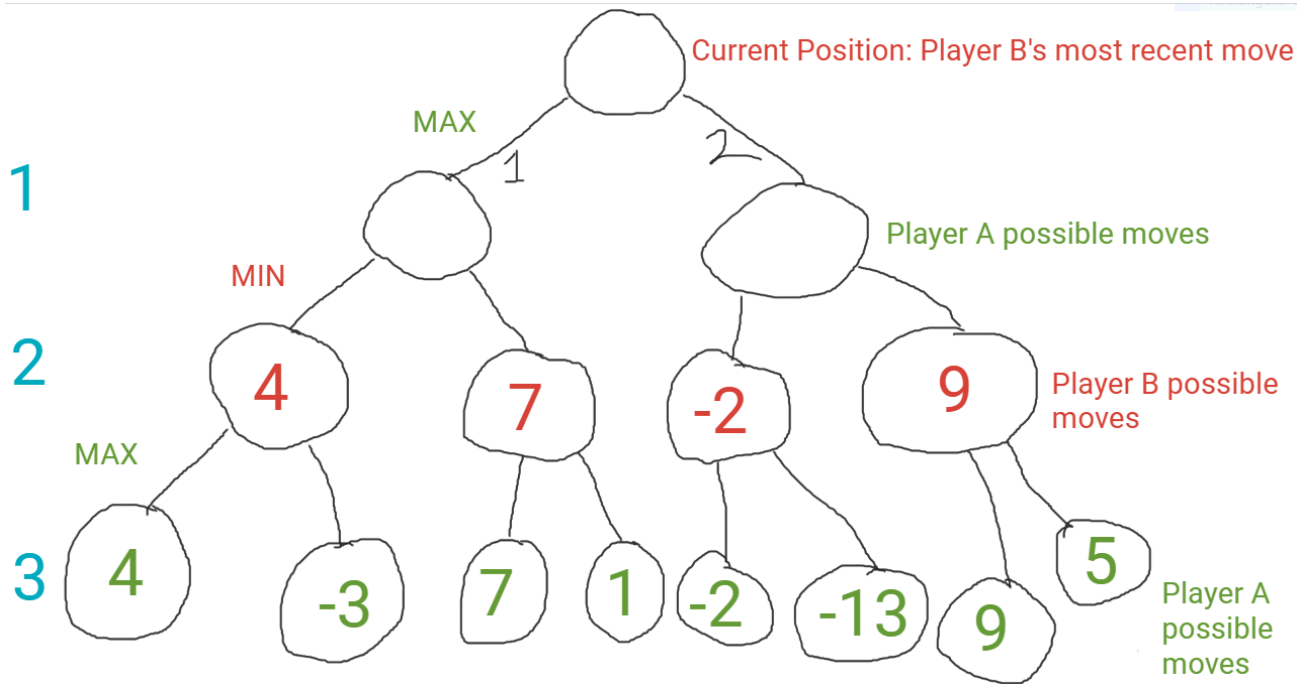
Now that our chess A.I has a method to evaluate any position on the board, we now require for it to choose the best move possible in relation to the difficulty applied to it. Most famous chess engines such as Stockfish and Rybka use the minimax algorithm to do this. The minimax algorithm is a recursive search tree that represents all of the possible moves that can be explored to a given depth. The root node of the tree is the original board, and all of the nodes branching from it are the reachable positions.

The minimax algorithm utilises the fact chess is a zero-sum game, a term used in game theory meaning that maximising your chance of winning is the same as minimising your opponent's chance of winning. Therefore, one player's goal could be to maximise the evaluation function, while the other tries to minimise it.

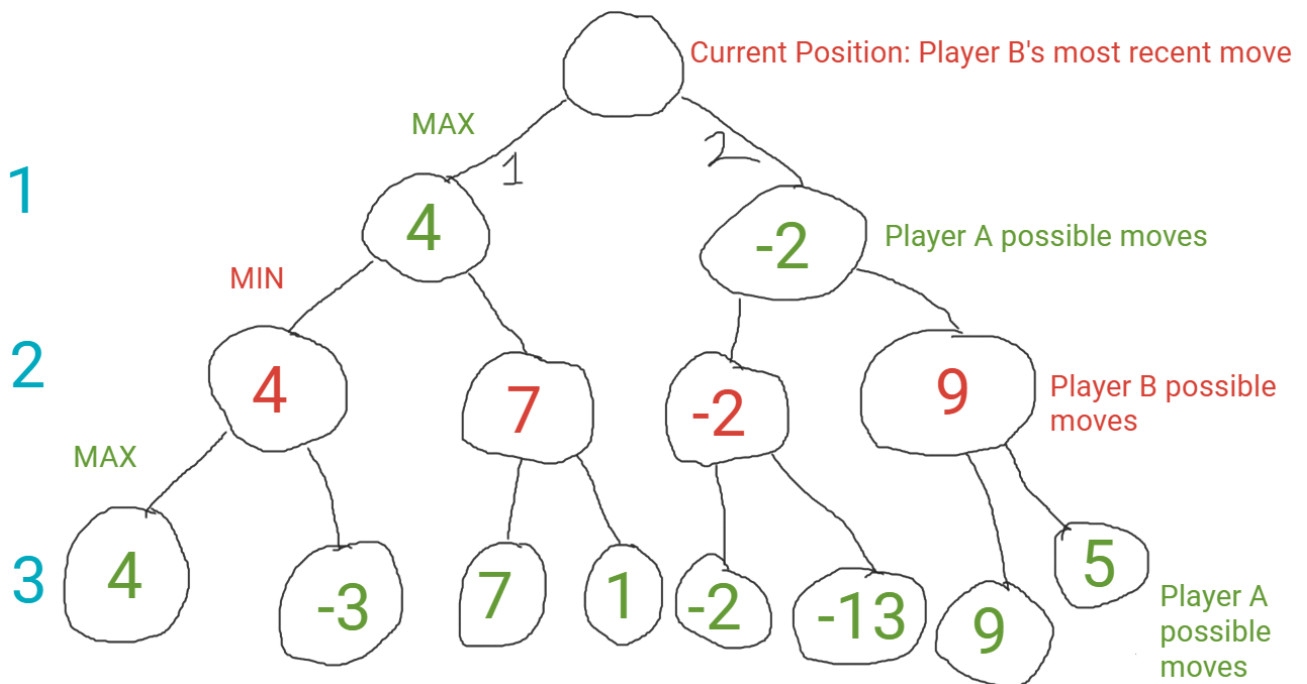
To illustrate the minimax search tree clearer, if we create a scenario where there are players A (the A.I) and B (a real-life player), in which the greater the value of the evaluation function, the better the game is for player A. Therefore, player A aims to maximise the evaluation function whereas player B wants to minimise it. The root node is the current position of the board after player B has moved, and player A wants to pick the best possible move of the 2 moves available which respectively lead to 'position 1' and 'position 2'.



The image shows a tree of all of the possible moves 3 moves into the future, meaning that in this instance, the A.I is using a depth of 3. Minimax begins from the bottom nodes (positions after 3 future moves), and works its way up towards the highest child nodes (position after 1 future move). The lowest child nodes have had their positions evaluated by the evaluation function, and have been rated as a numerical score. As its player A's move, it needs to pick the highest number in order to maximise the evaluation function. For example, if we observe the leftmost nodes, as $4 > -3$, player A will choose the move with the evaluation score of 4.



This process repeats for the remaining child nodes when eventually we get the tree to be like this. Now it's player B's turn, and the minimax algorithm assumes that player B will play the best possible move. Therefore, player B would choose the move in which the evaluation function produces the lowest score. For example, if we assess the leftmost nodes, as $4 < 7$, minimax will choose the position with the evaluation score of 4, as this would be the best possible move that player B would have.

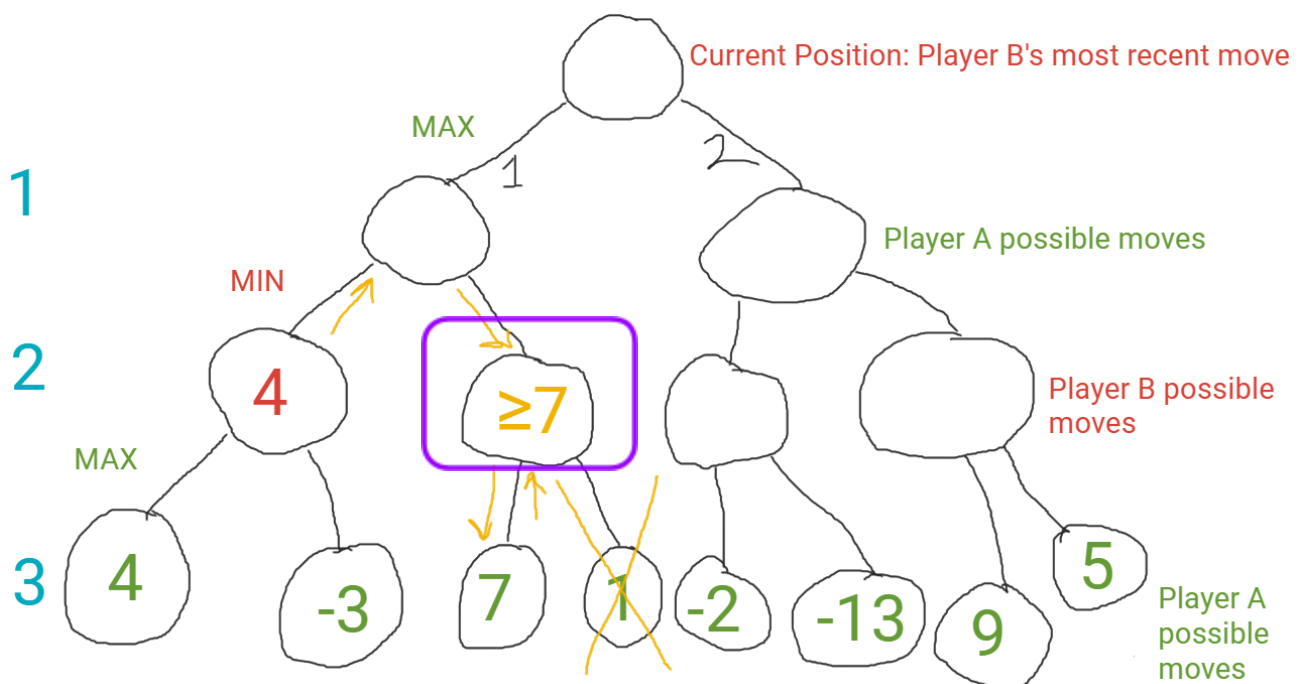


Now it's back to player A's turn and as always, its objective is to maximise the evaluation function. As $4 > -2$, the optimal move player A should make, given the current board, is the one which causes the board to transform into 'position 1'.

Optimising Minimax using Alpha-Beta Pruning

Limitations of ONLY using minimax: In reality, there are hundreds of available moves to make in chess for each position of the board, and each one of these hundreds would lead to many more and so on. Therefore it is impractical for our chess A.I to traverse every branch of the minimax tree as the time and space complexity would increase exponentially as the depth increases.

Alpha-beta pruning significantly reduces the computation time of the minimax algorithm, by cutting off branches in the game tree that do not require to be searched because there already exists a better move available. This allows us to strengthen the minimax algorithm, by increasing the depth we observe without causing any harsh time penalties.



To show alpha-beta pruning in application, we can rewind our minimax tree to when the A.I has stored the maximum value of 4 inside one of the sibling nodes. It then traverses to the other sibling node (boxed in purple), and accesses its child node with the value of 7 stored inside of it. As a result of this, the remaining child nodes (in this case the node which stores 1) do NOT need to be accessed. This is because the sibling node would absolutely store a value greater or equal to 7, as player A wants to maximise the evaluation score. Therefore, the minimum value player B has available to him/her will be the node that stores 4. Consequently, the purple node branch can safely be scrapped or 'pruned' with the chance of missing out on any better moves being impossible.

Maximising time complexity efficiency of alpha-beta pruning

The time complexity of the minimax algorithm massively varies, depending on the order the nodes (board positions) are to be searched. For instance, with the previous diagram, if we swapped the positions with the values '7' and '1', the algorithm would have no way of telling that the maximum number would be greater than 4, so the other value could not have been pruned resulting in more computation.

The time complexity ONLY using the minimax algorithm is $O(b^d)$ where:

- b = branching factor: the average number of child nodes per parent node, which in chess terms is the number of moves available from a specific position (average branching factor in chess is 35)
- d = depth: the number of moves ahead the minimax algorithm is planning on evaluating (the example shown before was using a depth of 3)

The worst case scenario for time complexity using minimax alongside alpha-beta pruning is also $O(b^d)$, in which all nodes have to be examined just as the original minimax algorithm does. This would happen if the moves that lead to more preferential positions were examined last. Inversely, the minimax function would operate at its fastest if the positions with the highest evaluation functions were observed first, which would lead to the most pruning. This would provide a time complexity of $O(b^{d/2})$, meaning that theoretically, the minimax algorithm could search at twice the depth whilst producing the same amount of computation, thanks to alpha-beta pruning.

Move ordering: The trick to mimicking the ideal case for the alpha-beta search is to sort the nodes so that the moves that lead to more promising results are analysed first, causing bad moves to quickly be eliminated. Due to the fact that it is impossible to tell what moves are best without actually performing a search on them, this method has to be guided by heuristics (approaches that produce estimations as solutions). Existing chess algorithms conduct shallow searches to get a general idea of how good the move is likely to be. For example, pawn promotions and captures are more likely to be strong moves, so these are ordered in the minimax tree so that it performs deep searches on these positions first.

Managing Game Difficulty Levels

Existing chess solutions usually tune the settings of their minimax algorithms and evaluation functions in order to vary the difficulty of the A.I of the game. There are 2 main ways to diversify the gameplay of the A.I, and both methods involve manipulating it to deliberately make incompetent decisions.

The first method further elaborates on the solutions suggested by Jacob and Jake: that better players consider positions further into the future. Computationally, this can be translated so that the minimax function only searches up to a predefined depth, depending on the difficulty set. For instance, for a beginner difficulty minimax employs a depth of 1, whereas a much higher depth is used for an expert difficulty.

A more creative approach involves tweaking the values of each material piece so that the evaluation function scores are distorted. The standard material values are: (king = 900, queen = 90, rook = 50, bishop/knight = 30, pawn = 10). As these values are truly accurate representations for the game, using them would produce accurate evaluation scores, so higher difficulty chess A.I's use this particular material table. However, a beginner's difficulty level could change some of these values (e.g. queen = 20, pawn = 30) so that the evaluation function creates misleading scores, tricking the A.I to choose a worse move. In this case, the A.I would wrongly exchange queens for pawns as the material table has defined it so that the pawn is a more valuable piece (obviously this an extreme case and would force unrealistic moves, so the tweaks to the material set would have to be less drastic).

Discussion with my Third-Parties

Questions for my Third-Parties

I have emailed each set stakeholders a different set of questions depending on the demographic they are representing; Humza and Seb are beginner players thus they are receiving questions based around what novices would be interested for in a chess game, whereas Jacob and Jake's responses to the questions given to them should help me form a greater understanding on the more technical aspects of the game, particularly in developing the chess A.I opponent.

Beginner demographic

My questions for Humza and Seb are as follows:

1. What parameters would you like to control before beginning your game?
 2. Would a timer be beneficial for your playing experience?
 3. If yes, what ranges of time would you feel would be appropriate for yourself?
 4. What additional features do you think could be implemented whilst playing in order to potentially accelerate your learning in chess?
 5. Do you have anything else to add?
- Question 1 allows me to decide what variable components should be implemented into my program when designing the main menu.
 - Questions 2 and 3 allow me to grasp a better understanding regarding novices' thoughts on whether having a timer is a good idea or not, as well as how it should be implemented if it were to be desired.
 - Question 4 informs me of any bonus functionality which may be useful for beginner players.
 - Question 5 asks if they have anything extra to say which I may have missed to ask about.

Technical advice

My questions to Jake and Jacob are as follows:

1. Which aspect of chess A.I would you expect to be the most challenging implementing computationally? (evaluating a chess position/searching for the best move/game difficulty)
 2. What difference in mindset is there between beginner players and experienced players?
 3. Do you have anything else to add?
- Question 1 enables me to forecast which element I should expect to have the most problems with when creating the A.I. Therefore, I could prioritise my time accordingly on the most difficult section.
 - Question 2 is probably the most significant question, as I am primarily relying on these responses in order to craft a better understanding of how I could vary the difficulties for the A.I of my game, as I found very little information whilst researching.
 - Question 3 asks if they have anything extra to say which I may have missed to ask about.

The Responses

Beginner Demographic

1. What parameters would you like to control before beginning your game?

- Humza: *'I think having the ability to choose which colour piece I could go before beginning the match would be a nice idea, as well as of course choosing the difficulty of the opponent against me.'*
- Seb: *'Well, I would like to change if possible the difficulty of the A.I as I gradually develop my skills. I am thinking maybe you should add 3 levels of easy, medium and hard difficulty?'*

2. Would a timer be beneficial for your playing experience?

- Humza: *'Honestly, I think I would prefer it if it weren't to be included. This is because I would want to prioritise my actual thinking skills rather than my time skills.'*
- Seb: *'I'm not too concerned about this though having a timer may annoy me more than anything.'*

3. If yes, what ranges of time would you feel would be appropriate for yourself?

- Both stakeholders chose no.

4. What additional features do you think could be implemented whilst playing in order to potentially accelerate your learning in chess?

- Humza: *'I am not sure about the practicality of implementing this, but I was thinking that if there was a button that when clicked it would show me the best possible move available to me, that would be great!'*
- Seb: *'Some sort of live feedback to me would be good I guess. For instance, at the end of each game the program could create a performance review of how well I played?'*

5. Do you have anything else to add?

- Humza: *'An extension could be to perhaps add some customization for the program?'*
- Seb: *'Nothing else. Good luck!'*

Technical advice

1. Which aspect of chess A.I would you expect to be the most challenging implementing computationally? (evaluating a chess position/searching for the best move/game difficulty)

- Jacob: *'Probably evaluating a chess position. This is because you have to compare the formation of the pieces as well as the material count. There are many components to consider when taking account of the formation such as: is the king in a safe spot? etc. Furthermore, you also will need to consider the stage of the game, as for instance an unsafe king is much more important in the middle of the game as opposed to at the end. Overall, there is a lot of detail you need to explore when deciding how good a chess position is for a player.'*
- Jake: *'I would say between tuning the game difficulty and evaluating a chess position, but I'm probably leaning more towards game difficulty. Mimicking the behaviour and mistakes of a bad human player into a programmable A.I in a realistic manner doesn't sound very easy!'*

2. What difference in mindset is there between beginner players and experienced players?

- Jacob: *'The biggest difference is probably that more advanced players look into the future when calculating their moves, whereas beginners do not. When they complete their move, they have already considered the potential ways to how the opposition will respond. Another difference is that better players usually have much better positional awareness. than beginners.'*
- Jake: *'Good chess players are much more likely to think 3 or 4 moves ahead, whereas novices may only look 1 move into the future, or not even at all. Also, experienced players tend to have a better conceptual knowledge of the game, their moves are calculated as opposed to being random, and have studied the openings of chess.'*

3. Do you have anything else to add?

- Jacob: *'I don't think so. I think we have covered everything to talk about.'*
- Jake: *'Nothing else to say other than good luck!'*

My thoughts on the Responses

Beginner demographic: I was actually really surprised that neither Humza nor Seb were enthusiastic in regards to having a timer in the program. Anyways, I decided that the timer will not be in my plans. I am very on board with both ideas suggested in order to improve the functionality of the game: the best-move pointer as well as the post-game report are definitely plausible ideas for implementation. Unfortunately I disagree with Humza's desire for customization as I feel like designing a robust and working playing environment should possess a higher priority instead.

Technical advice: Both stakeholders acknowledge that building the evaluation function will be a challenging task, so I will provide a significant chunk of my time and resources on this. Both Jacob and Jake also agree on the fact that the primary difference between a good and bad chess player is the number of moves they analyse ahead in the future. One way I could mimic this is by altering the depth of the minimax function within each level of difficulty.

My Proposed Objectives

1. Display menu: users need to apply the appropriate settings before playing

- 1.1. Choose the difficulty of the opponent A.I
 - 1.1.1. Buttons labelled 'Beginner', 'Intermediate' and 'Expert' need to be displayed
 - 1.1.2. When clicked, data regarding the difficulty chosen needs to be stored for further use as a variable
 - 1.1.3. If the user decides to change difficulty, the variable mentioned in 1.1.2 needs to be updated to the new chosen difficulty
- 1.2. Choose which colour piece to use
 - 1.2.1. White and black colours need to be displayed
 - 1.2.2. When clicked, data regarding the colour chosen needs to be stored for further use as a variable
 - 1.2.3. If the user decides to change colour, the variable mentioned in 1.2.2 needs to be updated to the new chosen colour
- 1.3. A 'Start' button needs to be displayed
 - 1.3.1. When clicked, a presence check regarding the information of 1.1.2 and 1.2.2 needs to be conducted
 - 1.3.2. If information is present, conceal menu and begin game

2. Creating the chessboard and pieces

- 2.1. Draw and display the board to the user using PyGame
 - 2.1.1. Create a chequered pattern
 - 2.1.2. Display labels 'a-h' at the end of each column, representing each file of the board
 - 2.1.3. Display labels '1-8' at the end of each row, representing each rank of the board
 - 2.1.4. Ensure that the labels when displayed is in the perspective of the colour chosen in 1.2
- 2.2. Create and display the pieces (in their starting position) to the user using PyGame
 - 2.2.1. Import images for each piece in both colours
 - 2.2.2. Ensure that the pieces of the user are displayed at the bottom of the screen
 - 2.2.3. Position the pieces so that they are in their starting positions
- 2.3. Convert the physical chessboard into a digital one
 - 2.3.1. Contents in the starting chessboard should be stored as a 2-D 8x8 array, used for later analysis
 - 2.3.2. Each chess piece should be given its own identity e.g. pawn = 1, bishop = 2, empty square = 0

3. Define the characteristics and behaviour for each chess piece using Object-Oriented Programming

- 3.1. Define movement patterns for each piece
- 3.2. Use the information from 1.1 in order to define the material value for each piece
- 3.3. Special moves such as castling for rooks and kings, or promotion for pawns should be available for the user to do
- 3.4. Validate movement actions by checking for checks

4. Updating/maintaining the chessboard

- 4.1. When a square containing a piece owned by the user is clicked, all of the valid squares the piece can move should be displayed to the user
 - 4.1.1. Initiate 4.2 if the user afterwards clicks on one of these valid destination squares
- 4.2. Appropriate changes should be made when making a move
 - 4.2.1. The chessboard needs to be updated for the user so that piece should be in its new position
 - 4.2.2. Gamedata, including information discussed in objective 5, need to be inspected and updated if necessary
- 4.3. Implement an undo function, which reverts all changes made in 4.2, for use in backtracking during minimax function
 - 4.3.1. OPTIONAL: Add a functionality which allows the user to undo a move
- 4.4. Appropriate changes should be made when the 'hint' function is initialised
 - 4.4.1. The minimax function should be called in perspective of the user
 - 4.4.2. The suggested move for the user stored as a tuple mentioned in 7.6.1 should be interpreted and implemented onto the board

5. Specific states of the game needs to constantly be inspected

- 5.1. Appropriate changes should be made when a king is in check
 - 5.1.1. The program should firstly examine when the king is in a checked state
 - 5.1.2. If a king is checked, restrict moves from the user that do not address the king being checked (only moves such as moving the king or blocking or capturing the attacking piece should be allowed)
- 5.2. Appropriate changes should be made when a player is in checkmate
- 5.3. Appropriate changes should be made when a player is in stalemate
- 5.4. Fetch the phase of the game at any current time
 - 5.4.1. The phase of the game should be stored as one of the following 3 strings: 'opening', 'middlegame', 'endgame'

6. Creating the evaluation function for the chess A.I to evaluate positions of the board

- 6.1. A numerical score for material count should be stored as a variable
- 6.2. A numerical score for pawn structure should be stored as a variable
- 6.3. A numerical score for centre control should be stored as a variable
- 6.4. A numerical score for king safety should be stored as a variable
- 6.5. The evaluation function should use the game phase argument in order to determine the weight used on each factor mentioned in 6.3, when calculating the overall score of the position.

7. Creating the minimax tree search algorithm so the chess A.I can search the best move

- 7.1. Parameters such as depth, the current position and the maximising player should be passed into the algorithm
 - 7.1.1. The difficulty referred in 1.1 should dictate the value of the depth passed into the function e.g beginner = low depth value
 - 7.1.2. The colour referred in 1.2 should dictate the identity of the player trying to maximise the function (e.g if colour chosen by the user is white, the maximising player is black because the A.I is controlling the black pieces)
- 7.2. All of the possible moves available from the current position should be compiled and stored as child nodes from the root node (the current board)
- 7.3. Recursion should be applied so that every position of the board is searched up until the given depth
- 7.4. The evaluation function should be called for each of the terminal nodes
- 7.5. Tree traversal should be utilised in order to compare each node and its score referred in 7.4.1 with the current best score
- 7.6. The most optimal move for the A.I should be returned when minimax is called, represented as a tuple

8. Implementing Alpha-Beta pruning into minimax to maximise time efficiency for the A.I

- 8.1. Move ordering should be used to ensure pruning is more reliable
 - 8.1.1. Heuristics should be used to determine the best likely moves, and these moves should be prioritised to be searched by minimax first
- 8.2. Branches should be pruned if there is no point in traversing the rest of it
 - 8.2.1. The program must ensure that there is no risk of there being any 'significant' moves present in the remaining nodes that are not being searched due to pruning

9. Generate the most suitable move for the A.I by using everything so far

- 9.1. The end goal is for the A.I to generate any move at any time with depth ≥ 4 (program looks at least 4 moves ahead into the future), within a 5 second time window
- 9.2. The move should be represented by stating the starting square and the ending square, as a tuple

10. Generate a post-game report for the user

- 10.1. A suitable algorithm should be devised in order to evaluate the suitability of each move made by the user and the user only
- 10.2. The winner of the game (if there was one), and the number of moves completed should be displayed by the user
- 10.3. The number of blunders and good moves etc. should be displayed to the user

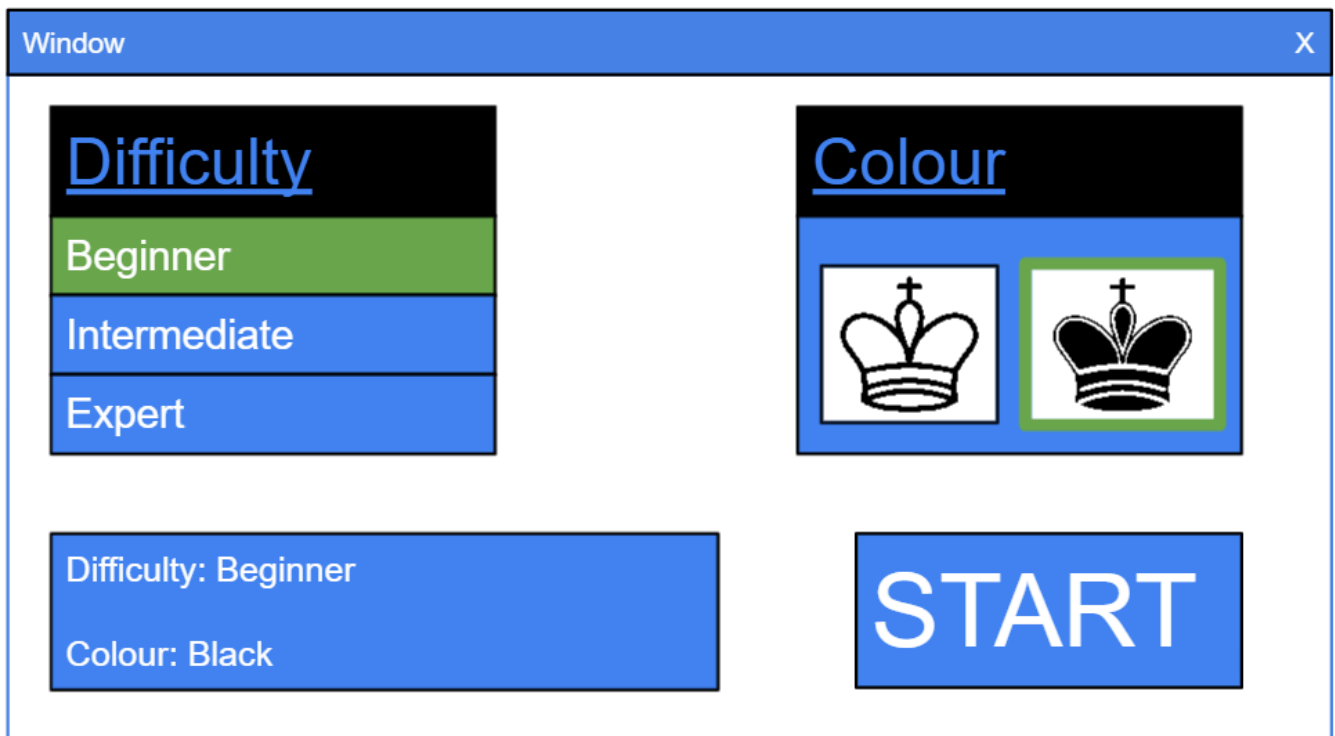
Documented Design

User-Interface Design

Main Menu Design

Links to objectives:

- **1. Display menu: users need to apply the appropriate settings before playing**



This is the GUI design for the main menu of the chess game, which is the first item displayed when the user runs the program. This allows the user to adjust the key settings before playing the game, as well as a 'start' button when they are ready to play.

Difficulty button (1.1)

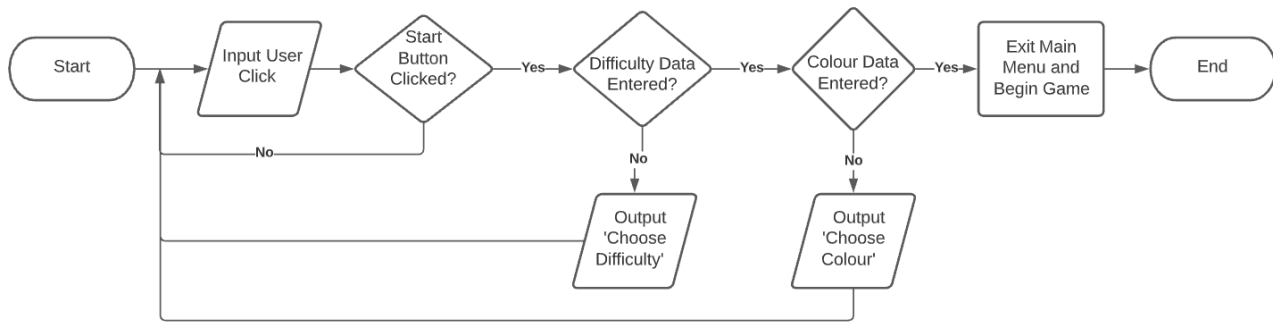
The user will choose from the 3 difficulties of A.I available which are displayed from above (1.1.1). The difficulty chosen will be highlighted (in the case above, the 'beginner' difficulty has been chosen and highlighted in green) and stored as a string for future use (1.1.2). If the difficulty is updated the previous one will no longer be highlighted and the variable which holds the difficulty chosen is updated (1.1.3).

Colour button (1.2)

In chess the user can either choose to be white or black with the pieces displayed above (1.2.1). This affects who goes first in a game. Similar to the difficulty button functionality, there is a variable which stores the colour the user chooses (1.2.2) which is updated the same way (1.2.3)

Start button (1.3)

When the 'start' button is clicked by the user the program needs to ensure that both data regarding difficulty and colour has been entered (1.3.1). Below is a flowchart illustrating an abstraction of the process. (1.3.2 and 1.3.3)

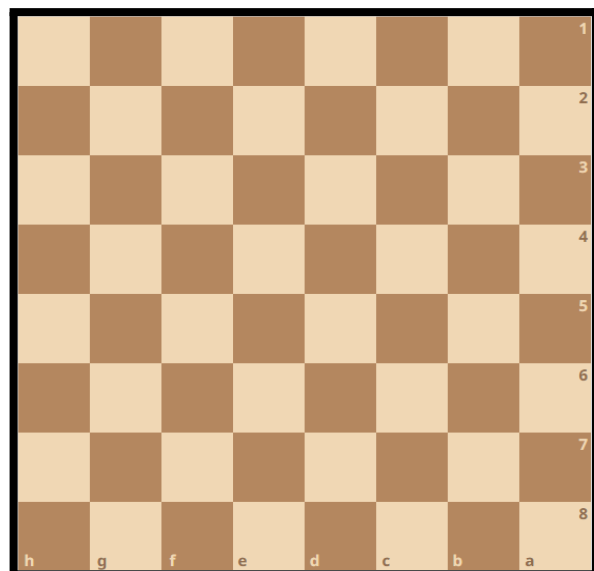
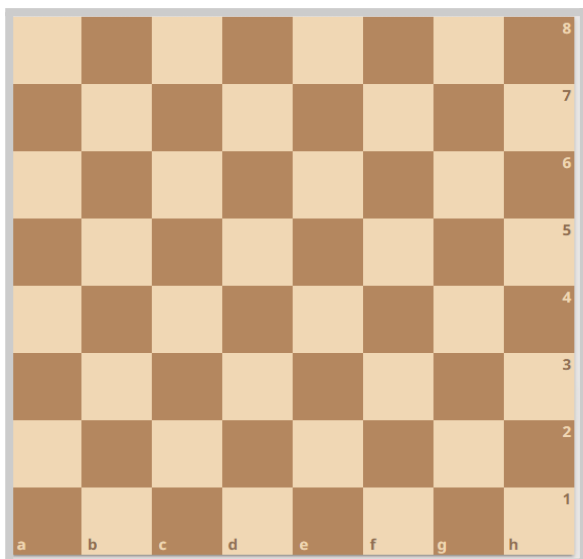


Chessboard and Chess Pieces Design

Links to objectives:

- **2. Creating the chessboard and pieces**

Chess board design (2.1)



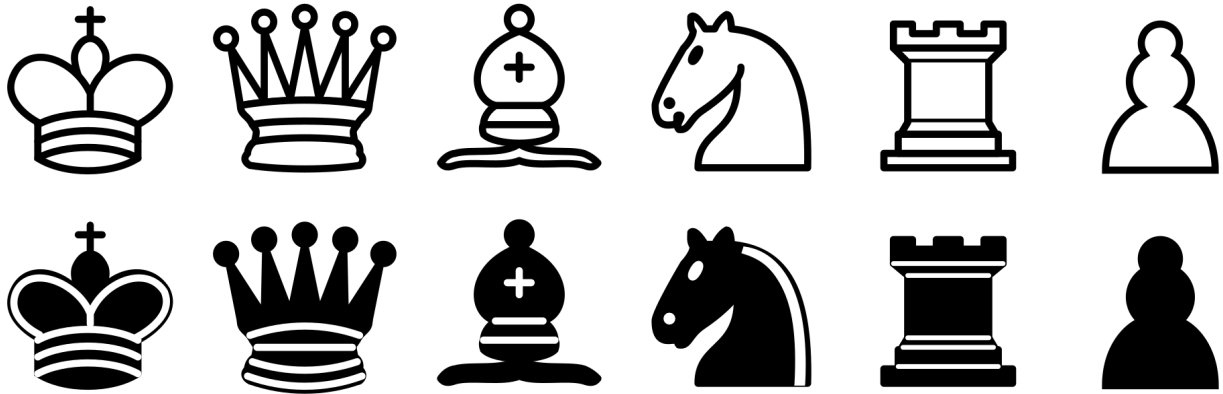
(2.1.1) The algorithm to design the checker pattern so that it looks like the images above will look similar to the pseudocode below. Displaying the labels on the sides for the files should prove to be an easier task and can be done solely using the in-built features in PyGame. (2.1.2 and 2.1.3)

```

SUBROUTINE
create_chessboard_pattern(colour_1, colour_2)
  fill screen colour_1
  cell_number ← 8
  FOR row ← 1 TO cell_number
    FOR col ← 1 TO cell_number
      IF ((row MOD 2) = 0 AND (col MOD 2) = 0) OR ((row MOD 2) = 1 AND (col MOD 2) = 1) THEN
        fill cell colour_2
    END FOR
  END FOR
ENDSUBROUTINE
  
```

(2.1.4) The image on the left on the previous page is a chessboard in the perspective of white, whereas the image on the right is in black's perspective. The only but key difference is the fact that the board is inverted for black, meaning that its pieces begin on the 7th and 8th rank in contrast to the 1st and 2nd rank for white, and that leftmost file for black is the 'h' file compared to the 'a' file for white (this is the same as if you were playing in person).

Piece design using dictionaries (2.2)



These are the graphics of the chess pieces that will be imported into the PyGame for use (2.2.1), and will all be stored in the data structure of a dictionary. For instance, let `IMAGES = {data}`, where each value of `data` consists of the name of a chess piece (key) mapped to the corresponding png image of itself (value). Therefore, accessing the png image of a black rook (abbreviation = 'bR') can be achieved by the call `IMAGES['bR']`.

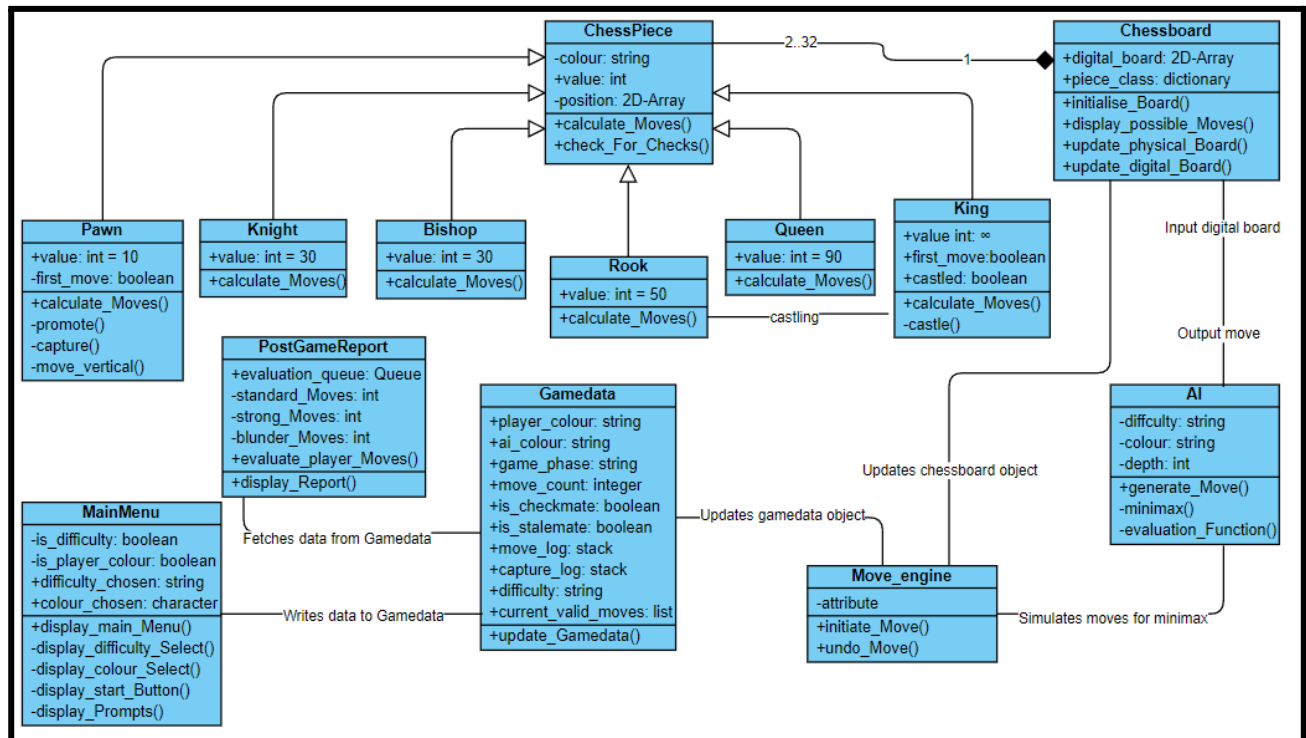
As demonstrated above, dictionaries are relatively easy to perceive, improving the readability of my code. This will make analysing and debugging my program if needed a simpler and quicker process.

Creating the Game Functionalities

Links to objectives:

- **3. Define the characteristics and behaviour for each chess piece using Object-Oriented Programming**
- **4. Updating/maintaining the chessboard**
- **5. Specific states of the game needs to constantly be inspected**

The OOP approach



Why OOP?

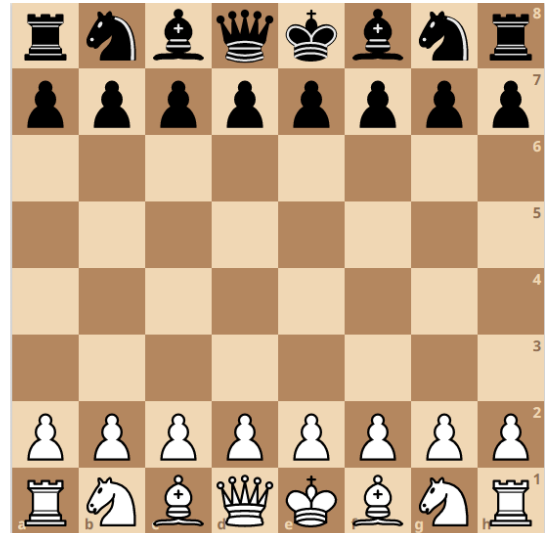
The pieces are one of the most defining attributes of a chess game. Although they are unique in their own ways, there is a lot of similarity between them. For instance, all chess pieces consist of a colour and value, and all are positioned on the same board. Instead of re-defining these identical attributes and methods for each piece, object-oriented programming allows me to create a generic class (in this case the `ChessPiece` class) that defines all of the overlapping properties once, so that the other objects can inherit these properties. Inheritance reduces the complexity of my chess game by eliminating repeated code.

OOP also provides the characteristic of polymorphism, meaning that different objects respond in their own way when being called to carry out the same instruction. For example, the `'calculate_Moves'` method in `Pawn.calculate_Moves()` behaves differently than the `'move'` method in `Queen.calculate_Moves()` even though both method names are identical to each other.

Digitalising the chessboard(2.3)

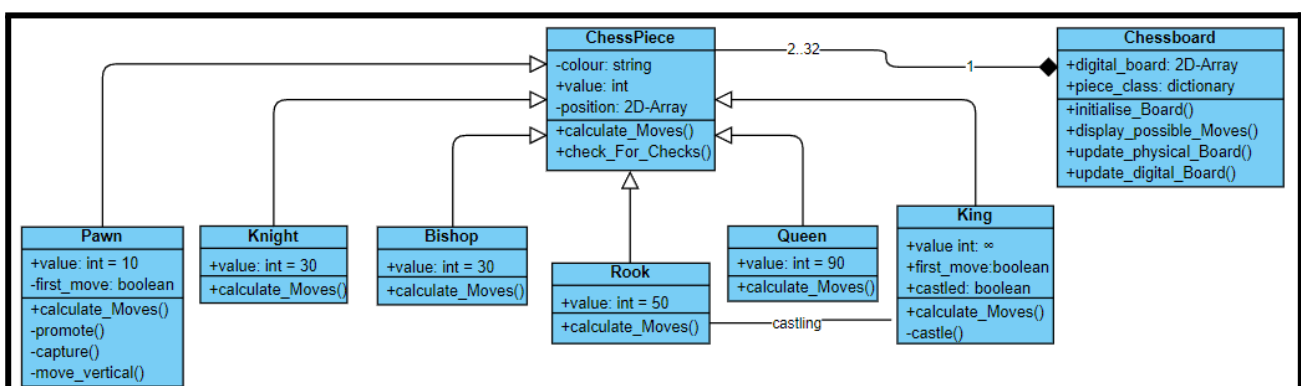
There needs to be a way of representing the contents of the board digitally, not only so that the program can check specific states of the game (checks ,stalemates etc.) but so that this digital board can be passed into the A.I algorithm for it to deduce and return back a 'suitable' move (will be elaborated in further detail later). The board can be viewed as an object so I decided that it should be designed in an object oriented-way. The attribute 'digital_board' will be stored as an 8x8 2-dimensional array so the contents can easily be manipulated and updated (2.3.1), and this attribute will be publicised because it needs to be called very commonly within the rest of the program.

```
digital_board =
['br', 'bn', 'bb', 'bq', 'bK', 'bb', 'bn', 'br']
['bp', 'bp', 'bp', 'bp', 'bp', 'bp', 'bp', 'bp']
['00', '00', '00', '00', '00', '00', '00', '00']
['00', '00', '00', '00', '00', '00', '00', '00']
['00', '00', '00', '00', '00', '00', '00', '00']
['00', '00', '00', '00', '00', '00', '00', '00']
['wp', 'wp', 'wp', 'wp', 'wp', 'wp', 'wp', 'wp']
['wr', 'wn', 'wb', 'wq', 'wk', 'wb', 'wn', 'wr']
```



The image above is a representation of the initial positioning of the pieces when the game begins, as an 8x8 2D-array that will be labelled as 'digital_board'. The prefix of each data represents the colour of the piece ('w' = white, 'b' = black), which is followed by the functionality of the piece itself ('p' = pawn, 'n' = knight, 'b' = bishop, 'r' = rook, 'q' = queen, 'k' = king). The data '00' represents an empty square (2.3.2). Due to the nature of 2D arrays in python, it must be vertically inverted for it to match a chessboard. For instance the bottom row of 'digital_board' (digital_board[7]), represents the 1st rank when seen on a physical chess board.

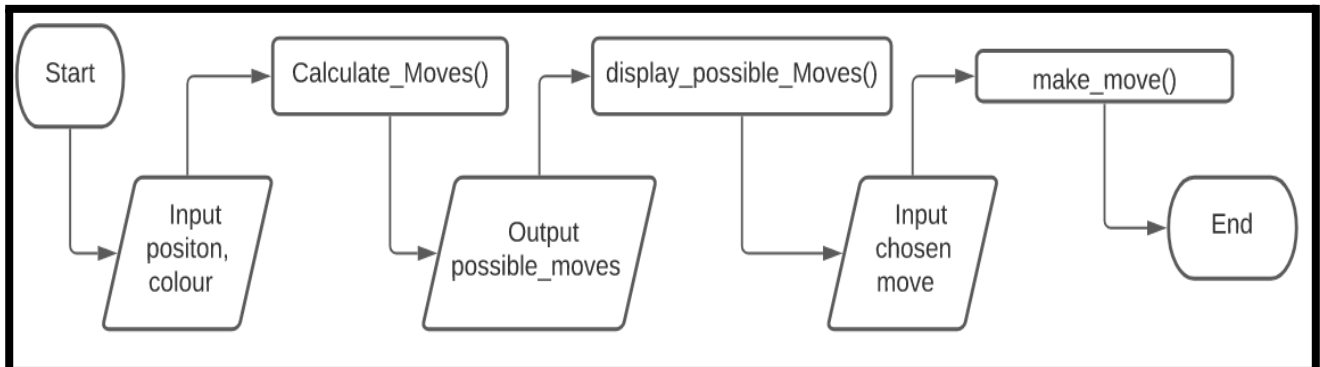
Relationship between pieces



This is a magnified representation of my class diagram, displaying the connections between the chess pieces and chessboard. All chess pieces inherit the properties of the 'ChessPiece' superclass. There is a composition relationship between 'Chessboard' and 'ChessPiece' as the chess pieces cannot operate without being on a board. One chessboard contains between 2 and 32 chess pieces at all times. The 'colour' and 'position' attributes will determine the valid moves a chess piece can make when passed into the 'calculate_Moves()' method, whereas the 'value' of the piece will be a component considered when designing the evaluation function (3.2)

Defining Movement Patterns of Chess Pieces

The diagram below demonstrates an abstraction of the process when a user intends to make a move by clicking on a square. The 'calculate_Moves' method is called and returns an array of the available moves, which are then displayed onto the physical chessboard by highlighting the appropriate squares. When any of the highlighted squares are clicked, the move is executed.



Lets zoom in on the 'Calculate_Moves' method, first defined in the 'ChessPiece' superclass. As each chess piece has varying functionality this method will need to be overridden in order to tailor their different ways of movement, using the polymorphism characteristic provided by OOP. As well as the position of the current piece (chosen by the user's click), the colour of the piece also needs to be passed into the method in order to identify potential obstructions (further elaboration later). Below are the pseudocode algorithms for the 'Calculate_Moves' method, redefined for each chess piece sub-class. (3.1)

The superclass Calculate Moves() method

```

SUBROUTINE
calculate_moves(board,vectors,position,colour,isRange)
  valid_moves ← []
  row,col ← position

  FOR vector IN vectors
    new_row ← row + vector[0]
    new_col ← col + vector[1]

    IF isRange = false THEN # for the king and knight
      IF (new_row >= 0) AND (new_row <= 7) AND (new_col >= 0) AND (new_col <= 7) THEN
        IF board[new_row][new_col].colour ≠ colour THEN
          # ensures that the new_position isn't already occupied by a piece of the same colour
          add to valid_moves
        ENDIF
      ENDIF

    ELSE IF isRange = True THEN # for the bishop, rook and queen
      WHILE (new_row >= 0) AND (new_row <= 7) AND (new_col >= 0) AND (new_col <= 7)
        IF board[new_row][new_col].colour ≠ none THEN #if not an empty square
          IF board[new_row][new_col].colour ≠ colour THEN #if it is an enemy piece
            add to valid_moves
          ENDIF
          break #look no further in this direction
        ENDIF

        add to valid_moves
        new_row ← new_row + row_vector
        new_col ← new_col + col_vector
      ENDWHILE
    ENDIF

  ENDFOR
  return valid_moves
ENDSUBROUTINE

```

This is my initial prototype of the 'Calculate_Moves' supermethod that will, in some shape or form, be inherited and modified by all of its subclass pieces (except from the pawn subclass, which will undergo a complete overhaul in its 'Calculate_Moves' method). In general, the operation 'board[row][col].colour' retrieves the colour of the piece at the square [row,col] (white/black/none).

The method will be given 5 key parameters: Each piece has a unique way in which they move which can be expressed as a list of **vectors**. Each vector within the list will be immutable so I decided that a tuple was the best data type to represent them. The current position the piece is at will also be passed in the function, which contains 2 integers between 0-7 to represent the row and column on the chessboard it is currently at. Due to the fact that the position of the piece is likely to change within the rest of the program, storing these

values in the form of a list is suitable. The colour of the piece is also a necessity as it determines whether a piece can be captured or not (it's impossible to go to a square that a piece of the same colour is already at). `isRange` is a boolean value that determines whether or not the vector specified are all of the possible positions a piece could travel to from its current square, or if set to 'true', can be considered as directions that can be traversed until either 'new_row' or 'new_col' are out of the board, or if there is a piece blocking that particular direction.

Bishop.Calculate_Moves()

For future reference, let the command `super.calculate_moves()` call the superclass method.

```
SUBROUTINE calculate_moves(board,position,colour) #argument of board will be 'digital_board'
  bishop_vectors ← [(1,1),(1,-1),(-1,-1),(-1,1)]
  valid_moves ← super.calculate_moves(board,bishop_vectors,position,colour,true)
  return valid_moves
ENDSUBROUTINE
```

A bishop can move diagonally in 4 different directions, in this case represented as `bishop_vectors`, expressing the directions bottom-right, bottom-left, top-left and top-right respectively. If we recall earlier in my program the first row in the array 'digital_board' represents the 8th rank of a physical chessboard. This is why 'new_row' needs to decrement in order to venture up the board (i.e a vector (-1,col) is upwards, and (1,col) is downwards).

As a bishop is a ranged piece, 'isRange' is initialised to `true`, meaning that that the program searches for 'valid_moves' continually using the same vector until either the square is outside the range of the 8x8 grid, or if the square is occupied by another piece (which blocks entry to squares further down the path). If the square occupied is an enemy piece it is still a valid move. On the first instance the program detects either of these two instances, it begins to search in another direction

Rook.Calculate_Moves()

```
SUBROUTINE calculate_moves(board,position,colour)
  rook_vectors ← [(1,0),(0,1),(-1,0),(0,-1)]
  valid_moves ← super.calculate_moves(board,rook_vectors,position,colour,true)
  return valid_moves
ENDSUBROUTINE
```

If you think about it, the operation of moving a rook is IDENTICAL to the operation of moving a bishop bar one exception: rooks move vertically and horizontally in contrast to the diagonal movement a bishop possesses. Programatically speaking however, this can be easily be tweaked by considering vectors; we will just input in `rook_vectors` as an argument instead to represent the movement of a rook. The list `rook_vectors` expresses the directions down, right, up and left respectively.

Queen.Calculate_Moves()

```

SUBROUTINE calculate_moves(board,position,colour)
  queen_vectors ← [(1,1),(1,-1),(-1,-1),(-1,1),(1,0),(0,1),(-1,0),(0,-1)]
  valid_moves ← super.calculate_moves(board,queen_vectors,position,colour,true)
  return valid_moves
ENDSUBROUTINE

```

In essence, the queen is just a bishop and a rook fused into one piece, so all available moves can be gathered for a queen by having `queen_vectors` by combining the vectors of a bishop and rook.

Knight.Calculate_Moves()

```

SUBROUTINE calculate_moves(board,position,colour)
  knight_vectors ← [(1,2),(2,1),(-1,2),(2,-1),(1,-2),(-2,1),(-1,-2),(-2,-1)]
  valid_moves ← super.calculate_moves(board,knight_vectors,position,colour,false)
  return valid_moves
ENDSUBROUTINE

```

As the knight is not a ranged piece we must declare the boolean 'isRange' as `false`. This in turn actually results in the knight being an easier piece to design due to the fact that we only need to determine if the square [new_row,new_col] does not contain a piece the same colour of the knight. Considering obstructions is not necessary.

King.Calculate_Moves()

```

SUBROUTINE calculate_moves(board,position,colour)
  king_vectors ← [(1,1),(1,-1),(-1,-1),(-1,1),(1,0),(0,1),(-1,0),(0,-1)]
  standard_moves ← super.calculate_moves(board,king_vectors,position,colour,false)
  castling_moves ← castle(colour,position,first_move)
  valid_moves ← standard_moves + castling_moves
  return valid_moves
ENDSUBROUTINE

```

This method is very similar to the method defined for the knight in the way we set 'isRange' to `false`. The only difference is the values of `king_vectors`.

King.castle() (3.3)

```

SUBROUTINE
castle(board,colour,position,first_move)
  castling_moves = []

  IF first_move = false #illegal to castle if false
    return castling_moves
  ENDIF
  row ← position[0] #only the row is required

  #castle queen side
  IF (board[row][1].colour = none) AND (board[row][2].colour = none) AND
    (board[row][3].colour = none) THEN
    #checks if the squares between the king and rook (queen side) are empty
    IF board[row][0] = colour + 'r' THEN #if rook is on edge is same colour as king
      add castle queen side to castling_moves
    ENDIF
  ENDIF

  #castle king side
  IF (board[row][5].colour = none) AND (board[row][6].colour = none) THEN
    #checks if the squares between the king and rook (king side) are empty
    IF board[row][7] = colour + 'r' THEN #if rook is on edge is same colour as king
      add castle king side to castling_moves
    ENDIF
  ENDIF

  return castling_moves
ENDSUBROUTINE

```

The attribute `first_move` is instantiated for a king to ensure that it cannot castle if the king has already moved. Castling queen side moves the king 3 squares to the left, whereas castling king side involves moving the king 2 squares to the right. When castling the rook also repositions to the right/left of the king.

Pawn.Calculate_Moves()

```

SUBROUTINE
calculate_moves(board,position,colour,first_move)
  valid_moves ← []
  IF colour = white THEN #white pawns move up the board
    row_vector ← -1
  ELSE #black pawns move down the board
    row_vector ← 1
  ENDIF

  FOR col_vector ← -1 TO 1
    IF col_vector = 0 THEN #pawn is only moving vertically, no capturing
      pawn.move_vertical(board,position,row_vector,valid_moves,first_move)
    ELSE #pawn is moving diagonally, capturing
      pawn.capture(board,position,row_vector,valid_moves,colour,col_vector)
    ENDIF
  ENDFOR
  return valid_moves
ENDSUBROUTINE

```

As mentioned before, pawns are an outlier to the other pieces in so many ways that they require their own special method. For instance they are the only piece that capture differently to how they move. Therefore, all possible cases need to be considered separately by creating 2 additional methods for each instance. Furthermore, pawns are the only chess piece that are not bi-directional, which is why the first chunk of the pseudocode focuses on whether the pawn is moving up or down (this depends on the colour of the pawn).

Pawn.move_vertical()

```

SUBROUTINE
move_vertical(board,position,row_vector,valid_moves,first_move)
  row,col ← position
  IF board[row + row_vector][col].colour = none THEN #checks if square in front is empty
    add to valid_moves
  ELSE
    return
  ENDIF
  IF first_move = true THEN #the pawn is yet to move
    IF board[row + row_vector * 2][col].colour = none
      add to valid_moves
    ENDIF
  ENDIF
ENDSUBROUTINE

```

The code 'row_vector * 2' mimics the effect of a pawn moving 2 squares in front when performing its first move. The 'return' line is present in order to eliminate the need to search any further along the line, as the second square cannot be accessible if the first square is blocked a another piece

Pawn.capture()

SUBROUTINE

capture(board,position,row_vector,valid_moves,colour,col_vector)

row,col ← position

IF (col + col_vector >= 0) AND (col + col_vector <= 7) THEN

IF board[row+row_vector][col+col_vector] ≠ none) THEN

IF board[row+row_vector][col+col_vector] ≠ colour) THEN

#the 2 lines above checks if square immediately diagonal to pawn is an enemy piece

add to valid_moves

ENDIF

ENDIF

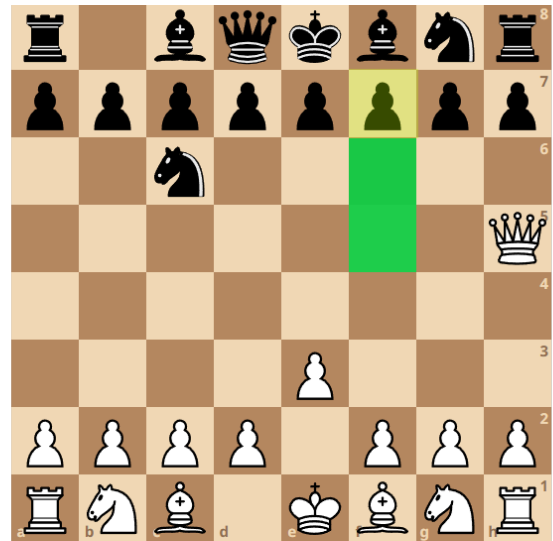
ENDIF

ENDSUBROUTINE

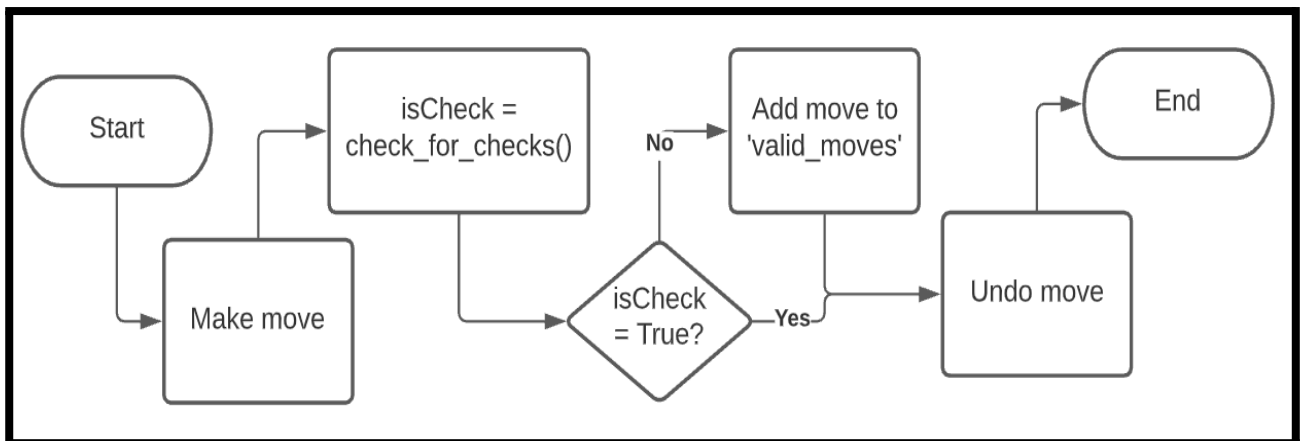
Identifying and Handling In-game States

You have probably been wondering so far - what about checks? And for that I do apologise, because I have lied: the list 'valid_moves' is not actually truly valid, but should be more accurately referred to as *potential* moves.

Consider the example to the right. If the 'f7' pawn was clicked by the user, in the current state of the algorithm `pawn.calculate_moves()`, it would judge the moves 'f6' and 'f5' as valid moves. Clearly these moves are illegal as it creates a discovered check by the queen on the black king. So we need to devise another algorithm to detect these potential checks.



The solution



We need to add another method that is called just before a potential move is added to the list 'valid_moves'. The idea is that we actually execute this potential move, and the updated 'digital_board' is passed into this method as a parameter. We will name this function 'check_for_checks' for now, and as hinted in its name, it will scan the 'digital_board' in order to identify if the king of the player who has made the previous move is under threat (i.e. can be captured within a single move). If 'check_for_check' returns 'True', then the king is under direct attack thus the potential move is invalid. Otherwise, the move is certainly valid.

Explaining the 'check_for_checks()' algorithm

A naive way of solving this problem would be to identify all of the pieces present on the 'digital_board', and for each piece, determine whether or not capturing the king is in their list of 'valid_moves'. However as you probably have noticed, this would cause the time complexity of my program to increase excessively and unnecessarily.

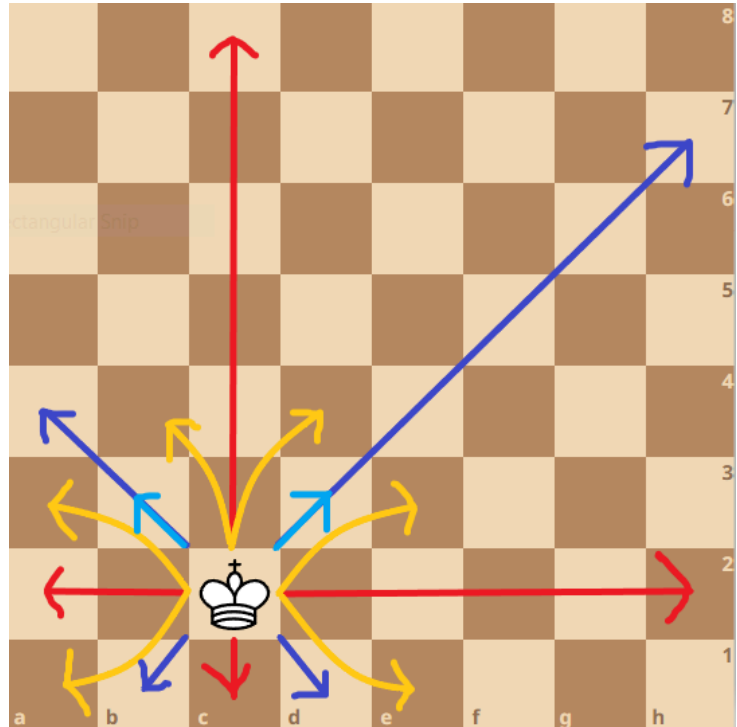
A less conventional but more efficient approach would be to consider the king as a superpiece, which can move horizontally, vertically, diagonally and in L-shaped patterns. The king traverses in each direction and, if it encounters a specific enemy piece along one of these lines, and the line of sight between such enemy piece and king is not impeded by another piece, this piece can capture the king, thus the function will return 'True'. If we take the example to the right we can understand which pieces specifically can capture the king along which lines as follows:

Light blue: Queen, bishop, pawn

Dark blue: Queen, bishop

Red: Queen, rook

Yellow: Knight



Let row,col = any position on an 8x8 chessboard. The method 'check_for_checks', with the parameters 'digital_board' and 'colour' (of king being investigated) will return 'True' (the king is under check) if either:

- digital_board[row][col] contains an enemy knight that is an L-shaped distance away from the king
- digital_board[row][col] contains an enemy queen or rook that is on the same file/rank from the king AND there are no pieces between the two pieces
- digital_board[row][col] contains an enemy queen, bishop or pawn that is IMMEDIATELY diagonal from the king (colour of the pawn will need to be considered)
- digital_board[row][col] contains an enemy queen or bishop that is diagonally positioned from the king AND there are no pieces between the two pieces

Otherwise, 'check_for_checks' will return 'False' and the potential move can be added to 'valid_moves'.

Storing key variables related to the state of the game inside the 'gamedata' classHandling Checks(5.1)

As displayed in the previous flowchart, a boolean attribute called 'isCheck' will need to be instantiated, initially to false (neither kings are in check).

As well as being used to check the validity of potential moves, there is another application for the 'check_for_checks' method: it can be used to check the status of both kings by tweaking the 'colour' parameter. For instance, imagine that black has chosen and executed its move: now the colour variable will be set to 'white' in order to check if the white king is under check. If 'True' is returned, the boolean 'isCheck' will also be assigned to 'True'. (5.1.1)

If at any time 'isCheck' is equal to 'True', the player whose turn it is must produce a move that accomplishes one of the following: (5.1.2)

1. Move its king to a safe square
2. Block the attacking piece (not applicable towards knights)
3. Capture the attacking piece

Fortunately all of these possibilities are covered within the 'calculate_moves' methods of the pieces abetted with the 'check_for_checks' function. When the player has responded to the check the 'isCheck' variable is resorted to 'False'.

Handling Checkmates(5.2)

A boolean variable called 'isCheckmate' will need to be instantiated, initially to 'false'.

Behind the scenes during a chess match the program will generate a list of every valid move a player has during its turn (labelled 'all_valid_moves'), using the 'calculate_moves' method of each piece on the board belonging to the player. If at any point during the match that 'isCheck' = True AND len(all_valid_moves) = 0, the variable 'isCheckmate' will be assigned to 'True', and the relevant changes will be made.

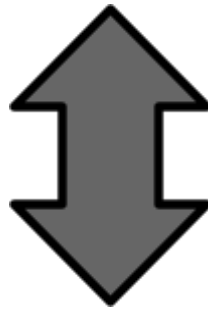
Handling Stalemates(5.3)

A boolean variable called 'isStalemate' will need to be instantiated, initially to 'false'.

If at any point during the match that 'isCheck' = False AND len(all_valid_moves) = 0, the variable 'isStalemate' will be assigned to 'True', and the relevant changes will be made.

Additional attributes inside 'gamedata' class

Attribute	Fetches from/when	Primary purpose
Difficulty	Main menu screen	Determines depth used, strength of evaluation function
ai_colour	Main menu screen	Determines when to make move on board
player_colour	Main menu screen	Determines when NOT to make a move on board
gamephase	Evaluated after every move	Determines weight of components for evaluation function
move_count	Incremented after every move	Used to calculate current gamephase
move_log_stack	Appended after every move	When popped, retrieves most recent move that can be re-traced when undoing a move
capture_log_stack	Appended after every capturing move	When popped, retrieves object of most recent piece captured, than can be re-added when undoing a move



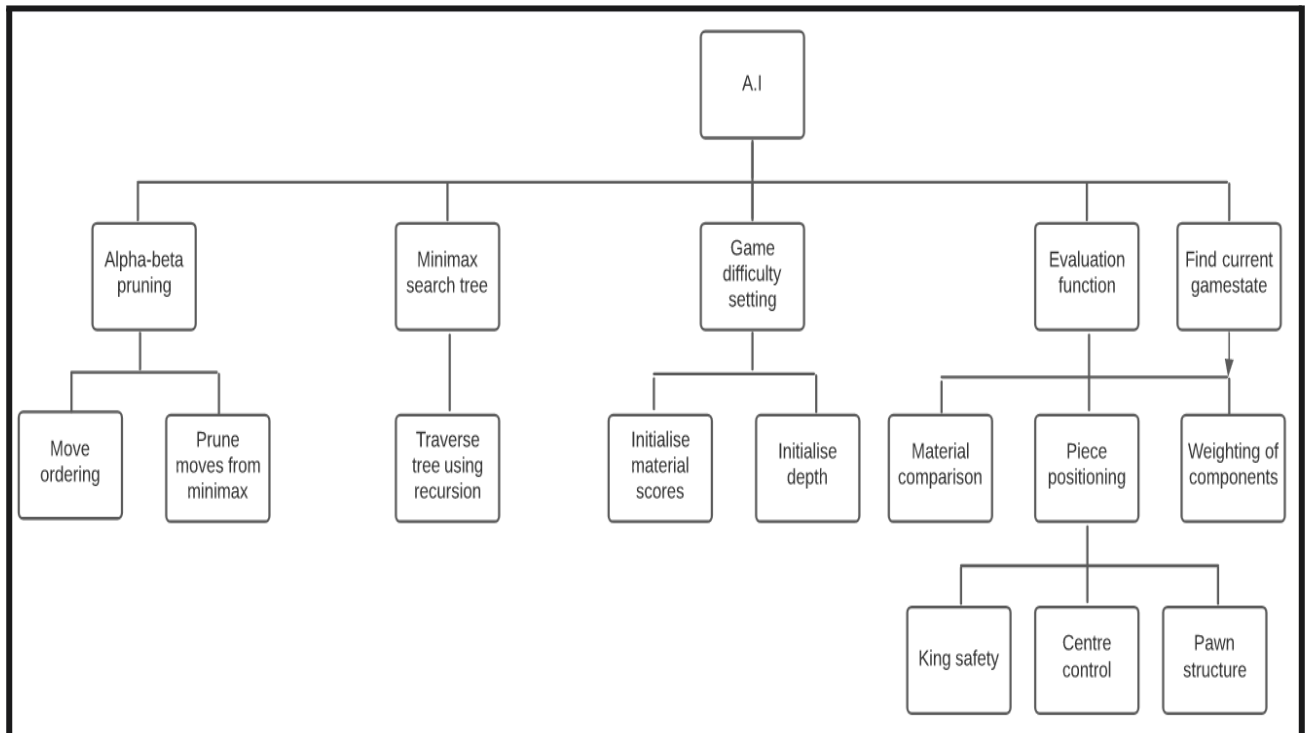
Gamedata
+player_colour: string
+ai_colour: string
+game_phase: string
+move_count: integer
+is_checkmate: boolean
+is_stalemate: boolean
+move_log: stack
+capture_log: stack
+difficulty: string
+update_Gamedata()

Creating the A.I Opponent

Links to objectives:

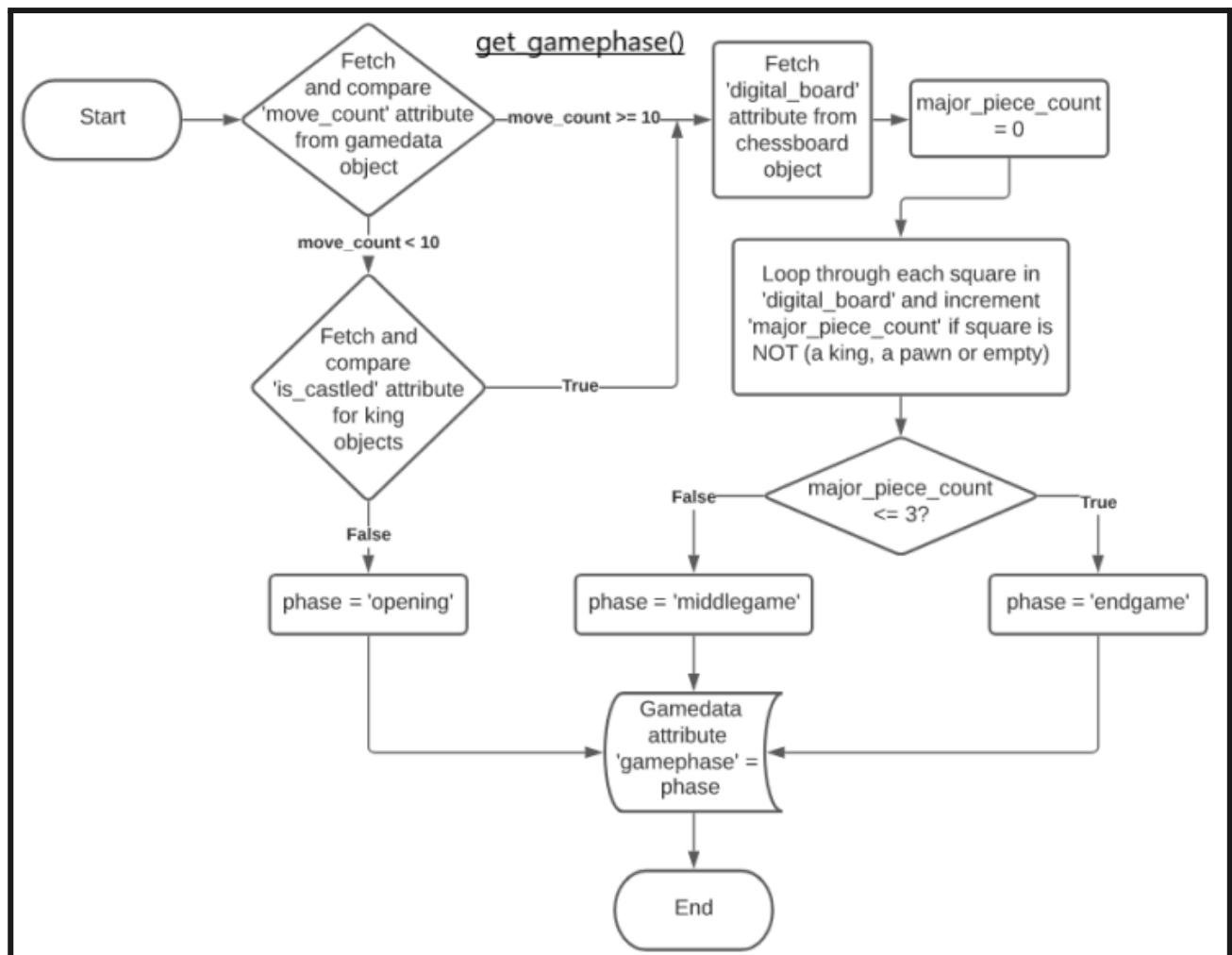
- **5. Specific states of the game needs to constantly be inspected**
- **6. Creating the evaluation function for the chess A.I to evaluate positions of the board**
- **7. Creating the minimax tree search algorithm so the chess A.I can find the best move**
- **8. Implementing Alpha-Beta pruning into minimax to maximise time efficiency for the A.I**

Hierarchy Chart (not 100% finalised)



As creating the A.I for my chess program will occupy the majority of my time and resources, I have decided that constructing a hierarchy chart is a great way to visualise how this can be decomposed into smaller, more manageable modules that can be designed and implemented separately.

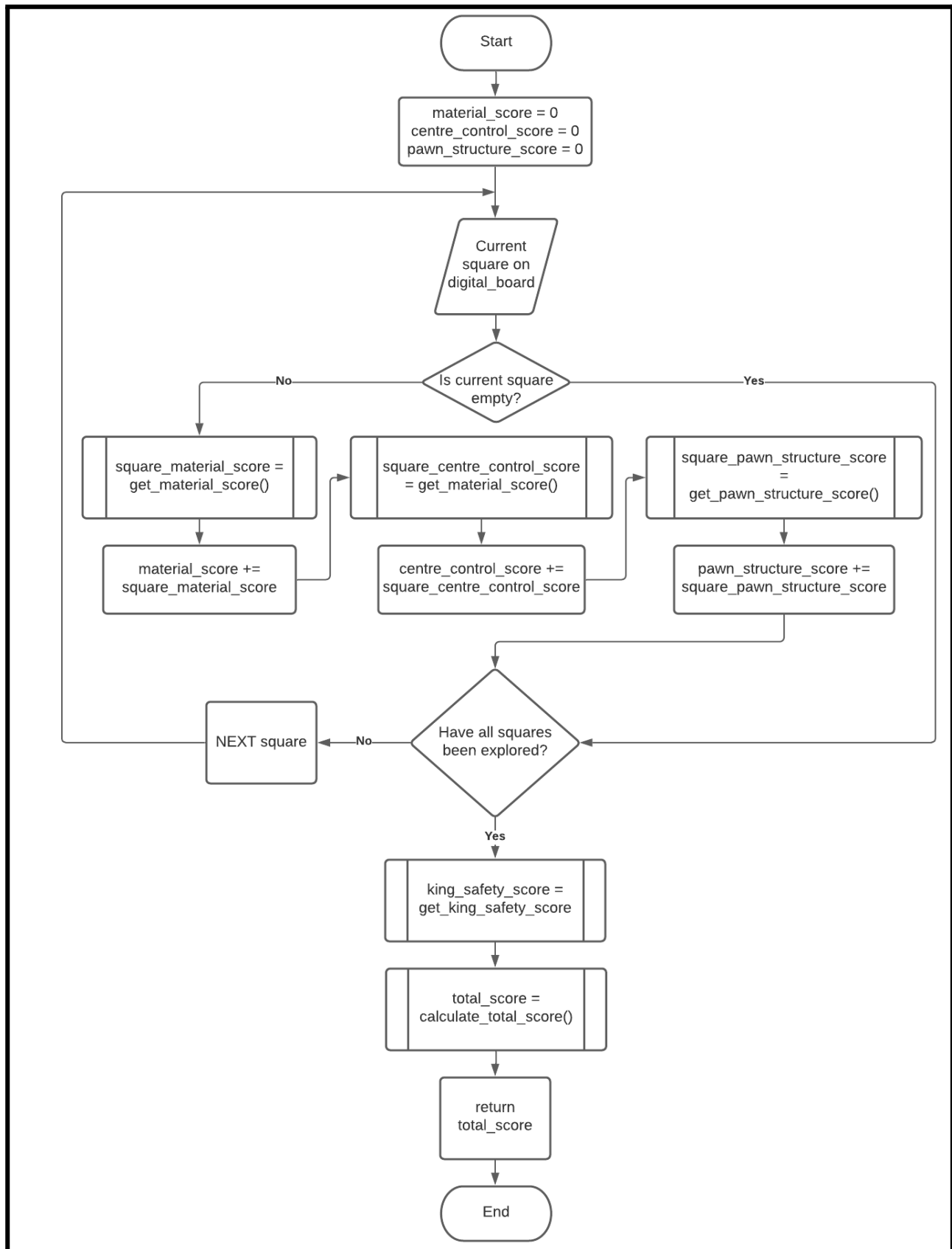
Calculating Current Gamephases



The flowchart above illustrates how I will obtain the current stage of the chess match at any time. A reminder that the three possible phases of the game 'opening', 'middlegame', 'endgame' determine the weights of the following components that will comprise the overall score of any given position:

- Material comparison
- Pawn structure
- Centre control
- King safety

(more detailed information regarding the specific weights of components on page 7)

Evaluation Function Design

Explanation

The flowchart above illustrates the sequence of instructions in order to obtain a floating value of the 'total_score' for the corresponding inputted 'digital_board'. The main concepts of this algorithm works as follows:

- *Iterate through each square of the board. For each square, if the square contains a chess piece, apply the following steps:*
 1. *Fetch the material value of the chess piece, and add it to the total material score count*
 2. *Evaluate how well it controls the centre, and add this rating to the total centre control count*
 3. *If the chess piece is a pawn, evaluate how well positioned it is relative to the other pawns, and add this rating to the pawn structure count*
- *After each square has been explored, evaluate the safety of the white and black kings, which will comprise a score for king safety*
- *Use the data of material score, centre control score, pawn structure score and king safety score, whilst also considering the current gamephase, in order to comprise a total score for the position of the current board. Return this score*

The flowchart also illustrates the top-down design model that the evaluation function system employs, as it is decomposed into the submodules 'get_material_score', 'get_centre_control_score', 'get_pawn_structure_score', 'get_king_safety_score' and 'calculate_total_score'. Details regarding the specific operations of each of these subroutines are explained below...

get_material_score()

```
SUBROUTINE
get_material_score(piece_object)
  return piece_object.value
ENDSUBROUTINE
```

Pretty simple isn't it?

This is due to the convenience of using object-oriented programming: when we create instances of 'ChessPiece' objects, we write the value of the piece ONLY there and then, and this value is immutable throughout the rest of the program. Although initialising the value attribute of the 'ChessPiece' depends on the type of piece it is, its colour, and the difficulty of the game chosen by the user, this data can be retrieved at any point in the game by simply calling [piece_name].value (e.g. pawn.value would return +/-10, depending on its colour)

ChessPiece
-colour: string
<u>+value: int</u>
-position: 2D-Array
+calculate_Moves()
+check_For_Checks()

NOTE: For each square being looped in the board, if the program detects that the square contains a piece, it will fetch the object of that piece with its current properties. This object is labelled as `piece_object`, and is passed in frequently to quickly access the squares the piece can manoeuvre to (by calling 'calculate_moves'), as well as to access its value, position and colour if needed.

get_centre_control_score()

```

SUBROUTINE
get_centre_control_score(piece_object)
  central_squares ← [(3,3),(3,4),(4,3),(4,4)]
  piece_in_centre_bonus ← 3 #values subject to change for optimisation
  pawn_controlling_centre_bonus ← 2
  non_pawn_controlling_centre_bonus ← 1
  piece_pos ← piece_object.position

  IF piece_object.colour = white THEN
    colour_multiplier ← 1
  ELSE #black piece
    colour_multiplier ← -1
  ENDIF

  #first check if piece is already in the centre
  IF piece_pos IN central_squares THEN
    return piece_in_centre_bonus * colour_multiplier
  ENDIF

  #check if piece is controlling a central square (if not already in one)
  watched_squares ← piece_object.calculate_moves() #gets a list of all squares the piece can move to
  FOR square IN watched_squares
    IF square IN central_squares THEN
      IF piece_object = pawn THEN
        return pawn_controlling_centre_bonus * colour_multiplier
      ELSE #piece is controlling the centre and is not a pawn
        return non_pawn_controlling_centre_bonus * colour_multiplier
      ENDIF
    ENDIF
  ENDFOR

  return 0 #piece not controlling centre/inside centre
ENDSUBROUTINE

```

In chess the central squares 'd5,e5,d4,e4' correspond to the 2D-arrays '(3,3),(3,4),(4,3),(4,4)' to my representation of the digital board. Although constants for the bonuses of a piece being in the centre, a pawn controlling the centre, or a non-pawn controlling the centre could and should be tweaked when testing in order to obtain the most accurate score for centre control, in all cases:

piece_in_centre_bonus > pawn_controlling_centre_bonus > non_pawn_controlling_centre_bonus
(more info as to why this is on page 5)

I will now introduce you to the concept of multipliers using the fact that we know that a better position for white returns a positive score, and for black a negative score. Instead of creating separate pawn control scores etc. for black and white, we can create a single constant for both pieces, and flip this constant to be positive or negative by multiplying it by a *colour_multiplier*. For instance, if a black knight is currently at the centre then a value of 3 * -1 is returned, and added to the overall 'centre_control_score'.

get_pawn_structure_score()

```

SUBROUTINE
get_pawn_structure_score(board,piece_object)
  IF piece_object ≠ pawn THEN
    return 0 #no need to be evaluated if piece is not a pawn
  ENDIF

  IF piece_object.colour = white
    colour_multiplier ← 1
  ELSE #black piece
    colour_multiplier ← -1
  ENDIF

  pawn_structure_score ← 0 #values subject to change for optimisation
  pawn_island_penalty ← -2
  double_pawn_penalty ← -5
  pawn_island ← true #we assume the pawn is isolated at first
  piece_row_pos,piece_col_pos ← piece_object.position
  surrounding_squares ← [(0,1),(0,-1),(1,0),(-1,0),(1,1),(1,-1),(-1,1),(-1,-1)]

  #checking if the pawn is isolated
  FOR square IN surrounding_squares
    new_row,new_col ← piece_row_pos + square[0],piece_col_pos + square[1]
    IF (0 ≤ new_row + square[0] ≤ 7) AND (0 ≤ new_col + square[1] ≤ 7) THEN
      IF (board[new_row][new_col] = pawn) AND (board[new_row][new_col].colour =
        piece_object.colour) THEN
        pawn_island ← false
        break #no need to look at other squares as we know the pawn is connected to an ally pawn
      ENDIF
    ENDIF
  ENDFOR

  IF pawn_island = true
    pawn_structure_score ← pawn_structure_score + pawn_island_penalty
  ENDIF

  #checking for doubled/tripled pawns
  WHILE piece_row < 7
    IF (board[piece_row+1][piece_col] = pawn) AND (board[piece_row+1][piece_col].colour =
      piece_object.colour) THEN
      pawn_structure_score ← pawn_structure_score + double_pawn_penalty
      break
    ENDIF
    piece_row += 1
  ENDWHILE

  return pawn_structure_score * colour_multiplier
ENDSUBROUTINE

```

The first chunk of pseudocode acts as input sanitisation, as most 'piece_objects' won't actually be pawns thus will be obsolete when evaluating pawn structure. If the input piece is identified as a pawn we begin by checking if the pawn is isolated, by scanning each square adjacent to it and determining if that also contains a pawn of the same colour (if so then the pawn is no longer on a [pawn_island](#)) If [pawn_island](#) is still set to true by the end of the search then this is of course bad for the pawn's team, so a penalty is issued.

The 2nd element of the pawn structure evaluation is penalising the inputted pawn object if it is a member of a double pawn structure (or worse). Notice how we only search one direction (downwards actually as 'piece_row' is incrementing) of the board: This is due to the fact that this subroutine only takes the argument of a single pawn, and not ALL of the pawns. For example if we are going to scan each row from top to bottom whilst reading the squares of each row from left to right, the 'h6' pawn will be evaluated first and detect that the 'h5' pawn is directly below it, raising the double-pawn penalty. Eventually the evaluation function will reach the 'h5' pawn and discover that the 'h3' pawn is directly below it, so the double-pawn penalty is activated again. Mathematically, if the double pawn penalty is called n times, then there are $n+1$ pawns on that particular file.

The colour multiplier still should function as usual despite the pawn_structure_score being negative. For example if a black pawn is classified as isolated then a value of $-2 * -1$ is returned and the total pawn structure score (+2 penalises black, benefits white).

NOTE: I have decided to completely scrap the aspect of evaluating whether or not the pawn is close to promoting, contrary to my initial thoughts in the analysis section. The reason for this is as the A.I will of course search and generate future positions relative to the current one, it hopefully will simulate the case when the pawn that was close to promoting actually does promote (to a better piece, therefore skewing the evaluation score in favour of the colour of the promoted pawn). Consequently the minimax function itself should cover the benefits of having a pawn closely promoted.

(more info regarding the benefits of occupying a good pawn structure on page 4)



get_king_safety_score()

```

SUBROUTINE
get_king_safety_score(board,white_king_object,black_king_object)
  king_safety_score ← 0
  castling_bonus ← 5 #values subject to change for optimisation
  pawn_shield_bonus ← 3 #per pawn
  white_king_row_pos,white_king_col_pos ← white_king_object.position
  black_king_row_pos,black_king_col_pos ← black_king_object.position

  #check if kings have castled
  IF white_king_object.castled = true THEN
    king_safety_score ← king_safety_score + castling_bonus
  IF black_king_object.castled = true THEN
    king_safety_score ← king_safety_score - castling_bonus
  ENDIF

  #check the presence of the pawn shield
  col_vectors ← [-1,0,1]
  white_king_row_pos -= 1 #this variable now stores the row in which the white pawn shield should be at
  black_king_row_pos += 1 #this variable now stores the row in which the black pawn shield should be at

  FOR col_vector IN col_vectors
    white_king_new_col,black_king_new_col ← white_king_col_pos + col_vector,black_king_col_pos +
    col_vector
    IF (0 <= white_king_row_pos <= 7) AND (0 <= white_king_new_col <= 7) THEN
      IF board[white_king_row_pos][white_king_new_col] = 'wp' THEN
        king_safety_score ← king_safety_score + pawn_shield_bonus
      ENDIF
    IF (0 <= black_king_row_pos <= 7) AND (0 <= black_king_new_col <= 7) THEN
      IF board[black_king_row_pos][black_king_new_col] = 'bp' THEN
        king_safety_score ← king_safety_score - pawn_shield_bonus
      ENDIF
    ENDIF
  ENDFOR

  return king_safety_score
ENDSUBROUTINE

```

Unlike the previous 2 subroutines, the 'king_safety_score' module is called only once within the evaluation function, and specifically calls in objects for both kings.

In order to see if either kings have castled we just need to access the 'castled' boolean attribute for each object.

King
+value int: ∞
+first_move:boolean
<u>+castled: boolean</u>
+calculate_Moves()
-castle()

When detecting the pawn shields, I returned to the idea of using vectors: we iterate through `col_vectors` in order to see if there is a matching pawn to the left,middle or right to the row in front of the kings (`white_king_row_pos/black_king_row_pos`), and add/subtract to the total 'king_safety_score' when appropriate. (more info regarding the benefits of having a safer king on page 6)

calculate_total_score()

```

SUBROUTINE
calculate_total_score(material_score,centre_control_score,pawn_Structure_score,king_safety_score)
  component_multipliers ← {'opening':[1,0.5,0.1,0.1],'middlegame':[1,0.3,0.4,0.7],'endgame':[1,0,0.6,0]}
  #values subject to change for optimisation
  gamephase ← gamedata.gamephase

  material_multiplier,centre_control_multiplier,pawn_structure_multiplier,king_safety_multiplier ←
  component_multipliers[gamephase]
  material_score ← material_score * material_multiplier
  centre_control_score ← centre_control_score * centre_control_multiplier
  pawn_structure_score ← pawn_structure_score * pawn_structure_multiplier
  king_safety_score ← king_safety_score * king_safety_multiplier

  total_score ← material_score + centre_control_score + pawn_structure_score + king_safety_score
  return total_score
ENDSUBROUTINE

```

This subroutine is called at the very end of the evaluation function, where we combine our hard-fought values of 'total_material_score', 'total_centre_control_score', 'total_pawn_structure_score', and 'king_safety_score' to generate a single floating value for the overall score of a given chessboard, now finally considering EVERYTHING.

Once again we use the help of multipliers, but this time slightly differently: **component_multipliers** is a dictionary data structure with properties:

- key: gamephase
- value: a list of the corresponding multipliers for that particular gamephase
 - Multipliers read from left to right: material multiplier, centre control multiplier, pawn structure multiplier, king safety multiplier

So for instance if our arguments in order were (-10, 4, 5, 8), and the current gamephase held in the gamedata object was the 'middlegame', then our calculation for 'total_score' would be:

$$(-10 * 1) + (4 * 0.3) + (5 * 0.4) + (8 * 0.7) = -1.2$$

NOTE: Just to re-emphasise, the particular multipliers established above are absolutely not concrete. Rigorous testing is required in order to find the correct balance between all components, for all phases of the game.

Defining the Minimax Function

Pseudocode

#the gamedata and move_engine object is referenced frequently within the minimax subroutine

SUBROUTINE

minimax(board, is_maximising, depth)

MAX_SCORE ← INFINITY

MIN_SCORE ← -INFINITY

IF gamedata.checkmate = True THEN

IF gamedata.winner = 'AI' THEN #AI has checkmated Player

return MAX_SCORE

ELSE #Player has checkmated AI

return MIN_SCORE

ENDIF

ELSE IF gamedata.stalemate = True THEN #draw

return 0

ELSE IF depth = 0 THEN #depth limit reached

return evaluation_function(board)

ENDIF

IF is_maximising = True THEN #AI is trying to maximise score

best_move_score ← MIN_SCORE

FOR available_move IN gamedata.available_moves

move_engine.make_move(available_move)

score ← minimax(board, False, depth-1)

IF score > best_move_score THEN

best_move_score ← score

ENDIF

move_engine.undo_move()

ENDFOR

ELSE IF is_maximising = False THEN #Player is trying to minimise score

best_move_score ← MAX_SCORE

FOR available_move IN gamedata.available_moves

move_engine.make_move(available_move)

score ← minimax(board, True, depth-1)

IF score < best_move_score THEN

best_move_score ← score

ENDIF

move_engine.undo_move()

ENDFOR

ENDIF

return best_move_score

ENDSUBROUTINE

Explanation

Firstly, we check whether the current state of the game is terminal, or if `depth` initialised has reached its limit by decrementing all the way down to zero. It is important to acknowledge that the constants 'MAX_SCORE' and 'MIN_SCORE', holding values of infinity and -infinity respectively, represent the best possible and worst possible scores for the AI, although it is impractical to represent infinity programmatically (use extremely large/small values instead)

A brief introduction to the 'move_engine' class

The method 'make_move' requires a single argument: the instruction of the move, which bar from any special chess moves (castling etc.), consists of the starting square a piece is initially at and the destination square the piece will move to. Using this piece of data, in order to 'make_move', the following actions must occur:

- Update the 'digital_board' attribute, using the argument given to replace the destination square object (even if there is already one) with the starting square object
- Update all the necessary attributes in the 'gamedata' class, including the 'available_moves' variable
- Update attributes for piece objects if necessary (e.g. pawn.first_move)

The method 'undo_move' will reverse the actions above, with help from the 'move_log' and 'capture_log' attributes.

(further details regarding how the following two methods will be implemented into code on page [insert page])

Now we approach the recursive part of the function (7.3). We first instantiate the variable 'best_move_score', which will store an integer value representing the score of the best available move to play that has been found so far. We always begin with the worst possible score for whoever's turn it is: if the current turn is the A.I (who wants to maximise the score), we set 'best_move_score' to 'MIN_SCORE' and maximise our way up, or if its the Player's turn (who wants to minimise the score), 'best_move_score' is set to 'MAX_SCORE' and the player aims to minimise the score down.

Regardless of who is currently maximising/minimising, the principles of the instruction that need to be carried out are the same and are as follows:

Iterate through all available moves (7.2). For each move, apply the following steps:

1. *Perform the move on the board*
2. *Call the function recursively, decrementing the depth and flipping the is_maximising variable*
3. *Undo the move on the board*
4. *If the score is higher/lower than the best score (depending on who is maximising), replace the best score with the new score*

Step 1 is achieved by using the 'move_engine.make_move' method. If the current turn is the maximising player, is_maximising will need to be `False` in the recursion call in order to simulate the switch in turn for the minimising player, otherwise is_maximising is set to `True` for the contrary. Finally, we execute step 4 by utilising the 'move_engine.undo' method.

At the end of the minimax algorithm, after looping through every available move, the value of the best score is returned.

Adding Alpha-Beta Pruning to the Minimax Function

SUBROUTINE

minimax(board,is_maximising,depth,alpha,beta)

#some code below...

The first amendment that needs to be done is to initialise 2 more parameters: **alpha** stores the best value that the maximising player can guarantee (the highest value) at the current depth or above, whereas **beta** contains the best value for the minimiser (the lowest value). When minimax is called the first time, the values of **alpha** and **beta** are initially set to 'MIN_SCORE' and 'MAX_SCORE' respectively as, similarly to 'best_move_score', we always begin with the least optimal score. After that, **alpha** aims to increase its value whereas **beta** aims to decrease its value. This can be visualised on a number line:



#added code from minimax, on behalf of alpha

IF is_maximising = True THEN #AI is trying to maximise score

best_move_score ← MIN_SCORE

FOR available_move IN gamedata.available_moves

move_engine.make_move(available_move)

score ← minimax(board,False,depth-1,alpha,beta)

IF score > best_move_score THEN

best_move_score ← score

ENDIF

move_engine.undo_move()

alpha ← max(alpha,best_move_score)

if beta <= alpha:

break #

ENDFOR

As we iterate through each possible node, **alpha** gradually increments. For instance, if we are at the root depth of our minimax algorithm (root depth must mean we are maximising the score first), let the first node out of the list 'gamedata.available_moves' return a score of -3. Our AI evaluates the higher value between the previous value of **alpha** ('MIN_SCORE') and the score of the current 'best' move found (-3 is the only move searched so far thus it must be the best) This of course is -3, which is passed in as the new value of **alpha** for the second move in the list, and so on (until a score higher than -3 is found, then alpha increases again...)

The exact same principle applies on the other end for **beta** when scores for the minimiser are generated, but **beta** is re-assigned to the lower value of its previous self and the score of the current 'best' move found at that moment, in contrast to the higher value for **alpha**.

It is essential to reiterate that as the search progresses, **alpha** and **beta** typically converge closer and closer together, which can be illustrated by using the same number line as earlier:



In some cases, the AI may observe that the value of **alpha** has equaled or exceeded the value of **beta**. When this happens, it knows that there is no point traversing the remainder of the current position and its children's nodes. This is due to the fact that even if a very large number for **alpha** arises from the current position's remaining child nodes, its **beta** value is low enough for the A.I to safely assume to know to just pick the other sibling node.

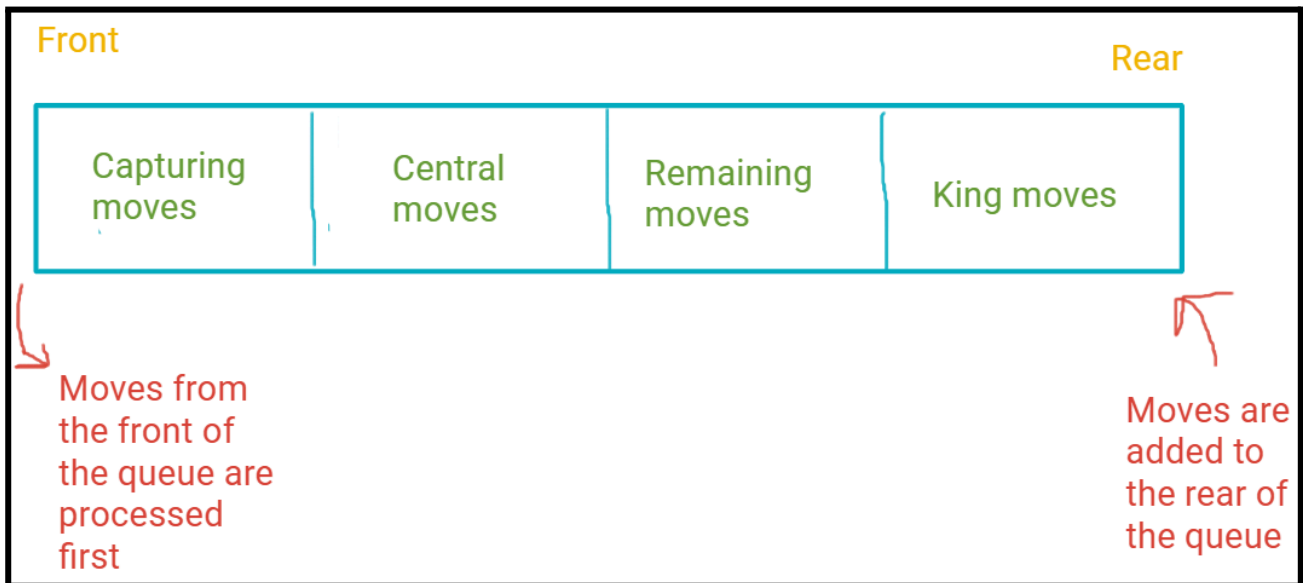


In essence, the number line has told us all along that in order for a node in the minimax tree to contain a possible path to the solution (thus be worthy to have its children to be traversed), $\alpha < N < \beta$, where N is the range of possible values for that particular node. The fact that there is no overlap between the ranges of alpha and beta suggests that the node can no longer possibly contain a possible solution, so it can be pruned. This is why the condition $\alpha \geq \beta$ works for pruning moves.

Designing move ordering in order to optimise alpha-beta pruning

In the examples above, the variable 'gamedata.available_moves' contained a list of all the possible moves the player on that particular turn could make. Currently, there is no order and arrangement to how these moves are sorted into the list.

This can be solved by implementing the idea of **queues**. The items at the front of the queue are examined first by the minimax algorithm, and likewise the moves at the rear of the queue are processed last. A reminder that by transforming 'gamedata.available_moves' into a queue data structure so that it is ordered by best move descending, this increases the likelihood of more nodes in the minimax tree being pruned by alpha-beta, thus considerably improving the time efficiency for the A.I generating the optimal move.



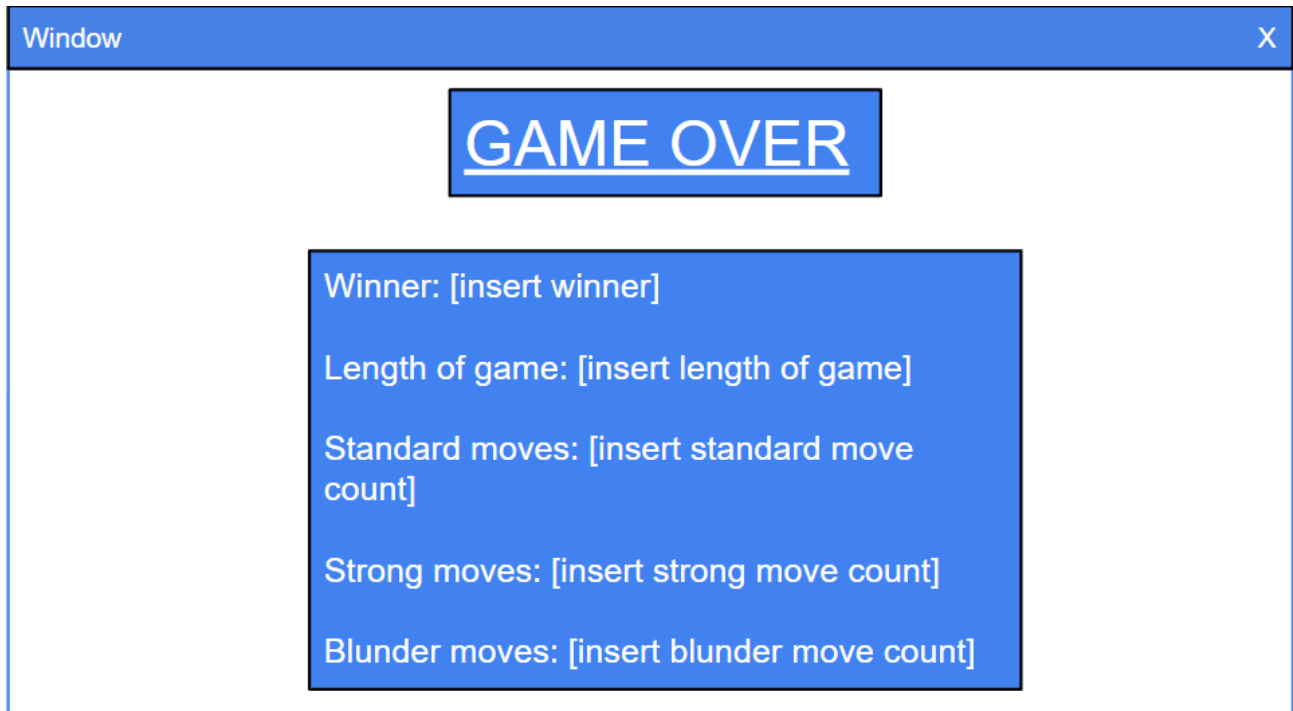
Above illustrates a diagram of the 'gamedata.available_moves' queue, including the order for some of the particular types of chess moves. Capturing moves have the greatest chance of producing a high evaluation score (as the material evaluation possesses the greatest weight in the evaluation function). A central move can be defined as any action that involves moving the piece to one of the 'e4,d4,e5,d5' squares, and doing this can potentially lead to a higher centre control score than the standard move. Interestingly, I have decided that any moves involving the motion of the king are to be evaluated last by minimax, as moving the king (apart from castling) is generally a bad idea. Linking this more specifically to our evaluation function, moving the king could damper its king safety score as it may have left its pawn shield.

Of course, the concept of move ordering is only a heuristic one that can be done quickly: moving the king may not be a bad move, likewise capturing a piece may not be a good move. The evaluation function is the more complex algorithm that will provide the more precise approximations to the suitability of each action. Therefore, it is important that the algorithm to sort the 'available_moves' in order is time efficient (a move ordering algorithm that is NOT time efficient would otherwise be completely redundant)

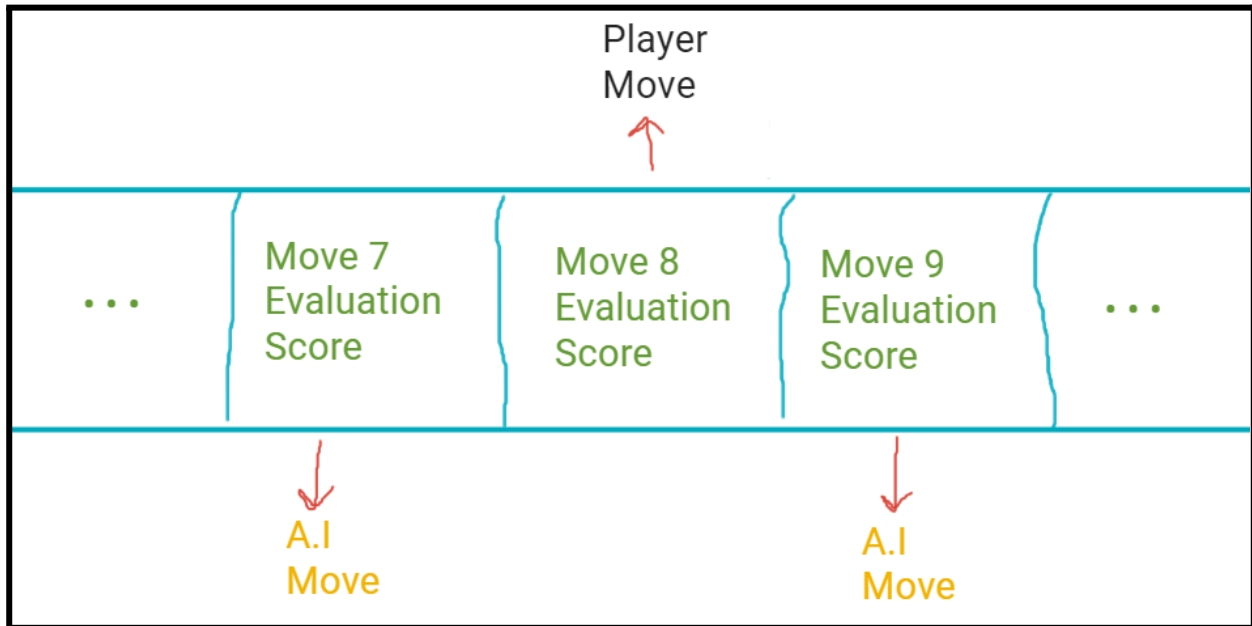
Further User Interface Design: Post Game Report Design

Links to objectives:

- 10. Generate a post-game report for the user



(10.2/10.3) This is the GUI design for the post game report of the chess game, which is the final item displayed when the game is completed. This allows the user to visualise all of the key details for the game they have just played, without over-saturating the report with too much information in order to add a form of abstraction and simplicity.

Initial ideas for tackling the player move evaluation algorithm (10.1)

The idea is that I create another queue data structure, which after each move made by the A.I, a numerical score of the evaluation for the current board after said move is added to the rear of the queue.

In order to determine the strength of each move made by the user, we can observe and compare the evaluations of the board at either end of the move (the A.I's move before and after). If the evaluation of the board AFTER the player's move is more beneficial for the player than it was BEFORE, then we can confidently say that the move made by the player was 'good'.

In this example let a higher evaluation score be better for the player. If the evaluation score queue held the values $[-3, 4, -1]$ after moves 7, 8, 9 respectively, where move 8 is the score of the board when the player makes the move, a suitable calculation my program could make would be:

- $[A.I \text{ evaluation after} - A.I \text{ evaluation before}]$
- $-1 - 3 = 2$

As there is a net evaluation positive ($2 > 0$), the move made by the player in move 8 must've benefitted him/her, as the response of the A.I put it in a worse position than it was beforehand.

The quantity of the net evaluation score also matters: if the net evaluation score is higher than a certain amount, the move described by the player can be considered as 'strong'. Alternatively, if the net evaluation score is lower than a certain amount, the move described by the player can be considered as a 'blunder'. Any score in the middle can be considered as a 'standard' move.

Technical Solution and Testing

Preliminary information

Familiarising with PyGame

The PyGame module is commonly used in providing an interface for simple 2D games that are not memory intensive, which is an ideal compliment for my chess game (displaying chess is NOT memory intensive as fundamentally, only a checkerboard and jpeg images of chess pieces need to be changed).

The PyGame module can facilitate with:

1. Drawing images and playing sounds
2. Providing player input via mouse clicking keyboard pressing
3. Displaying text and shapes

Full detail regarding the functionality of PyGame operations can be found via their official documentation:

<https://www.pygame.org/docs/>

Introduction to the main loop

```
def main(): #main game loop
    global mouse_pos,current_turn,game_active
    move_data = [] #contains 2 tuples: location of start and end squares clicked by the user
    while True:
        mouse_pos = (0,0)
        for event in pygame.event.get():
            if event.type == pygame.QUIT: #close window
                pygame.quit()
                sys.exit()

            if event.type == pygame.KEYDOWN and game_active:
                if event.key == pygame.K_u: # 'U' key is pressed = U = Undo move
                    if len(gamedata.move_log_stack) != 0 and len(gamedata.move_log_stack) != 1:
                        #cannot undo a move if no moves have taken place

                        move_engine.undo_move()
                        post_game_report.pop_evaluation_queue()
                        move_engine.undo_move() #called twice as A.I move also needs to be undone
                        post_game_report.pop_evaluation_queue()
                        move_data = []
                        game_active = True
                    elif event.key == pygame.K_h and chessboard.hint_move == []: # 'H' key is pressed = H = Show hint
                        chessboard.hint_move = ai.generate_move(True)

            if event.type == pygame.MOUSEBUTTONDOWN: #user click
                mouse_pos = pygame.mouse.get_pos()
                if game_active:
                    clicked_pos = appendices.get_col_row_from_pos(mouse_pos[0],mouse_pos[1])
                    if move_data == []:
                        move_data.append(clicked_pos)
                    else:
                        if clicked_pos == move_data[0]: #same square is clicked twice: deselect
                            move_data = []
                        else:
                            move_data.append(clicked_pos)

            if not game_active: #display main menu/post_game_report if false, display game if true
                if gamedata.is_checkmate or gamedata.is_stalemate: #after game is finished
                    post_game_report.display(gamedata.move_count,gamedata.winner)
                else: #before game begins
                    main_menu.display()
            else: #game is active, display chessboard
                chessboard.display_board(MINT,ARCTIC)
                if current_turn != gamedata.player_colour: #AI's turn
                    move_data = ai.generate_move()
                    move_data = move_engine.initiate_move(move_data)
```

NOTE: Not all of the main loop is shown here. Further details regarding each part of the main loop can be found later

The infinite main loop is the heart and foundation for all PyGame archetypes in the way it allows algorithms and data structures to constantly be inspected/monitored/updated as the program runs.

Introduction to the appendices static class

```

class appendices:
    #this class neatly stores all additional functions that are frequently used
    #within the rest of the program, but are not distinctly part of a specific class

    @staticmethod
    def flip_current_turn(turn):...

    @staticmethod
    def map_piece_to_image():...

    @staticmethod
    def map_pos_to_square():...

    @staticmethod
    def get_col_row_from_pos(x_pos,y_pos):...

    @staticmethod
    def text_schematic(text,x_pos,y_pos,size,colour,point):...

    @staticmethod
    def border_schematic(rect):...

    @staticmethod
    def begin_game(difficulty_chosen,colour_chosen):...

    @staticmethod
    def initiate_post_game():...

```

The purpose of the appendices class is that it acts as a tidy solution of collating and encapsulating all of the supplementary functions used in various places throughout the program in a single location, improving the readability of my code. Furthermore, the appendices class only consists of static methods (using the `@staticmethod` decorator), and doesn't actually possess any unique attributes for itself, meaning that no object instantiation is required (less objects instances = improved speeds and memory utilisation within executing the program).

The specifics regarding the operation to each static method is delved into later.

Implementing the Main Menu (1.)

Initialising the Main Menu Class

```
class main_menu:
    def __init__(self):
        self.is_difficulty = False #true if difficulty has been chosen, false if not
        self.difficulty_chosen = ''
        self.is_player_colour = False #true if player colour has been chosen, false if not
        self.player_colour_chosen = ''
```

The attributes `is_difficulty` and `is_player_colour` are boolean values to ensure that all necessary data has been entered, and are both initially set to 'False'. Throughout other class methods these attributes may be modified to 'True' depending if the player has inputted a colour/difficulty. It is important to note now that the game CANNOT start unless `is_difficulty` and `is_player` are both True. #show instantiation of main menu object

The Main Menu Display Method

```
def display(self):
    screen.fill(GREY)
    self.display_start_button()
    self.display_prompts()
    self.display_title()
    self.display_and_update_piece_select()
    self.display_and_update_difficulty_select()
    self.display_status_box()
```

The code below is within the main infinite loop, and illustrates how/when the `display` method is called within the program (`main_menu.display()` is ONLY called here and nowhere else)

```
if not game_active: #display main menu/post_game_report if false, display game if true
    if gamedata.is_checkmate or gamedata.is_stalemate: #after game is finished
        appendices.initiate_post_game()
    else: #before game begins
        main_menu.display()
```

Implementing the A.I Difficulty UI

Updating user input for difficulty chosen

```
def process_difficulty_input(self, beginner_textbox, intermediate_textbox, expert_textbox, clicked_pos):
    # if the 'beginner', 'intermediate' or 'expert' buttons are clicked, update the difficulty
    if beginner_textbox.collidepoint(clicked_pos[0], clicked_pos[1]) == 1:
        self.difficulty_chosen = 'Beginner'
    elif intermediate_textbox.collidepoint(clicked_pos[0], clicked_pos[1]) == 1:
        self.difficulty_chosen = 'Intermediate'
    elif expert_textbox.collidepoint(clicked_pos[0], clicked_pos[1]) == 1:
        self.difficulty_chosen = 'Expert'
```

The attribute `difficulty_chosen` stores the currently chosen difficulty by the player (1.1.2)

In general, the method `textbox.collidepoint()` (built in to the PyGame module), where 'textbox' is a particular piece of area in an x-y plane, takes a set of x-y coordinates as arguments, and outputs a 1 or 0, depending on whether or not the coordinate is inside the space of the 'textbox'.

For instance, if the user clicks on the 'expert' button (encapsulated by the 'expert' textbox), the middle if statement above will become satisfied, and the `difficulty_chosen` is updated to 'Intermediate' (1.1.3)

Displaying the difficulty buttons

```
def display_and_update_difficulty_select(self):
    if self.difficulty_chosen == 'Beginner': #highlight the text 'Beginner' in green only
        self.is_difficulty = True
        beginner_colour = GREEN
        intermediate_colour = WHITE
        expert_colour = WHITE
    elif self.difficulty_chosen == 'Intermediate': #highlight the text 'Intermediate' in green only
        self.is_difficulty = True
        beginner_colour = WHITE
        intermediate_colour = GREEN
        expert_colour = WHITE
    elif self.difficulty_chosen == 'Expert': #highlight the text 'Expert' in green only
        self.is_difficulty = True
        beginner_colour = WHITE
        intermediate_colour = WHITE
        expert_colour = GREEN
    else: #difficulty not yet chosen, don't highlight any text
        beginner_colour = WHITE
        intermediate_colour = WHITE
        expert_colour = WHITE
```

My main menu uses colour to convey which settings are currently picked by the user. In this case, if a setting is in green then that is the currently picked difficulty of the AI the user wants to play against, and white otherwise. The boolean attribute `is_difficulty` is tweaked from 'False' to 'True' here and only here.

```
#creating and displaying surfaces for the difficulty UI
difficulty_text_surface,difficulty_text_rect = appendices.text_schematic('Difficulty',50,200,40,WHITE,'midleft')
screen.blit(difficulty_text_surface,difficulty_text_rect)

difficulty_text_underline_rect = pygame.Rect(difficulty_text_rect.left,difficulty_text_rect.bottom,
difficulty_text_rect.width,10)
pygame.draw.rect(screen,WHITE,difficulty_text_underline_rect)

beginner_surface,beginner_rect = appendices.text_schematic('Beginner',difficulty_text_underline_rect.left,
difficulty_text_underline_rect.bottom,30,beginner_colour,'topleft')
screen.blit(beginner_surface,beginner_rect)

intermediate_surface,intermediate_rect = appendices.text_schematic('Intermediate',beginner_rect.left,
beginner_rect.bottom+10,30,intermediate_colour,'topleft')
screen.blit(intermediate_surface, intermediate_rect)

expert_surface,expert_rect = appendices.text_schematic('Expert',intermediate_rect.left,intermediate_rect.bottom+10,
30,expert_colour,'topleft')
screen.blit(expert_surface,expert_rect)

self.process_difficulty_input(beginner_rect,intermediate_rect,expert_rect,mouse_pos)
```

After the colours have been defined, all surfaces related to the difficulty are displayed. This includes the buttons for the 'Beginner', 'Intermediate' and 'Expert' levels, as well as the text title 'Difficulty' above its underline rectangle (1.1.1). The `text_schematic` function is one of the many static methods contained inside the `appendices` class, and returns the necessary surfaces and rectangles required.

```
@staticmethod
def text_schematic(text,x_pos,y_pos,size,colour,point):
    #creates text of any colour,font and size in a given position

    font = pygame.font.Font(game_font, size)
    surface = font.render(str(text),True,colour)
    if point == 'centre': #if x_pos,y_pos is a central point
        rect = surface.get_rect(center = (x_pos,y_pos))
    elif point == 'midleft': #if x_pos,y_pos is the middle-left point
        rect = surface.get_rect(midleft = (x_pos,y_pos))
    elif point == 'topleft': #if x_pos,y_pos is the top-left point
        rect = surface.get_rect(topleft = (x_pos,y_pos))
    elif point == 'topright': # if x_pos,y_pos is the top-right point
        rect = surface.get_rect(topright = (x_pos,y_pos))
    elif point == 'bottomleft': #if x_pos,y_pos is the bottom-left point
        rect = surface.get_rect(bottomleft = (x_pos,y_pos))
    else:
        return 'Invalid information'
    return surface,rect
```

Finally the method call of `process_difficulty_input()` is performed at the very end of the function.

Implementing the Colour Selection UI

As you can see there is a lot of overlap between implementing the colour and difficulty UI's for the main menu, other than tweaking different attributes instead. (1.2.2/1.2.3)

```
def process_colour_input(self, white_piece_image, black_piece_image, clicked_pos):
    #if the images of the black or white pieces are clicked, update the attributes of the colour chosen
    if white_piece_image.collidepoint(clicked_pos[0], clicked_pos[1]) == 1:
        self.player_colour_chosen = 'w'
    elif black_piece_image.collidepoint(clicked_pos[0], clicked_pos[1]) == 1:
        self.player_colour_chosen = 'b'
```

```
def display_and_update_piece_select(self):
    if self.player_colour_chosen == 'w': #highlight around the white piece only
        self.is_player_colour = True
        white_colour = GREEN
        black_colour = WHITE

    elif self.player_colour_chosen == 'b': #highlight around the black piece only
        self.is_player_colour = True
        white_colour = WHITE
        black_colour = GREEN

    else: #player piece not yet chosen, don't highlight around either piece
        white_colour = WHITE
        black_colour = WHITE

    colour_text_surface, colour_text_rect = appendices.text_schematic('Colour', 450, 200, 40, WHITE, 'centre')
    screen.blit(colour_text_surface, colour_text_rect)

    colour_text_underline_rect = pygame.Rect(colour_text_rect.left, colour_text_rect.bottom, colour_text_rect.width, 10)
    pygame.draw.rect(screen, WHITE, colour_text_underline_rect)
```

However, I had to use a slightly different approach to displaying the black and white kings (1.2.1) (to represent each colour). As these are images, the text_schematic method was not applicable. As a result I resorted to using the `transform.rotozoom()` built in PyGame method, which produces a scaled surface of the inputted image (in this case we wanted a surface of a scaled image of the black and white kings)

```
#creating and displaying the image of the white king
white_king_display = pygame.transform.rotozoom(piece_images_dictionary['wK'], 0, 1.5)
white_rect = white_king_display.get_rect(topleft = (colour_text_underline_rect.left, colour_text_underline_rect.bottom+10))
white_border_rect = appendices.border_schematic(white_rect)
screen.blit(white_king_display, white_rect)
pygame.draw.rect(screen, white_colour, white_border_rect, 5)

#creating and displaying the image of the black king
black_king_display = pygame.transform.rotozoom(piece_images_dictionary['bK'], 0, 1.5)
black_rect = black_king_display.get_rect(topleft = (white_rect.right+30, white_rect.top))
black_border_rect = appendices.border_schematic(black_rect)
screen.blit(black_king_display, black_rect)
pygame.draw.rect(screen, black_colour, black_border_rect, 5)

self.process_colour_input(white_rect, black_rect, mouse_pos)
```

As usual, the method that detects whether or not the user has chosen a colour (by clicking on it) is called at the end.

Creating the image piece dictionary

```
white_king_display = pygame.transform.rotozoom(piece_images_dictionary['wK'],0,1.5)
```

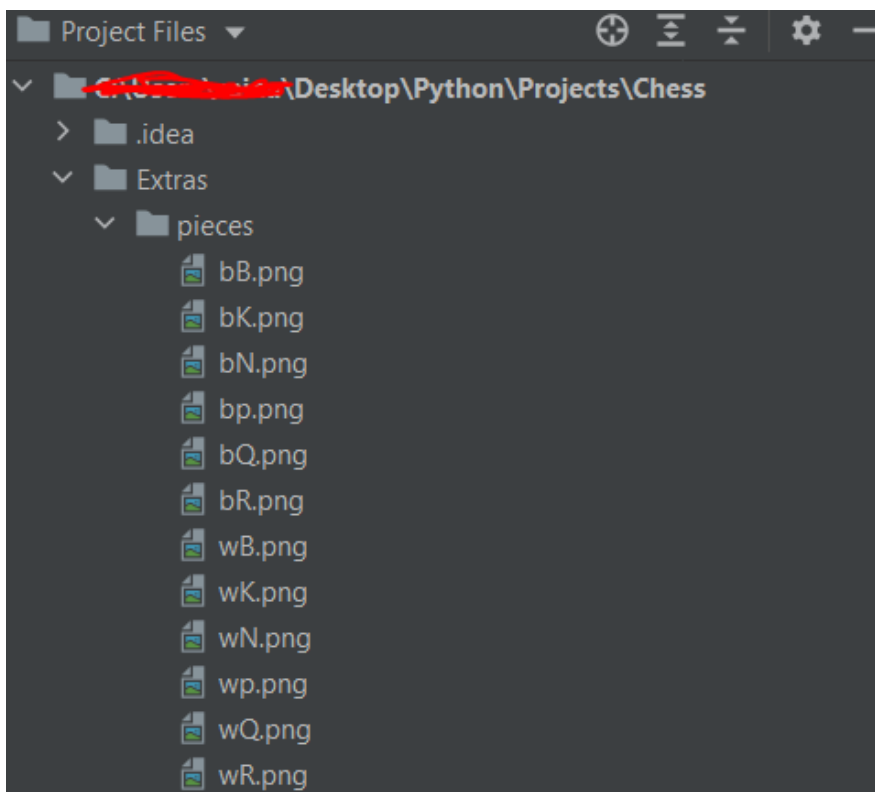
(1.5 tells the program to scale the image 1.5x its usual size, 0 means that the image is rotated 0 degrees :))

The purpose of `piece_image_dictionary` is that it takes in an abbreviation of a chess piece as a key, and returns its corresponding png image. The method below is code for creating this dictionary in the `appendices` class, and is called at the very beginning of the program.

```
@staticmethod
def map_piece_to_image():
    dictionary = {} #use of global dictionary allows easy access when mapping a piece to its image
    pieces = ['bB','bK','bN','bp','bQ','bR','wB','wK','wN','wp','wQ','wR']
    for piece in pieces:
        image_png = pygame.image.load('Extras/pieces/'+piece+'.png').convert_alpha()
        dictionary[piece] = pygame.transform.scale(image_png,(cell_size,cell_size))
        #initial size of pieces set to cover a square on the board

    return dictionary
```

In PyCharm, files can only be accessed and used if they are in the same directory as in the python file. Below is a display of how this is achieved.



Implementing the Start Button Mechanism

Displaying the start button

```
def display_start_button(self):
    if self.is_difficulty == True and self.is_player_colour == True:
        border_colour = GREEN #highlight start button in green if it is allowed to be clicked
    else:
        border_colour = RED #highlight start button in red if details are missing

    start_button_surface, start_button_rect = appendices.text_schematic('START', 580, 650, 50, WHITE, 'centre')
    border_rect = appendices.border_schematic(start_button_rect)
    pygame.draw.rect(screen, border_colour, border_rect, 5)
    screen.blit(start_button_surface, start_button_rect)
```

The process to create and display the surfaces and rectangles for the start button is similar to the difficulty and colour buttons previously. The RGB value `border_colour` surrounds the start button, and if green it means that all necessary details have been entered and the game would begin if pressed (red otherwise).

Examining the status of the main menu settings

```
def check_status(self, start_button_textbox):
    if start_button_textbox.collidepoint(mouse_pos[0], mouse_pos[1]) == 1:
        #if the start button has been clicked, check if game can begin
        if self.is_difficulty == True and self.is_player_colour == True:
            #start button is clicked and necessary data has been entered
            appendices.begin_game(self.difficulty_chosen, self.player_colour_chosen)
```

The `check_status` method is continuously being run in the background whilst the main menu is being displayed, and determines whether or not the game can begin. In order for this to happen, the following 2 conditions must be met: (1.3.1)

1. User's current click is on the start button textbox
2. Details of difficulty and colour are BOTH chosen

If both of these conditions are met, the game can begin. (1.3.2)

```
@staticmethod
def begin_game(difficulty_chosen, colour_chosen):
    #instantiates/updates all necessary variables/objects when user clicks start button (successfully)
    global game_active
    game_active = True
    gamedata.difficulty = difficulty_chosen

    if colour_chosen == 'w': #user chose white, A.I is black
        gamedata.player_colour = 'w'
        gamedata.ai_colour = 'b'
        ai.ai_colour = 'b'
    else: #user chose black, A.I is white
        gamedata.player_colour = 'b'
        gamedata.ai_colour = 'w'
        ai.ai_colour = 'w'

    chessboard.initialize_pieces()
    ai.initialize_depth()
    gamedata.obtain_all_valid_moves(chessboard.digital_board, 'b', chessboard.class_dictionary, 'first move')
    post_game_report.player_colour = colour_chosen
    # if A.I is white, it already will have some moves to choose from
```

Any unfamiliar methods/attributes here will be explained shortly. :)

Testing the Main Menu (1.)

Test Sequence 1

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
01	1. Click the start button without entering anything else	Erroneous: Shows that game will not begin when data on colour and difficulty has not been entered	• Game will not begin • Prompts asking the user to enter data for difficulty and colour still displayed	0:00 - 0:27 Pass
02	1. Click on the 'Beginner' button 2. Click the start button	Erroneous: Shows that game will not begin when data on colour and difficulty has not been entered	• Game will not begin • Prompts asking the user to enter data for colour still displayed	0:27 - 0:53 Pass
03	1. Click on the 'Expert' button 2. Click on the white king button 3. Click the start button	Normal: Shows that game will begin, and main menu is concealed when all required data has been entered	• Prompts disappear after colour has been entered • Start button becomes green • Game begins	0:53 - 2:18 Pass
04	1. Click on the 'Intermediate' button 2. Click on the black king button 3. Click on the 'Beginner' button 4. Click on the white king button 5. Click on the 'Expert' button 6. Click the start button	Normal: Shows that system can adapt when user intends to change the settings	• Prompts disappear after colour has been entered • Start button becomes green • Beginner button becomes green, intermediate button becomes white • White king button becomes green, black king button becomes white • Expert button becomes green, beginner button becomes white • Game begins	2:18 - 3:41 Pass

The advantage of using mouse input instead of keyboard input for my user interface is that this severely restricts and filters the range of data the user can enter (all the user can do to add input is to click), eliminating the likelihood of error and the need for exception handling/defensive programming.

Access to test sequence 1 can be found by scanning the QR code, or by using the following link: <https://youtu.be/8zFbainMuic>



Implementation of the Game Functionalities (2./3./4./5.)

Introduction to the chessboard class

The primary purpose of the chessboard class is to digitalize the contents of the physical board into manageable constructs which can be interpreted by the game engine. This is done by using a front-end back-end approach, where the 2D-array `digital_board` (2.3.1) is a surface-level abstraction of the current board to be displayed directly to the user (hides the complexities of operations from the user), and the `class_dictionary` attribute comprises of objects of the individual pieces that constitute the `digital_board`. It is important to note that of the two attributes, the `class_dictionary` attribute is accessed and modified more frequently in the background by the game engine and A.I when the program is running.

```
class chessboard:
    def __init__(self):
        self.class_dictionary = {}
        #enter a co-ordinate in order to return an object (if any) of the piece on that square

        self.digital_board = [
            ['bR', 'bN', 'bB', 'bQ', 'bK', 'bB', 'bN', 'bR'],
            ['bp', 'bp', 'bp', 'bp', 'bp', 'bp', 'bp', 'bp'],
            ['00', '00', '00', '00', '00', '00', '00', '00'],
            ['00', '00', '00', '00', '00', '00', '00', '00'],
            ['00', '00', '00', '00', '00', '00', '00', '00'],
            ['00', '00', '00', '00', '00', '00', '00', '00'],
            ['wp', 'wp', 'wp', 'wp', 'wp', 'wp', 'wp', 'wp'],
            ['wR', 'wN', 'wB', 'wQ', 'wK', 'wB', 'wN', 'wR']
        ]
```

The image above shows the sequence of instructions that take place when the chessboard object is instantiated. The key for representing each square of the board in the 2D-array is the same as the one I came up with in the design phase (2.3.2). The `class_dictionary` attribute is empty for now, but would've been properly defined once the game begins.

Creating and Displaying the Chessboard and Pieces

```
def display_board(self, colour1, colour2):
    self.display_chessboard_pattern(colour1, colour2)
    self.display_labels(colour1, colour2)
    self.display_pieces()
```

The code below is within the main infinite loop, and illustrates how/when the display method for the chessboard is called within the program (`chessboard.display_board()` is ONLY called here and nowhere else) The method takes in two parameters, which dictate the colour scheme for the board.

```
if not game_active: #display main menu/post_game_report if false, display game if true
    if gamedata.is_checkmate or gamedata.is_stalemate: #after game is finished
        appendices.initiate_post_game()
    else: #before game begins
        main_menu.display()
else: #game is active, display chessboard
    chessboard.display_board(MINT, ARCTIC)
```

Implementing the chequered pattern

```
def display_chessboard_pattern(self, colour1, colour2): #input 2 colours, and a chequered pattern is created
    screen.fill(colour1)
    for row in range(cell_number):
        for col in range(cell_number):
            if ((row % 2) == 0 and (col % 2) == 0) or ((row % 2) == 1 and (col % 2) == 1):
                square_rect = pygame.Rect(row*cell_size, col*cell_size, cell_size, cell_size)
                pygame.draw.rect(screen, colour2, square_rect)
```

Nothing extraordinary here, just transforming the pseudocode from the design section into python code :) (2.1.1)

Displaying the labels

```
def display_labels(self, colour1, colour2): #creating the file labels 'a-h', and rank labels '1-8'
    file_order = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
    if gamedata.player_colour == 'w':
        rank_order = ['8', '7', '6', '5', '4', '3', '2', '1']
    else: #if player_colour = black
        rank_order = ['1', '2', '3', '4', '5', '6', '7', '8']

    for square in range(cell_number):
        if square % 2 == 1:
            label_colour = colour1
        else:
            label_colour = colour2

        file_surface, file_rect = appendices.text_schematic(file_order[square], square*cell_size,
            cell_number*cell_size, 20, label_colour, 'bottomleft')
        rank_surface, rank_rect = appendices.text_schematic(rank_order[square], cell_number*cell_size,
            square*cell_size, 20, label_colour, 'topright')
        screen.blit(file_surface, file_rect)
        screen.blit(rank_surface, rank_rect)
```

As the 720x720 `screen` will be covered by the entire chessboard, all that needed to be done in order to obtain the x-y coordinates for the label was to iterate through each row, and place the label to the bottomleft/topright of a square. (2.1.2/2.1.3) The list `rank_order` varies depending on whether the user is the white or black pieces because, as mentioned during the design phase, the perspective of the board differs for each colour (2.1.4)

Displaying the Chess Pieces

```
def display_pieces(self):
    if gamedata.player_colour == 'b':
        inverse_multiplier = 7 #used in order to optimise blacks perspective by vertically inverting the board
    else:
        inverse_multiplier = 0
    for row in range(cell_number):
        for col in range(cell_number):
            square = self.digital_board[row][col]
            if square != '00': #if the position is not an empty square
                piece_image = piece_images_dictionary[square]
                square_rect = piece_image.get_rect(topleft = (col*cell_size,abs(inverse_multiplier-row)*cell_size))
                screen.blit(piece_image,square_rect)
```

The code above conveys another application of the `pieces_images_dictionary`, which returns the png image of the piece on a particular square (2.2.1)

If you observe the topology of the `digital_board` attribute again, it is defined so that the black pieces are always at the top, and the white pieces are always at the bottom irrespective of which colour the user chooses. This is why the piece of maths `abs(inverse_multiplier-row)` exists, which calculates the row a chess piece needs to be for and only for when displayed, as the goal is to have the pieces belonging to the user initially being displayed at the bottom of the screen (2.2.2).

For instance if the user is black, its black pawns are fetched when `row` = 0. As a result, these pawns will be displayed to the user at the (7-0)th row (i.e. the 7th row, at the bottom of the screen).

As its supermodule `display_board` is continuously being run within the main infinite loop, this `display_pieces` method is also constantly being recalculated, so any updates to the `digital_board` (i.e. when a player/A.I makes a move) will result in a change in physical representation of the board. However when first run, the display will mimic the starting position of `digital_board` (2.2.3)

Defining Piece Characteristics using Complex OOP

Creating the chesspiece superclass

```
class chesspiece:
    def __init__(self, colour, position, signature, is_promoted, value):
        self.initial = colour[0]+signature
        self.colour = colour[0] #first letter of colour is used ('w' = white, 'b' = black)
        self.position = position
        self.move_count = 0
        self.is_promoted = is_promoted #If 'True', the chess piece is stemmed from a promoted pawn
        self.valid_move_list = []

        if self.colour == 'w':
            self.value = value
        elif self.colour == 'b': #black pieces possess negative value
            self.value = value * -1

    def calculate_moves(self, vectors, board, is_range, king_pos):...

    def is_checks(self, board, end_square, king_pos):...
```

The code above defines the chesspiece superclass, which consists of a set of attributes for each unique piece, as well as 2 methods which depend on these attributes in order to perform later calculations. The `position` attribute is an array (not a tuple) as it will likely be modified as pieces change position, and the `move_count` attribute can be interpreted to determine whether or not a piece has yet moved (i.e if `move_count` = 0, then `first_move` is true). When an object of a chess piece is created, the boolean `is_promoted` is set to 'False' in default, unless the object had previously been a pawn and has become promoted.

The role of the chesspiece superclass is so that its subclasses (e.g. queen, pawn objects) can inherit these attributes. Polymorphism will also be utilised as the `calculate_moves` method will be overridden by these subclasses.

Implementing the Calculate_moves() supermethod

```

def calculate_moves(self,vectors,board,is_range,king_pos):
    #this method will be overridden by most of its sub-classes
    valid_moves = [] #each piece of data will contain:(start_square,end_square),start_square = self.position
    row,col = self.position

    for vector in vectors:
        row_vector,col_vector = vector[0],vector[1]
        new_row,new_col = row+row_vector,col+col_vector

        if not is_range: #used for the king and knights, as they do not possess range
            if 0 <= new_row <= 7 and 0 <= new_col <= 7:
                if board[new_row][new_col][0] != self.colour:
                    #ensures that the new position isn't already occupied by a piece of the same colour
                    if not self.is_checks(board,(new_row,new_col),king_pos):
                        start_square = tuple(self.position)
                        end_square = (new_row,new_col)
                        valid_moves.append([start_square,end_square])

        else: #Used for the queen, rook and bishop, as they do possess range
            while 0 <= new_row <= 7 and 0 <= new_col <= 7:
                if board[new_row][new_col] != '00': #if not an empty square
                    if board[new_row][new_col][0] != self.colour: #if it is an enemy piece
                        if not self.is_checks(board,(new_row,new_col),king_pos):
                            start_square = tuple(self.position)
                            end_square = (new_row,new_col)
                            valid_moves.append([start_square,end_square])
                        break

                if not self.is_checks(board,(new_row,new_col),king_pos):
                    start_square = tuple(self.position)
                    end_square = (new_row,new_col)
                    valid_moves.append([start_square,end_square])
                new_row += row_vector
                new_col += col_vector

    return valid_moves

```

Nothing too new here, just translating the pseudocode from the design section into python code. The only fundamental change from the initial designed pseudocode is the addition of the `is_check` method call, which ensures that a potential move doesn't result in immediate exposure to the king (5.1.2). It follows the nature of the flowchart I designed earlier in order to solve the 'checking problem' (page 32). 'deepcopy' is imported from the copy module, and ensures that the original board is left unchanged when we simulate the move to be certified by the `check_for_checks` algorithm.

```
from copy import deepcopy
```

```

def is_checks(self,board,end_square,king_pos):
    #ensures that a potential valid move will NOT leave its own king being exposed

    duplicate_board = deepcopy(board)
    #this variable will be tweaked without changing the contents of the original board

    #Make the move on the duplicate board
    duplicate_board[end_square[0]][end_square[1]] = duplicate_board[self.position[0]][self.position[1]]
    duplicate_board[self.position[0]][self.position[1]] = '00'

    if duplicate_board[end_square[0]][end_square[1]][1] == 'K':
        king_pos = end_square[0],end_square[1] #position of king needs to be updated if king is moving

    return check_for_checks(duplicate_board,self.colour,king_pos)

```

Implementing the check for checks algorithm

```

def check_for_checks(board, colour, king_pos, get_updated_squares = False):
    changed_pieces = []
    king_row, king_col = king_pos
    #king_pos: position of the king to be inspected
    #colour: colour of the king to be inspected

    #For each possible vector traverse direction. If the matching opposite piece is within the path return
    #True (king is being checked)

    piece_vectors = [(1,0),(0,1),(-1,0),(0,-1), (1,1),(1,-1),(-1,1),(-1,-1),
                    (1,2),(2,1),(-1,2),(2,-1),(1,-2),(-2,1),(-1,-2),(-2,-1)]
    #1-4: rook,queen,king|5-8: bishop,queen,king|9-16: knight

    for vector_no, vector in enumerate(piece_vectors):
        row_vector, col_vector = vector[0], vector[1]
        king_new_row, king_new_col = king_row + row_vector, king_col + col_vector

        if 0 <= king_new_row <= 7 and 0 <= king_new_col <= 7:
            if (0 <= vector_no <= 7) and board[king_new_row][king_new_col][1] == 'K':
                changed_pieces.append((king_new_row, king_new_col))
                if board[king_new_row][king_new_col][0] != colour and not get_updated_squares:
                    return True # if an enemy king is adjacent to the king
            elif (8 <= vector_no <= 15) and board[king_new_row][king_new_col][1] == 'N':
                changed_pieces.append((king_new_row, king_new_col))
                if board[king_new_row][king_new_col][0] != colour and not get_updated_squares:
                    return True # if an enemy knight is an 'L' shaped distance away from the king
            if 8 <= vector_no <= 15:
                continue # vectors 9-16 are only for knights. so the code below can be skipped

        # while loop to check for ranged pieces only
        while 0 <= king_new_row <= 7 and 0 <= king_new_col <= 7:
            if board[king_new_row][king_new_col] != '00': # if not an empty square
                if (0 <= vector_no <= 3) and (board[king_new_row][king_new_col][1] == 'R' or
                board[king_new_row][king_new_col][1] == 'Q'):
                    changed_pieces.append((king_new_row, king_new_col))
                    if board[king_new_row][king_new_col][0] != colour and not get_updated_squares: #enemy piece
                        return True # if an enemy queen/rook is horizontal or vertical to the king
                elif (4 <= vector_no <= 7) and (board[king_new_row][king_new_col][1] == 'B' or
                board[king_new_row][king_new_col][1] == 'Q'):
                    changed_pieces.append((king_new_row, king_new_col))
                    if board[king_new_row][king_new_col][0] != colour and not get_updated_squares: #enemy piece
                        return True # if an enemy queen/bishop is diagonal to the king
                break

            #for each direction, keep traversing the paths for the king until the square is outside the board,
            #or there is a piece blocking the direction (also examine said piece)
            king_new_row += row_vector
            king_new_col += col_vector

    # check if pawns are attacking
    if colour == 'w':
        row_vector = -1 #black pawns will be on the top end of the board
    else:
        row_vector = 1 #white pawns will be on the bottom end of the board

    if 0 <= king_row + row_vector <= 7:
        col_vectors = [1,-1]
        for col in col_vectors:
            if 1 <= king_col + col <= 7:
                if board[king_row + row_vector][king_col + col][0] != colour:
                    if board[king_row + row_vector][king_col + col][1] == 'p':
                        return True

    return False # no checks identified

```

Ignore the lines highlighted in red for now: their purpose will be explained later. Apart from special cases (which will be explained later), for now always assume that the boolean `get_updated_squares` is to be False.

The reason why the `check_for_checks` algorithm is its own function, and not associated with any piece classes is because it has wider uses elsewhere (e.g. for evaluating checkmates/stalemates). The python code above implements the core conditions that needed to be inspected when determining if a king is in check, listed in the design phase (page 33)

Implementing subclass definitions

As I had already planned the class definitions for each piece in the design phase, all I needed to do was to format the pseudo code beforehand into python syntax.

`Value` is a static variable (or class variable), meaning that when modified, ALL instances of the class are affected. `Value` is formally defined once data for the difficulty for the game to be played has been inputted and confirmed by the user (as discussed in the analysis section, material values is dependent on game difficulty)

Rook Subclass

```
class rook(chesspiece):
    value = None

    def __init__(self, colour, position, is_promoted_piece = False):
        super().__init__(colour, position, 'R', is_promoted_piece, rook.value)

    def calculate_moves(self, board, king_pos):
        rook_vectors = [(1,0),(0,1),(-1,0),(0,-1)] #movement patterns for a rook
        self.valid_move_list = super().calculate_moves(rook_vectors, board, True, king_pos)
        return self.valid_move_list
```

Bishop Subclass

```
class bishop(chesspiece):
    value = None

    def __init__(self, colour, position, is_promoted_piece = False):
        super().__init__(colour, position, 'B', is_promoted_piece, bishop.value)

    def calculate_moves(self, board, king_pos):
        bishop_vectors = [(1,1),(1,-1),(-1,1),(-1,-1)] #movement patterns for a bishop
        self.valid_move_list = super().calculate_moves(bishop_vectors, board, True, king_pos)
        return self.valid_move_list
```

Queen Subclass

```
class queen(chesspiece):
    value = None

    def __init__(self, colour, position, is_promoted_piece = False):
        super().__init__(colour, position, 'Q', is_promoted_piece, queen.value)

    def calculate_moves(self, board, king_pos): #queen_moves = rook_moves + bishop_moves
        queen_vectors = [(1,0),(0,1),(-1,0),(0,-1),(1,1),(1,-1),(-1,1),(-1,-1)]
        self.valid_move_list = super().calculate_moves(queen_vectors, board, True, king_pos)
        return self.valid_move_list
```

Knight Subclass

```

class knight(chesspiece):
    value = None

    def __init__(self, colour, position, is_promoted_piece = False):
        super().__init__(colour, position, 'N', is_promoted_piece, knight.value)

    def calculate_moves(self, board, king_pos):
        knight_vectors = [(1,2), (2,1), (-1,2), (2,-1), (1,-2), (-2,1), (-1,-2), (-2,-1)]
        self.valid_move_list = super().calculate_moves(knight_vectors, board, False, king_pos)
        return self.valid_move_list

```

King Subclass

```

class king(chesspiece):
    value = MAX_SCORE #999999999

    def __init__(self, colour, position, is_promoted_piece = False):
        super().__init__(colour, position, 'K', is_promoted_piece, king.value)
        self.castled = False

    def calculate_moves(self, board, king_pos):
        king_vectors = [(1,0), (0,1), (0,-1), (-1,0), (1,-1), (-1,1), (1,1), (-1,-1)]
        standard_moves = super().calculate_moves(king_vectors, board, False, king_pos)
        castling_moves = self.castle(board)
        self.valid_move_list = castling_moves + standard_moves
        return self.valid_move_list

    def castle(self, board):...

```

As you can see, the king subclass is the only one whose static variable (`value`) has already been defined. Although it is set to the constant `MAX_SCORE`, in essence the material value of the king can be initialised to anything. This is because as the white and black kings both must always be present on the chessboard, thus their respective equal but opposite material values will always cancel out to zero.

Implementing the castling mechanic

```

def castle(self,board):
    castling_moves = []
    if self.move_count != 0: #illegal to castle if king has already moved
        return castling_moves

    row = self.position[0] #only row is required

    #castling queen side
    if (board[row][1] == '00') and (board[row][2] == '00') and (board[row][3] == '00'):
        #checks if squares between rook (queen side) and king are empty

        if board[row][0] == self.colour + 'R': #if rook on edge is same colour to king
            start_square = tuple(self.position)
            end_square = (row,2)
            castling_moves.append([start_square,end_square]) #add castle queen side to castling_moves

    #castling king side
    if (board[row][5] == '00') and (board[row][6] == '00'):
        # checks if squares between rook (king side) and king are empty

        if board[row][7] == self.colour + 'R':
            start_square = tuple(self.position)
            end_square = (row,6)
            castling_moves.append([start_square,end_square]) #add castle king side to castling moves

    return castling_moves

```

Pawn Subclass

```

class pawn(chesspiece):
    value = None

    def __init__(self,colour,position,is_promoted_piece = False):
        super().__init__(colour,position,'p',is_promoted_piece,pawn.value)
        self.capture_move_list = [] #used to evaluate centre control

    def calculate_moves(self,board,king_pos):
        if self.colour == 'w': #white pawns move up the board
            row_vector = -1
        else: #black pawns move down the board
            row_vector = 1

        standard_moves = self.move_vertical(board,row_vector,king_pos) #pawn is only moving vertically, no capturing
        self.capture_move_list = self.capture(board,row_vector,king_pos) #pawn is moving diagonally, capturing

        self.valid_move_list = self.capture_move_list + standard_moves
        return self.valid_move_list

    def move_vertical(self,board,row_vector,king_pos):...

    def capture(self,board,row_vector,king_pos):...

    def check_for_promote(self):...

```

```

def move_vertical(self, board, row_vector, king_pos):
    standard_moves = []
    row, col = self.position
    new_row = row + row_vector

    if board[new_row][col] == '00': #checks if square in front is empty
        if not self.is_checks(board, (new_row, col), king_pos):
            start_square = tuple(self.position)
            end_square = (new_row, col)
            standard_moves.append([start_square, end_square])
    else:
        return standard_moves #no need to look further if square in front is blocked

    if self.move_count == 0: #first move
        new_row = row + (row_vector * 2)
        if board[new_row][col] == '00':
            if not self.is_checks(board, (new_row, col), king_pos):
                start_square = tuple(self.position)
                end_square = (new_row, col)
                standard_moves.append([start_square, end_square])

    return standard_moves

```

```

def capture(self, board, row_vector, king_pos):
    capture_moves = []
    row, col = self.position
    new_row = row + row_vector
    col_vectors = [-1, 1]

    for col_vector in col_vectors:
        new_col = col + col_vector
        if 0 <= new_col <= 7 and 0 <= new_row <= 7:
            if board[new_row][new_col] != '00': #if not an empty square
                if board[new_row][new_col][0] != self.colour: #if it is an enemy piece
                    if not self.is_checks(board, (new_row, new_col), king_pos):
                        start_square = tuple(self.position)
                        end_square = (new_row, new_col)
                        capture_moves.append([start_square, end_square])

    return capture_moves

```

The method `check_for_promote` for the pawn object will be discussed shortly :)

Instantiating objects for the chess pieces

The method `initialize_pieces` belongs to the `chessboard` class, and is called only once when the game begins (inside the `begin_game` method in the `appendices` class, page 60).

```
def initialize_pieces(self):
    #initiating material scores for each piece, king value already defined
    if gamedata.difficulty == 'Expert':
        #these are the optimal values
        pawn.value = 1
        knight.value = 3
        bishop.value = 3
        rook.value = 5
        queen.value = 9
    if gamedata.difficulty == 'Intermediate':
        pawn.value = 1
        knight.value = 4
        bishop.value = 4
        rook.value = 4
        queen.value = 8
    if gamedata.difficulty == 'Beginner':
        pawn.value = 1
        knight.value = 3
        bishop.value = 3
        rook.value = 5
        queen.value = 5

    #create black major pieces
    self.class_dictionary[(0,0)] = rook('b',[0,0])
```

Firstly, the method uses the difficulty entered by the user in order to determine the set of values each piece should possess (by modifying the static variable `value` from 'none' to an integer for each piece). I didn't ponder too long regarding the exact specifics of these material numbers; as long as the values when playing as Intermediate and Beginner difficulty were less accurate and less truly representative than when playing as Expert, that would be enough.

```

#create black major pieces
self.class_dictionary[(0,0)] = rook('b',[0,0])
self.class_dictionary[(0,1)] = knight('b',[0,1])
self.class_dictionary[(0,2)] = bishop('b',[0,2])
self.class_dictionary[(0,3)] = queen('b',[0,3])
self.class_dictionary[(0,4)] = king('b',[0,4])
self.class_dictionary[(0,5)] = bishop('b',[0,5])
self.class_dictionary[(0,6)] = knight('b',[0,6])
self.class_dictionary[(0,7)] = rook('b',[0,7])

#create white major pieces
self.class_dictionary[(7,0)] = rook('w',[7,0])
self.class_dictionary[(7,1)] = knight('w',[7,1])
self.class_dictionary[(7,2)] = bishop('w',[7,2])
self.class_dictionary[(7,3)] = queen('w',[7,3])
self.class_dictionary[(7,4)] = king('w',[7,4])
self.class_dictionary[(7,5)] = bishop('w',[7,5])
self.class_dictionary[(7,6)] = knight('w',[7,6])
self.class_dictionary[(7,7)] = rook('w',[7,7])

#create black and white pawns
for create_pawn in range(cell_number):
    self.class_dictionary[(1,create_pawn)] = pawn('b',[1,create_pawn])
    self.class_dictionary[(6,create_pawn)] = pawn('w',[6,create_pawn])

```

After the material values for each piece have been finalised, I created object instances for each of the 32 pieces above, and prepended each object to the `class_dictionary`, where the key is a tuple of the position of the corresponding piece (e.g. at the start of the game, `class_dictionary[(7,7)]` returns the object of the white rook.)

Updating and Maintaining the Game using the Chessboard, Gamedata and Move_engine Classes

The chessboard, gamedata and move_engine classes can be described as the 'holy trinity' in the way that these three interact and synergise with each other in order to sustain the program once the game begins. We are already familiar with the functionality of the chessboard object, and now let's observe the operation of the gamedata and game_engine classes.

Introduction to the gamedata class

```
class gamedata: #this class stores all important information for the game that will be played
    def __init__(self):
        self.difficulty = '' #must be uppercase - first letter
        self.player_colour = '' # one letter, w or b
        self.ai_colour = ''

        self.white_king_pos = (7,4)
        self.black_king_pos = (0,4)
        self.game_phase = 'Opening' # o = opening, m = middlegame, e = endgame
        self.move_count = 0

        self.is_check = False

        self.is_checkmate = False
        self.is_stalemate = False

        self.white_move_list = []
        self.black_move_list = []
        self.current_valid_move_list = []

        self.move_log_stack = []
        self.capture_log_stack = []

        self.winner = ''

    def update(self, board, colour, class_dictionary, move_data, is_undo):...

    def obtain_all_valid_moves(self, board, colour, class_dictionary):...

    def update_gamestate(self, colour):...

    def update_gamephase(self, class_dictionary):...

    def set_is_check(self, board):...

    def log_captured_piece(self, capturing_square, class_dictionary):...

    def push_capture_log(self, captured_piece):...

    def pop_capture_log(self):...

    def push_move_log(self, move_data):...

    def pop_move_log(self):...
```

The screenshot above depicts all of the attributes and methods belonging to the gamedata class. The specifics regarding the operation and purpose of each method will be explored shortly.

Introduction to the move_engine class

```
class move_engine:
    @staticmethod
    def initiate_move(move_data):...

    @staticmethod
    def initiate_move_king(start_square,end_square,king_object):...

    @staticmethod
    def undo_move_king(start_square,end_square,king_object):...

    @staticmethod
    def pawn_promotion(end_square,pawn_object):...

    @staticmethod
    def check_depromote(end_square,piece_object):...

    @staticmethod
    def undo_move():...
```

The move_engine class only consists of static methods, and there will not be any instances and objects of the class. As briefly mentioned when introducing the appendices class, it is good practice to avoid instantiating objects unnecessarily as this slows execution times, which is especially paramount in a A.I themed chess game where any action to maximise time efficiency (even if minor) is worthwhile.

The specifics regarding the operation and purpose of each method will be explored shortly.

So now that the chessboard, gamedata and game_engine classes have been created, the chess pieces have been programmed to function and are positioned and displayed in their starting positions, all we need now in order to have a fully functional chess game (excluding A.I for now) is to let the user play.

In my chess game, the user can undergo 3 actions:

1. Click on the squares of the board to play a move
2. Undo a move
3. Call the program in order for it to display the best-move to the user at any point.

By far the most common action the user will do is to play a move, so let's begin with that.

Making a move

Implementing the move_data list

```
move_data = [] #contains 2 tuples: location of start and end squares clicked by the user
```

```
if event.type == pygame.MOUSEBUTTONDOWN: #user click
    mouse_pos = pygame.mouse.get_pos()
    if game_active:
        clicked_pos = appendices.get_col_row_from_pos(mouse_pos[0],mouse_pos[1])
        if move_data == []:
            move_data.append(clicked_pos)
        else:
            if clicked_pos == move_data[0]: #same square is clicked twice: deselect
                move_data = []
            else:
                move_data.append(clicked_pos)
```

The lines of code shown above are written inside the main infinite loop. The first screenshot shows the instantiation of the `move_data` list.

The second screenshot relays the process to how information given by the user is converted and written into the list. The operation to mapping a set of raw mouse coordinates into compatible chessboard coordinates using the `get_row_col_from_pos` method from the `appendices` class is as follows.

```
@staticmethod
def get_col_row_from_pos(x_pos,y_pos):
    if gamedata.player_colour == 'w':
        clicked_row = y_pos // cell_size
    else: #player_colour is black, so vertically invert 'clicked_row'
        clicked_row = 7 - (y_pos // cell_size)

    clicked_col = x_pos // cell_size

    return clicked_row,clicked_col
```

Just to reiterate, in my chess game a move is performed by the user by first clicking on the square of the piece it wants to move, followed by a click on the destination square of where the piece wants to move to.

```
if len(move_data) == 1: #user has clicked on a square once
    available_moves = chessboard.display_valid_moves(move_data[0])
elif len(move_data) == 2:
    if move_data in available_moves:
        #if the second square clicked is a possible destination from the first

        move_data = move_engine.initiate_move(move_data)
        chessboard.display_board(MINT,ARCTIC) #update board
        if gamedata.is_checkmate or gamedata.is_stalemate:
            game_active = False
    else: #if the second square clicked is independent from the first
        move_data.pop(0) #have the second square chosen be the first square now
```

Part 1: Displaying (and generating) all valid squares to the user

As a result of the statement made above, different actions are executed by the program depending on the length of `move_data`: if it is containing a single item then this means that the user has inputted a starting square, but not its destination square thus far. In order to assist the user, the role of the method `display_valid_moves` from the chessboard object is to indicate all possible destination squares (if any) the user is allowed to traverse to, from their picked starting square.

```
def display_valid_moves(self,clicked_pos):
    available_moves = self.generate_valid_moves(clicked_pos)
    #returns a list of moves, containing the start and end squares

    if gamedata.player_colour == 'b':
```

Of course, the program must fetch the data of the destination squares before they can be displayed, which is why the `generate_valid_moves` method is called.

```
def generate_valid_moves(self,clicked_pos):
    if self.digital_board[clicked_pos[0]][clicked_pos[1]][0] == current_turn:
        #user has clicked on one of their pieces

        piece_object = self.class_dictionary[(clicked_pos[0],clicked_pos[1])]
        if piece_object.colour == 'w':
            king_pos = gamedata.white_king_pos
        else:
            king_pos = gamedata.black_king_pos
        available_moves = piece_object.calculate_moves(self.digital_board,king_pos)
        return available_moves
    else: #user has clicked on an empty square or on a square of a piece it cannot control
        return []
```

The first line acts as input validation in the way that it ensures that the square the user entered is not empty AND contains a piece belongs to them (e.g. if user is playing as black, and clicks on the white pieces → `available_moves = []` → no moves displayed to the user). If the square is validated, the coordinates of the `clicked_pos` tuple is passed in as a key to the `class_dictionary`, in order to obtain the object of the piece at that square. Finally, the polymorphic method for the `piece_object` is called with the appropriate arguments so that a list of all valid moves for that piece is retrieved.

```
def display_valid_moves(self, clicked_pos):
    available_moves = self.generate_valid_moves(clicked_pos)
    #returns a list of moves, containing the start and end squares

    if gamedata.player_colour == 'b':
        inverse_multiplier = 7 #used in order optimise blacks perspective by vertically inverting the board
    else:
        inverse_multiplier = 0
    for move in available_moves:
        end_square_x = move[1][1]
        end_square_y = move[1][0]
        x_pos = (end_square_x * cell_size) + (cell_size // 2)
        y_pos = (abs(inverse_multiplier - end_square_y) * cell_size) + (cell_size // 2)
        #the 2 lines above provide the co-ordinates of the centre of each square the piece could go to
        pygame.draw.circle(screen, GREY, (x_pos, y_pos), 10)
        pygame.display.update()

    return available_moves
```

Above is a complete version for the `display_valid_moves` method. As each item in `available_move` comprises the starting square as well as the ending square, list manipulation had to be applied in order to obtain the x and y coordinates for the second tuple only in the list (as we are only interested in the ending squares for each `available_move`). Each x and y position on the chessboard needed to be rescaled to fit on the pygame screen, and I denoted each valid square by highlighting a grey circle on them.

Part 2: Executing the move by calling move_engine.initiate_move()

The image below is the same as the one 2 pages earlier (saves time flicking back and forth between :))

```

if len(move_data) == 1: #user has clicked on a square once
    available_moves = chessboard.display_valid_moves(move_data[0])
elif len(move_data) == 2:
    if move_data in available_moves:
        #if the second square clicked is a possible destination from the first

        move_data = move_engine.initiate_move(move_data)
        chessboard.display_board(MINT,ARCTIC) #update board
        if gamedata.is_checkmate or gamedata.is_stalemate:
            game_active = False
    else: #if the second square clicked is independent from the first
        move_data.pop(0) #have the second square chosen be the first square now

    #this also handles the situation when the first square entered by user is 'invalid'.
    #if invalid, available_moves list will be empty, so when the second item of move_data
    #is entered, the first invalid item will ALWAYS be popped

```

We have handled the scenarios when move_data only contains the starting square thus far. However when move_data is two items long (start and end square), before performing the move we must verify that the user has chosen one of the squares highlighted by a circle (valid squares). Otherwise we cancel the move (4.1.1)

move_engine.initiate_move()

```

@staticmethod
def initiate_move(move_data):
    #this is called to initiate a move. Everything in the program is updated here including:
    #1. Updating the gamedata attribute capture stack
    #2. updating the chessboard object
    #3. Initializing special moves such as castling and pawn promotions (if executed by player/A.I)
    #4. Updating the gamedata object
    #5. Flipping the current_turn
    #6. Emptying the contents of move_data

    global current_turn
    start_square = move_data[0]
    end_square = move_data[1]

    #1.
    gamedata.log_captured_piece(end_square,chessboard.class_dictionary)

    #2.
    chessboard.update(start_square,end_square,False)

    #3.
    piece_object = chessboard.class_dictionary[(end_square[0],end_square[1])]
    #gets object of piece that just moved
    if type(piece_object).__name__ == 'king': # if the piece_object moved is the king
        move_engine.initiate_move_king(start_square,end_square,piece_object)

    if type(piece_object).__name__ == 'pawn': # if the piece_object moved is a pawn, may promote
        move_engine.pawn_promotion(end_square,piece_object)

    #4.
    gamedata.update(chessboard.digital_board,current_turn,chessboard.class_dictionary,move_data,False)

    #5.
    current_turn = appendices.flip_current_turn(current_turn)

    #6.
    return []

```

This method is very important as not only is this called for the user when he/she enters a valid set of start/end squares, but will also be called thousands of times on behalf of the A.I in the minimax function. Detail into each specific step is elaborated below (other than steps 5 and 6 - they are self explanatory).

Step 2. The chessboard.update method

```

def update(self,start_square,end_square,is_undo):
    #updates all the common necessary objects and attributes, called every time a piece moves
    self.update_class_dictionary(start_square,end_square,is_undo)
    self.update_digital_board(start_square,end_square)

def update_class_dictionary(self,start_square,end_square,is_undo):
    self.class_dictionary[(end_square[0],end_square[1])] = self.class_dictionary[(start_square[0],start_square[1])]
    #transfer the piece object to its new square (overwrites any piece object on that square before)
    self.class_dictionary.pop((start_square[0], start_square[1])) #delete the piece object from its old square

    self.class_dictionary[(end_square[0],end_square[1])].position = [end_square[0],end_square[1]]
    #updates the attribute 'position' of the piece object

    #increments/decrements the attribute 'move_count' by 1
    if is_undo:
        self.class_dictionary[(end_square[0], end_square[1])].move_count -= 1
    else:
        self.class_dictionary[(end_square[0], end_square[1])].move_count += 1

def update_digital_board(self,start_square,end_square):
    self.digital_board[end_square[0]][end_square[1]] = self.digital_board[start_square[0]][start_square[1]]
    #new square marked as signature of new piece
    self.digital_board[start_square[0]][start_square[1]] = '00'
    #old square marked as empty

```

The second and third functions alter the `class_dictionary` and `digital_board` respectively (4.2.1), and are called by and only by the `update` method. If you notice the comment at the first line of the `update` method, by saying 'common attributes' I am referring to attributes in the piece classes that are ALWAYS going to change when they move (e.g. when a piece moves, their `position` and `move_count` attributes will always change. The attribute `is_promoted` is not 'common' as it likely will not be changing state every time it moves).

Step 3. Handling moves by the king and pawns, including castling and pawn promotion

```

@staticmethod
def initiate_move_king(start_square,end_square,king_object):
    # 1.check if castled
    if abs(end_square[1]-start_square[1]) == 2: # if the king has moved 2 squares to the right/left i.e.castled
        king_object.castled = True
        # rook also needs to move, which is what the lines below achieve
        if end_square[1] == 6: # if player castled king side
            rook_start_pos = (start_square[0],7)
            rook_new_pos = (start_square[0],5)
        else: # if player castled queen side
            rook_start_pos = (start_square[0],0)
            rook_new_pos = (start_square[0],3)
        chessboard.update(rook_start_pos,rook_new_pos,False)

    # 2.update white/black_king_pos attributes
    if king_object.colour == 'w':
        gamedata.white_king_pos = (end_square[0],end_square[1])
    else:
        gamedata.black_king_pos = (end_square[0],end_square[1])

```

```
@staticmethod
def pawn_promotion(end_square,pawn_object):
    if pawn_object.check_for_promote(): #pawn can promote. Replace pawn class object to queen
        chessboard.add_new_object(end_square,queen(pawn_object.colour,[end_square[0],end_square[1]],True))
        #final argument 'True' represents the fact that this object originates from a promoted pawn
```

The method `check_for_promote` is an additional method inherited by pawns only (as only pawns can promote), and is called every time a pawn moves.

```
def check_for_promote(self):
    #called every time a pawn is moved. If pawn at end row, then it should promote

    current_row = self.position[0]
    if self.colour == 'w': #the top row is the row where white pawns promote
        opposite_row = 0
    else: #the bottom row is the row where black pawns promote
        opposite_row = 7
    if current_row == opposite_row: #pawn is at opposite row of board
        return True

    return False
```

If `check_for_promote` returns True, then the pawn object is transformed into a **queen** object using the `add_new_object` method belonging to the chessboard object. Unlike the default queen objects, this one has the optional parameter `is_promoted_piece` to True, meaning that the attribute `is_promoted` will be True for this particular queen object (useful later in checking for depromotions).

```
def add_new_object(self,square,class_info):
    #creates new object in the class dictionary, and updates digital board
    self.class_dictionary[(square[0],square[1])] = class_info
    self.digital_board[square[0]][square[1]] = class_info.initial
```

Why Queen only?: Of course in chess a player has the right to promote a pawn into a rook, bishop or knight piece, as well as a queen. However, as my chess game is tailored to beginners, I concluded that by restricting the user's choices so that a promoted pawn is automatically transformed into a queen, will portray pawn promotion as a less sophisticated and arduous process. Automatic queen promotion can also encourage good habits to the user (as 99% of cases, pawns promoting to queens is the optimal move).

I relayed my intention about the automatic queen promotion concept to my beginner demographic third parties (friend Seb and brother Humza), and they both approved of this change.

Step 4. The gamedata.update method

```
def update(self, board, colour, class_dictionary, move_data, is_undo):
    #this method will be called after every move is completed, and inspects and updates
    #most of its attributes (if necessary)
    #colour is of player who has made move

    self.set_is_check(board)
    self.obtain_all_valid_moves(board, colour, class_dictionary, move_data)

    if not is_undo: #regular move
        self.move_count += 1
        self.push_move_log(move_data)
    else: #undo command
        self.move_count -= 1

    self.update_gamestate(colour)
    self.game_phase = self.update_gamephase(class_dictionary)
```

All 'common' changes made to the gamedata object are completed within this function (4.2.2). A detailed viewing of each sub-method inside (apart from `update_gamephase` - covered in A.I section later) are below.

Obtaining a list of all valid moves

```
def obtain_all_valid_moves(self, board, colour, class_dictionary):
    self.white_move_list = []
    self.black_move_list = []
    self.current_valid_move_list = [] #all moves of the next player AFTER the current move is made

    for piece_pos in class_dictionary:
        piece_object = class_dictionary[piece_pos]
        if piece_object.colour == 'w':
            king_pos = self.white_king_pos
        else:
            king_pos = self.black_king_pos
        piece_move_list = piece_object.calculate_moves(board, king_pos)
        #method call also modifies the 'valid_move_list' attribute for the piece

        if len(piece_move_list) != 0:
            if piece_object.colour == 'w':
                for move in piece_move_list:
                    self.white_move_list.append(move)
            else:
                for move in piece_move_list:
                    self.black_move_list.append(move)

    # we want all moves of the next player AFTER the current move is made
    if colour == 'w': #after white's turn, black move now
        self.current_valid_move_list = self.black_move_list
    else: #after black's turn, white move now
        self.current_valid_move_list = self.white_move_list
```

The generation of the `current_valid_move_list` serves two primary purposes:

1. Used in evaluating the possibility of checkmates and stalemates
2. Provides a set of readily available moves for the A.I to evaluate

Both of these applications will be demonstrated shortly.

Evaluating checks, checkmates and stalemates

```
def set_is_check(self, board):
    if check_for_checks(board, 'w', self.white_king_pos) or check_for_checks(board, 'b', self.black_king_pos):
        #check the safety of both kings

        self.is_check = True
    else:
        self.is_check = False
```

The code above outlines the application of the `check_for_checks` function we met earlier in order to calculate whether or not the current state of the board is in check. (5.1.1)

```
def update_gamestate(self, colour): #searches if there is a check/checkmate/stalemate
    self.is_checkmate = False
    self.is_stalemate = False

    if len(self.current_valid_move_list) == 0: #game has ended
        if self.is_check: #discovered earlier in another subroutine
            self.is_checkmate = True

            if colour == 'w': # white's turn when move was made
                self.winner = 'w'
            else: # black's turn when move was made
                self.winner = 'b'

        else:
            self.is_stalemate = True
```

Finally, using the attributes `is_check` and `current_valid_move_list` (both already updated as their methods were called before this one in the main `update` method), we can now truly analyse the state of the game.

Step 1 (and still 4). Introduction to the move and capture logs

The principles of operation regarding the attributes `move_log_stack` and `capture_log_stack` (both stored in the `gamedata` object) is that they are **stacks** (whoa!). They have to be operating as a stack because when we eventually undo an action, we have to obtain access to the most recent move, which will be the last/top item added into the data structure.

For now, let's observe how the program pushes items on top of the stack, which for the `move_log_stack` is rather simple. Here a 2 itemed tuple, containing positions of the start and end squares for the move that has just been played, are inserted into the `move_log_stack`. (both moves from player and A.I are included in order)

```
def push_move_log(self, move_data):
    self.move_log_stack.append(move_data)
```

This method is called within the `update` method and only within the `update` method, and only when the `move_engine` is processing a regular move (when the argument is `_undo = False`).

The operation of the `capture_log_stack` is slightly more complex.

```
def log_captured_piece(self, capturing_square, class_dictionary): #capturing square = end square
    if (capturing_square[0], capturing_square[1]) in class_dictionary:
        #if there is an object at the end square, not empty

        captured_piece = class_dictionary[(capturing_square[0], capturing_square[1])]
        self.push_capture_log(captured_piece)
        #object of captured piece is saved into the log (including its unique properties)
        #can be re-gathered and re-implemented into the game at any time
    else:
        self.push_capture_log('.') # '.' means that no piece was captured this turn

def push_capture_log(self, captured_piece):
    self.capture_log_stack.append(captured_piece)
```

I have designed the `capture_log_stack` so that a dot is inserted into the stack even if no piece had been captured on that turn, in order to sync the move count with any captured piece.

For instance, if `capture_log_stack = [ •, •, •, (unique pawn object), • ]` the program can identify that a pawn has been captured on move 4, and 2 moves from its current position.

Explanation to why `log_captured_piece` is performed at the beginning of `initiate_move` method: Ideally, I would've preferred to modify the capture and move logs at the same time, inside the `gamedata.update` method. This would've improved the readability of my code. However, I had to sacrifice this as it was imperative that `log_captured_piece` is called BEFORE the `chessboard.update` method (step 2.), due to the fact that the chessboard update method altered the contents of the `class_dictionary`. (the variable `capturing_square` will always be one of the keys in the `class_dictionary`, so the capture log will always just include the object of the moving piece).

Undo a move

move_engine.undo_move()

```

@staticmethod
def undo_move():
    #this is called to undo a move. Everything in the program is updated here including:
    #1. Access the previous move played from the move log held in the gamedata class
    #2. Updating the chessboard object (for piece moving back)
    #3. Recover the captured piece (if there is one for that turn)
    #4. De-castle (if necessary)
    #5. De-promote (if necessary)
    #6. Flipping the current turn
    #7. Updating the gamedata object
    global current_turn

    #1.
    previous_move = gamedata.pop_move_log()
    start_square = previous_move[1]
    end_square = previous_move[0]

    #2.
    chessboard.update(start_square,end_square,True)

    #3.
    recovered_piece = gamedata.pop_capture_log() #obtains the object of the last captured piece (if any)
    if recovered_piece != '.':
        chessboard.add_new_object(start_square,recovered_piece)

    #4.
    piece_object = chessboard.class_dictionary[(end_square[0],end_square[1])] #object of piece moving back
    if type(piece_object).__name__ == 'king': #if the piece_object moved is the king
        move_engine.undo_move_king(start_square,end_square,piece_object)

    #5.
    move_engine.check_depromote(end_square,piece_object)

    #6.
    current_turn = appendices.flip_current_turn(current_turn)

    #7.
    gamedata.update(chessboard.digital_board,current_turn,chessboard.class_dictionary,
        (start_square,end_square),True)

```

Additional comments:

- Whilst not as obvious as the `initiate_move` method, the implementation of any undo functionality is paramount. This is because the undo function will be used later on by the A.I in the minimax algorithm in order to back track from the moves it simulates, so that it can return back to the original node (original state of board/game).
- This function is similar to the `initiate_move` method in many ways. The majority of functions being called when making the move are still called here, with the only difference being the tweak in the `is_undo` parameter from False to True.
- Notice that unlike the `initiate_move` method, the global variable current turn is flipped BEFORE the gamedata object `update` method call. This is because the colour parameter for that update method should be the player who instigated the action (action = move/undo). For instance, if the user undoes a move while it is white's turn, it's actually the black piece that changes position → black instigated the action → black needs to be passed in as colour.
- Observing step 1 you can notice how the tuples `start_square` and `end_square` have been assigned to the reverse of what you usually would think. This is because the destination square when undoing a move is the same as the starting square when initially performing said move (those final two points may take a couple of reads to grasp that concept :))

All that is new left to you are the `undo_move_king` and `check_depromote` sub-methods.

Step 4. Handling undo moves for the king, including decastrling

```
@staticmethod
def undo_move_king(start_square,end_square,king_object):
    # 1.check if de-castled
    if abs(end_square[1]-start_square[1]) == 2: # if the king has moved 2 squares to the right/left i.e.castled
        king_object.castled = False

        #rook also needs to move, which is what the lines below achieve
        if start_square[1] == 6: #if player castled king side
            rook_start_pos = (start_square[0],5)
            rook_new_pos = (start_square[0],7)
        else: # if player castled queen side
            rook_start_pos = (start_square[0],3)
            rook_new_pos = (start_square[0],0)
        chessboard.update(rook_start_pos,rook_new_pos,True)

    # 2.update white/black_king_pos attributes
    if king_object.colour == 'w':
        gamedata.white_king_pos = (end_square[0],end_square[1])
    else:
        gamedata.black_king_pos = (end_square[0],end_square[1])
```

Step 5. Handling potential promotions

```
@staticmethod
def check_depromote(end_square,piece_object):
    if piece_object.is_promoted and piece_object.move_count == -1:
        #if the piece object stems from a pawn and hasn't moved yet

        chessboard.add_new_object(end_square,pawn(piece_object.colour,[end_square[0],end_square[1]]))
```

This method illustrates the application of the `is_promoted` attribute, as it is used to uniquely identify the fact that the piece being moved stemmed from a pawn, thus it has the ability to demote back to a pawn. The reason why the `move_count` attribute needing to equal -1 makes sense is that when the promoted piece is just created, `move_count` = 0 (any undo move after that decrements `move_count` from 0 to -1).

Allowing the user to call the undo method

```
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_u: # 'U' key is pressed = U = Undo move
        if len(gamedata.move_log_stack) != 0 and len(gamedata.move_log_stack) != 1:
            #cannot undo a move if no moves have taken place

            move_engine.undo_move()
            move_engine.undo_move() #called twice as A.I move also needs to be undone
            move_data = []
            game_active = True
```

This piece of code is within the infinite main loop of the program, allowing the user to perform an undo by pressing the 'U' key on the keyboard (4.3.1). The second 'if' statement acts as input sanitisation.

Testing the Functionality of the Game (2./3./4./5.)

It is paramount that the engine running my chess game is fully operational before implementing any A.I: (there's no point running the best A.I possible on a broken engine!) Therefore, thorough testing on this section especially is required to ensure that every aspect of my chess game engine is in working order.

Test Sequence 2

For simplicity, assume from test 09 onwards, the game is played in the perspective of white. During all tests, the A.I is picking a **random** move from the list of all valid moves (not testing A.I at this point in time).

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
05	1. Begin the game as white and observe opening screen	Normal: This test ensures that the perspective of the game is accurately displayed when user is playing as white	- The white pieces are displayed at the bottom of the screen - Files displayed from left to right: a,b,c,d,e,f,g,h - Ranks displayed from top to bottom: 8,7,6,5,4,3,2,1	0:00 - 0:42 Pass
06	1. Begin the game as black and observe opening screen	Normal: This test ensures that the perspective of the game is accurately displayed when user is playing as black	- The black pieces are displayed at the bottom of the screen - Files displayed from left to right: h,g,f,e,d,c,b,a - Ranks displayed from top to bottom: 1,2,3,4,5,6,7,8	0:42 - 1:17 Pass
07	1. Begin the game as black 2. Click on an empty square 3. Click on a square belonging to a white piece	Erroneous: Shows that the system only accepts inputs (and displays possible moves) when square clicked belongs to player	No observable changes occur for both actions	1:17 - 2:03 Pass
08	1. Begin the game as black 2. Click on the 'f7' pawn 3. Click on an empty square 4. Click on the 'f7' pawn 5. Click on the 'g7' pawn	Erroneous: Shows that in order for the program to execute a move, the end square inputted by the user must be dependent on the start square clicked. This test also shows that pawn movement is functional	- Squares of where the 'f7' pawn can go are displayed - Move is cancelled when empty square is clicked - Squares of where the 'f7' pawn can go are displayed again - Previous move is cancelled, and squares of where the 'g7' pawn can go is displayed	2:03 - 2:46 Pass

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
09	1. Pre-position a queen to the 'e4' square 2. Click on the 'e4' square	Normal: Shows that queen movement is functional	All squares where the 'e4' queen can go are displayed	2:46 - 4:25 Pass
10	1. Pre-position a rook to the 'e4' square 2. Click on the 'e4' square	Normal: Shows that rook movement is functional	All squares where the 'e4' rook can go are displayed	4:25 - 5:05 Pass
11	1. Pre-positon a bishop to the 'e4' square 2. Click on the 'e4' square	Normal: Shows that bishop movement is functional	All squares where the 'e4' bishop can go are displayed	5:05 - 5:17 Pass
12	1. Pre-position a knight to the 'e4' square 2. Click on the 'e4' square	Normal: Shows that knight movement is functional	All squares where the 'e4' knight can go are displayed	5:17 - 5:32 Pass
13	1. Pre-position the black queen to the 'h4' square 2. Click on the 'f2' white pawn	Erroneous: Shows that the program will invalidate any <i>potential</i> moves that result in immediate exposure to the king	No observable changes occur	5:32 - 5:57 Pass
14	1. Pre-position the white queen to the 'h4' square 2. Perform a regular move by moving the queen from the 'h4' square to the 'a4' square 3. Perform a capture by moving the queen from the 'a4' square to the 'a7' square	Normal: Shows that the game engine is functional by performing the 2 standard types of moves: a regular move and a capturing move	- Queen transfers from 'h4' to 'a4' - Queen transfers from the 'a4' square, and replaces the pawn on the 'a7' square - NOTE: The A.I will perform 2 <u>random</u> moves in between, to sync the current turn	5:57 - 6:36 Pass
15	1. Click the 'U' key twice	Normal: Shows that the undo functionality is working properly	- Queen moves back from 'a7' to 'a4', and the pawn on 'a7' is regenerated - Queen moves back from 'a4' to 'h4' - NOTE: The A.I will also undo both of his moves	6:36 - 7:20 Pass
16	1. Pre-position the white pawn to the 'g7' square 2. Promote the pawn by capturing the rook on the 'h8' square 3. Undo the move, which should demote the pawn	Normal: Tests the functionality of pawn promotion/demotion	- The black rook on 'h8' replaced by the newly-promoted white queen - After undo, rook is restored on 'h8', and pawn is returned to the 'g7' square	7:20 - 8:11 Pass

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
17	1. Empty the board, and pre-position the white king on 'e1', the white rooks on 'a1' and 'h1', the black rook on 'h8', and the black king at 'e8' 2. Click on the white king 3. Castle queen side 4. De-castle	Normal: Tests the functionality of castling/de-castling	• All valid moves for the king are correctly displayed (only queen side castling is allowed, not king side) • White king successfully castles queen side • White king and rook return to their standard position	8:11 - 9:17 Fail - King side castling incorrectly displayed
18	1. Pre-position the black pawn initially from the 'd7' square to the 'd5' square 2. Move the 'e' pawn two spaces 3. Perform a check by moving the 'f1' bishop to 'b5'	Normal: Shows that when a check occurs, the game engine will only allow moves to address the check	• A.I responds to check by either capturing bishop, moving the king or blocking bishop (NOTE: this is not a test on the A.I's smartness - as of now it is randomly picking from one of the <u>valid</u> moves)	10:20 - 11:10 Pass
19	1. From the starting board, move the 'e' pawn 2. Move the bishop from 'f1' to 'c4' 3. Move the queen from 'd1' to 'h5' 4. Capture the 'f7' pawn using the queen	Normal: The sequence of moves in the test data actually performs the 4 move scholar's mate! This test shows that the program can detect checkmates	• The pawn, bishop and queen pieces change position • After the queen captures, the game ends, and the program says that white wins	11:10 - 12:04 Pass
20	1. Empty the board, and pre-position the black king on 'h8', the white king on 'e1', and the white queen on 'g5' 2. Move the white queen from 'g5' to 'g6'	Normal: Shows that the system can detect stalemates	• After the queen moves the game ends, and the program says that it is a stalemate	12:04 - 12:38 Pass

Fixing the castling issue (TestID 17)

I traced down the error back to the castle method employed by the king object, which initial code is screenshotted below.

```

def castle(self,board):
    castling_moves = []
    if self.move_count != 0: #illegal to castle if king has already moved
        return castling_moves

    row = self.position[0] #only row is required

    #castling queen side
    if (board[row][1] == '00') and (board[row][2] == '00') and (board[row][3] == '00'):
        #checks if squares between rook (queen side) and king are empty

        if board[row][0] == self.colour + 'R': #if rook on edge is same colour to king
            start_square = tuple(self.position)
            end_square = (row,2)
            castling_moves.append([start_square,end_square]) #add castle queen side to castling_moves

    #castling king side
    if (board[row][5] == '00') and (board[row][6] == '00'):
        # checks if squares between rook (king side) and king are empty

        if board[row][7] == self.colour + 'R':
            start_square = tuple(self.position)
            end_square = (row,6)
            castling_moves.append([start_square,end_square]) #add castle king side to castling_moves

    return castling_moves

```

The issue is that currently there is no implementation of any checking validation (no use of the `is_checks` function).

```

def castle(self, board, king_pos):
    castling_moves = []
    if self.move_count != 0: #illegal to castle if king has already moved
        return castling_moves

    row = self.position[0] #only row is required

    #castling queen side
    if (board[row][1] == '00') and (board[row][2] == '00') and (board[row][3] == '00'):
        #checks if squares between rook (queen side) and king are empty

        if board[row][0] == self.colour + 'R': #if rook on edge is same colour to king
            end_square = (row, 2)
            if not self.is_checks(board, end_square, king_pos):
                start_square = tuple(self.position)
                castling_moves.append([start_square, end_square]) #add castle queen side to castling_moves

    #castling king side
    if (board[row][5] == '00') and (board[row][6] == '00'):
        # checks if squares between rook (king side) and king are empty

        if board[row][7] == self.colour + 'R':
            end_square = (row, 6)
            if not self.is_checks(board, end_square, king_pos):
                start_square = tuple(self.position)
                castling_moves.append([start_square, end_square]) #add castle king side to castling moves

    return castling_moves

```

The screenshot above shows the revised 'castling' algorithm, with the red boxes highlighting the changes made.

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
17 Take Two	1. Empty the board, and pre-position the white king on 'e1', the white rooks on 'a1' and 'h1', the black rook on 'h8', and the black king at 'e8' 2. Click on the white king 3. Castle queen side 4. De-castle	Normal: Tests the functionality of castling/de-castling	• All valid moves for the king are correctly displayed (only queen side castling is allowed, not king side) • White king successfully castles queen side • White king and rook return to their standard position	9:17 - 10:20 Pass

Access to test sequence 2 can be found by scanning the QR code, or by using the following link:

<https://youtu.be/eyOdEKWMg7Y>



Implementing the A.I (5./6./7./8./9.)

Commentary on this section will be limited as most concepts here have already been discussed in the design phase. :)

Implementing the gamephase algorithm

```
def update_gamephase(self, class_dictionary):
    white_king_object = class_dictionary[self.white_king_pos]
    black_king_object = class_dictionary[self.black_king_pos]

    if self.move_count < 10 and not(white_king_object.castled and black_king_object.castled):
        return 'Opening'

    major_piece_count = 0
    for piece_object in class_dictionary:
        piece_name = type(class_dictionary[piece_object]).__name__
        print(piece_name)
        if piece_name != 'king' and piece_name != 'pawn': #if the piece is major
            major_piece_count += 1

    print(major_piece_count)
    if major_piece_count > 3:
        return 'Middlegame'

    return 'Endgame'
```

The algorithm above is a method belonging to the gamedata class, and represents the flowchart initially created in the design phase (page 37). A reminder that the purpose for calculating the gamephase is that it determines the weights of each of the components in the evaluation function.

Introduction to the A.I class

```
class AI:
    def __init__(self, move_engine_class, gamedata_object, chessboard_object):
        self.gamedata_object = gamedata_object
        self.chessboard_object = chessboard_object
        self.move_engine = move_engine_class
        self.ai_colour = self.gamedata_object.ai_colour
        self.difficulty_depth = 0 #not defined yet
        self.hint_depth = 3 # highest depth available: we want most accurate move as hint for user
        self.max_depth = 0 #changes per minimax call - depending on whether minimax call is normal or for hint

        self.best_move = [] #stores only a single tuple - the best move currently found by minimax

    def initialize_depth(self): #called after user picks difficulty from main menu
        if self.gamedata_object.difficulty == 'Expert':
            self.difficulty_depth = 3
        elif self.gamedata_object.difficulty == 'Intermediate':
            self.difficulty_depth = 2
        else: #difficulty = beginner
            self.difficulty_depth = 1

    def generate_move(self, generate_hint = False):...

    def evaluate_material_score(self, piece_object):...

    def evaluate_centre_control(self, piece_object):...

    def evaluate_pawn_structure(self, board, piece_object):...

    def evaluate_king_safety(self, board):...

    def calculate_total_score(self, material_score, centre_control_score, pawn_structure_score, king_safety_score):...

    def minimax(self, board, is_maximising, depth, alpha, beta):...
```

The screenshot above depicts all of the attributes and methods belonging to the A.I class. In my program the A.I class actually stores other classes and instances we have met before as its own attributes, and for good reason:

- gamedata_object possesses key attributes (e.g. gamespahse), to be used by the evaluation function
- move_engine allows the A.I to initiate and undo moves when traversing each node in the minimax tree
- chessboard_object contains the digital_chessboard 2D-array, which is fundamental for use in the evaluation function (the class_dictionary attribute is also widely used)

The method `initialize_depth` defines the `difficulty_depth` attribute to be passed into the minimax function (7.1.1), and is called only once when the game begins (inside the `begin_game` method in the appendices class, page 60).

In the `__init__` function, the colour of the A.I is also defined here as `ai_colour` (7.1.2).

The specifics regarding the operation and purpose of each method will be explored shortly.

Implementing the Evaluation Function

NOTE: For the screenshots in this section, specific component values (e.g. pawn_island_penalty, castling_bonus) may not be their final form in the completed code version. Their official values will be defined at the end of testing :)

```
def evaluation_function(self,board):
    material_score = 0
    centre_control_score = 0
    pawn_structure_score = 0

    """
    * loop through every square on board. For each square:
    #1. Add/subtract to material score (if there is a piece on the square)
    #2. Add/subtract to centre control score if piece controls any central squares
    #3. Add/subtract to pawn structure score if piece is a pawn (evaluate its position)

    * Afterwards, evaluate the safety of the white and black kings, which comprise the king safety score

    * Calculate the weights of each components, depending on the phase of the game
    """

    for row_pos,row in enumerate(board): #index of position also obtained using enumerate
        for col_pos,piece in enumerate(row):
            if piece != '00':
                piece_object = self.chessboard_object.class_dictionary[(row_pos,col_pos)]
                material_score += self.evaluate_material_score(piece_object)
                centre_control_score += self.evaluate_centre_control(piece_object)
                pawn_structure_score += self.evaluate_pawn_structure(board,piece_object)

    king_safety_score = self.evaluate_king_safety(board)
    total_score = self.calculate_total_score(material_score,centre_control_score,pawn_structure_score,
    king_safety_score)

    return total_score
```

The algorithm above is the main evaluation function, which calls all of its secondary modules that comprise an evaluation. The operation of the python code above simulates the flowchart initially created in the design phase (page 38). As you can see, the board (digital_board from chessboard object) is passed into the function. Each sub-module called within here will be explained shortly.

Evaluating the material score

```
def evaluate_material_score(self,piece_object):
    return piece_object.value
```

We have already seen the initialization of the `value` attribute.

Evaluating the centre control score

```
def evaluate_centre_control(self, piece_object):
    central_squares = [(3,3), (3,4), (4,3), (4,4)]
    piece_in_centre_bonus = 3 #can be tweaked for optimisation
    pawn_controlling_centre_bonus = 2
    non_pawn_controlling_centre_bonus = 1

    if piece_object.colour == 'w':
        row_vector = -1
        multiplier = 1
    else: #Black piece
        row_vector = 1
        multiplier = -1

    if tuple(piece_object.position) in central_squares: #if the piece is already in the centre
        return piece_in_centre_bonus * multiplier

    if type(piece_object).__name__ == 'pawn': #pawns are more valuable to control centre, thus more points
        value = pawn_controlling_centre_bonus
        watched_squares = piece_object.capture('filler', row_vector, 'filler', True)
        #pawns can only control squares diagonal to them, which is why we use the capture method
    else:
        value = non_pawn_controlling_centre_bonus
        #other pieces score less points than pawns if they control the centre
        watched_squares = piece_object.valid_move_list

    for square in watched_squares:
        if square[1] in central_squares: #if one of the destination squares is a central square
            return value * multiplier

    return 0 #pieces is not controlling any central squares
```

Implementing the initial 'centre control' pseudocode from the design section (page 40) into python syntax was straightforward bar one exception: originally I did not fully consider the complexity regarding the influence of squares controlled by pawns. You see, the squares accessed vertically by the pawn cannot be considered as 'controlling squares', as the entire purpose regarding controlling certain squares is that if any enemy piece positions itself on said square, the initial piece controlling the square can immediately capture it (pawns cannot capture by moving vertically!).

Therefore, in order for a pawn to be deemed as controlling a square, the square must be diagonally relative to the pawn. Fortunately, we already have created a method similar to this requirement for the pawn object...

Renovating the capture method

```

def capture(self, board, row_vector, king_pos, centre_scanning = False):
    central_moves = []
    capture_moves = []
    row, col = self.position
    new_row = row + row_vector
    col_vectors = [-1, 1]

    for col_vector in col_vectors:
        new_col = col + col_vector
        if 0 <= new_col <= 7 and 0 <= new_row <= 7:
            start_square = tuple(self.position)
            end_square = (new_row, new_col)

            if centre_scanning: #doesn't matter whether diagonal square is filled or not
                central_moves.append([start_square, end_square])
            else:
                if board[new_row][new_col] != '00': #if not an empty square
                    if board[new_row][new_col][0] != self.colour: #if it is an enemy piece
                        if not self.is_checks(board, end_square, king_pos):
                            capture_moves.append([start_square, end_square])

    if centre_scanning:
        return central_moves

    return capture_moves

```

Above shows the updated version of the capture method in the pawn class, with the red lines/boxes highlighting the key new changes made. `centre_scanning` is another example of an optional parameter, and is only ever set to true within the 'centre control' method. The key difference between the `central_moves` and `capture_moves` lists is that diagonal manoeuvres are always added to `central_moves`, independent of whether or not the `end_square` is empty (which meets our requirements when evaluating centre control).

```

watched_squares = piece_object.capture('filler', row_vector, 'filler', True)
#pawns can only control squares diagonal to them, which is why we use the capture method

```

Above is just a zoomed in screenshot for the `evaluate_centre_control` method, illustrating how the newly written capture method is called. The parameters `board` and `king_pos` are not required in this instance, and the `row_vector` determining which direction the pawn travels is defined earlier in the method.

Evaluating pawn structure score

```

def evaluate_pawn_structure(self, board, piece_object):
    if type(piece_object).__name__ != 'pawn': #no need to be evaluated if piece is not a pawn
        return 0

    pawn_structure_score = 0 #can be tweaked for optimisation
    pawn_island_penalty = -2
    double_pawn_penalty = -4 #each pawn on same row e.g. 3 pawns on same row = -8
    pawn_island = True

    if piece_object.colour == 'w':
        multiplier = 1
    else: #Black piece
        multiplier = -1

    row, col = piece_object.position
    surrounding_squares = [(0,1),(0,-1),(1,0),(-1,0),(1,1),(1,-1),(-1,1),(-1,-1)]

    for square in surrounding_squares:
        new_row, new_col = row+square[0], col+square[1]
        if 0 <= new_row <= 7 and 0 <= new_col <= 7:
            if board[new_row][new_col] == piece_object.initial:
                #this means that there is a friendly pawn next to this one, NOT isolated
                pawn_island = False
                break

    if pawn_island:
        pawn_structure_score += pawn_island_penalty

    while row < 7: #only need to look one way, if its below one pawn, then its above another pawn
        if board[row+1][col] == piece_object.initial: #another pawn on same row, doubled (or worse)
            pawn_structure_score += double_pawn_penalty
            break
        row += 1

    return pawn_structure_score * multiplier

```

Implementing the initial 'pawn structure' pseudocode from the design section (page 41) into python syntax.

Evaluating king safety score

```

def evaluate_king_safety(self, board):
    king_safety_score = 0

    white_king_pos = self.gamedata_object.white_king_pos
    white_king_object = self.chessboard_object.class_dictionary[white_king_pos]
    black_king_pos = self.gamedata_object.black_king_pos
    black_king_object = self.chessboard_object.class_dictionary[black_king_pos]

    col_vectors = [-1,0,1]
    castling_bonus = 3 #can be tweaked for optimisation
    pawn_shield_bonus = 1 #per pawn

    #deduce if kings have yet castled
    if white_king_object.castled: #white king has castled
        king_safety_score += castling_bonus
    if black_king_object.castled: #black king has castled
        king_safety_score -= castling_bonus

    #checking the presence of pawn shield
    for col_vector in col_vectors:
        #check the white pawn shield first
        new_row = white_king_pos[0] - 1
        new_col = white_king_pos[1] + col_vector
        if 0 <= new_row <= 7 and 0 <= new_col <= 7:
            if board[new_row][new_col] == 'wp':
                king_safety_score += pawn_shield_bonus

        #now check the black pawn shield
        new_row = black_king_pos[0] + 1
        new_col = black_king_pos[1] + col_vector
        if 0 <= new_row <= 7 and 0 <= new_col <= 7:
            if board[new_row][new_col] == 'bp':
                king_safety_score -= pawn_shield_bonus

    return king_safety_score

```

Implementing the initial 'king safety' pseudocode from the design section (page 43) into python syntax. For clarity, `new_row` acts as the 'white_king_row_pos/black_king_row_pos' variable mentioned in the initial pseudocode in the way that it stores the row where the white/black pawn shield should be at.

Calculating the total score

```

def calculate_total_score(self,material_score,centre_control_score,pawn_structure_score,king_safety_score):
    multipliers = {'Opening':[2,0.7,0.2,0.15], 'Middlegame':[2,0.2,0.3,0.5], 'Endgame':[2,0,0.6,0]}
    #multipliers can be tweaked for optimisation
    #multipliers left to right: material score, centre control, pawn structure, king safety

    material_multiplier,centre_control_multiplier,pawn_structure_multiplier,king_safety_multiplier = \
    multipliers[self.gamedata_object.game_phase]

    material_score *= material_multiplier
    centre_control_score *= centre_control_multiplier
    pawn_structure_score *= pawn_structure_multiplier
    king_safety_score *= king_safety_multiplier

    total_score = material_score + centre_control_score + pawn_structure_score + king_safety_score
    total_score = round(total_score,2) #total score is rounded to 2 d.p

    return total_score

```

Implementing the initial 'calculate total score' pseudocode from the design section (page) into python syntax.

Implementing the Minimax Algorithm (w/ alpha-beta pruning)

```
def minimax(self, board, is_maximising, depth, alpha, beta): #board = the main board object
    # A.I is maximising and wants highest score if white, minimizing and wants lowest score if black

    if self.gamedata_object.is_checkmate: #game has ended - return score
        if self.gamedata_object.winner == 'w': #white wins
            score = MAX_SCORE
        elif self.gamedata_object.winner == 'b': # black wins
            score = MIN_SCORE
        else: #stalemate
            score = 0

        return score

    if depth == 0: #maximum depth reached
        return self.evaluation_function(board) #Evaluate the current board and find its current score

    ...

    Iterate through all available moves. For each move, apply the following steps:
    step 1. Perform the move on the board
    step 2. Call the function recursively, decrementing the depth and flipping the isMaximising variable
    step 3. Undo the move on the board
    step 4. If the score is higher/lower than the best score (depending on who is maximising),
    replace the best score and store that move (if move is at highest depth)
    step 5. Check if alpha >= beta. If true, prune all necessary moves
    ...

    if is_maximising: #white is maximising, wants highest score
        best_move_score = MIN_SCORE #we start with the worst possible score for white
        for available_move in self.gamedata_object.current_valid_move_list:
            self.move_engine.initiate_move(available_move) #step 1.
            score = self.minimax(self.chessboard_object.digital_board, False, depth-1, alpha, beta) #step 2.
            self.move_engine.undo_move() # step 3.
            if score > best_move_score: #step 4.
                best_move_score = score
                if depth == self.max_depth:
                    self.best_move = available_move #new optimal move found, and is stored here

            alpha = max(alpha, best_move_score) #step 5.
            if beta <= alpha:
                break

    if not is_maximising: #black is minimising, wants lowest score
        best_move_score = MAX_SCORE #we start with the worst possible score for white
        for available_move in self.gamedata_object.current_valid_move_list:
            self.move_engine.initiate_move(available_move) #step 1.
            score = self.minimax(self.chessboard_object.digital_board, True, depth-1, alpha, beta) #step 2.
            self.move_engine.undo_move() #step 3.

            if score < best_move_score: # step 4.
                best_move_score = score
                if depth == self.max_depth:
                    self.best_move = available_move #new optimal move found, and is stored here

            beta = min(beta, best_move_score) #step 5.
            if beta <= alpha:
                break

    return best_move_score
```

This is the full recursive minimax algorithm that searches through a tree of moves, with the implementation of alpha-beta pruning highlighted in red. The code above is heavily inspired by the pseudocodes I produced in the design section (pages 45-48)], and there's not much more to say that hasn't already been said!

Implementing move ordering to optimise alpha-beta pruning

```

def order_moves(self, board):
    ordered_move_queue = []
    capturing_moves = []
    central_moves = []
    king_moves = []
    remaining_moves = []

    central_squares = [(3,3),(3,4),(4,3),(4,4)]

    for move in self.current_valid_move_list:
        start_square = move[0]
        piece_in_start_square = board[start_square[0]][start_square[1]]
        end_square = move[1]
        piece_in_end_square = board[end_square[0]][end_square[1]]

        if piece_in_end_square != '00': #if move is a capturing move
            capturing_moves.append(move)
        elif end_square in central_squares: #if piece is moving to centre
            central_moves.append(move)
        elif piece_in_start_square[1] == 'K': #if the king is moving
            king_moves.append(move)
        else: #all other moves that were not any of the above
            remaining_moves.append(move)

    for capturing_move in capturing_moves:
        ordered_move_queue.append(capturing_move) #enqueue capturing moves first
    for central_move in central_moves:
        ordered_move_queue.append(central_move) #enqueue central moves second
    for remaining_move in remaining_moves:
        ordered_move_queue.append(remaining_move) #enqueue remaining moves third
    for king_move in king_moves:
        ordered_move_queue.append(king_move) #enqueue king moves last

    return ordered_move_queue

```

The screenshot above implements the idea of queues (`ordered_move_queue`) first mentioned in the design phase (page [insert page]) so that the `current_valid_move_list` is approximately sorted from the best to worst move when being searched by the minimax function (8.1.1).

Calling order_moves()

```

def update(self, board, colour, class_dictionary, move_data, is_undo):
    #this method will be called after every move is completed, and inspects and updates
    #most of its attributes (if necessary)
    #colour of player who has made move
    self.set_is_check(board)
    self.obtain_all_valid_moves(board, colour, class_dictionary, move_data)

    if not is_undo: #regular move
        self.move_count += 1
        self.push_move_log(move_data)
        self.current_valid_move_list = self.order_moves(board)
    else: #undo command
        self.move_count -= 1

    self.update_gamestate(colour)
    self.game_phase = self.update_gamephase(class_dictionary)

```

The sorting method `order_moves` is called under the update method for the gamedata object, in order to officially transform the previously arbitrary `current_valid_move_list` into a sorted queue of available moves. Notice that the function is only called ONLY during a 'forwarding' move; this is to further improve time efficiency as ordering the move list after undoing a move would be redundant (the move list would have already been determined for that particular branch in the tree by then).

Generating the best move by calling the minimax function

```
def generate_move(self, generate_hint = False):
    self.best_move = []

    if self.gamedata_object.ai_colour == 'w': #white = maximizing
        is_maximising = True
    else: #black = minimizing
        is_maximising = False

    if generate_hint:
        is_maximising = not is_maximising #e.g. if A.I = white, player = black, so needs to be False
        self.max_depth = self.hint_depth #highest depth available: we want most accurate move as hint for user
    else:
        self.max_depth = self.difficulty_depth #depends on difficulty of A.I

    #call minimax
    score = self.minimax(self.chessboard_object.digital_board, is_maximising, self.max_depth, MIN_SCORE, MAX_SCORE)
    #we don't actually need the score, but may be handy for testing later

    best_move = self.best_move #the best_move attribute is altered during the evaluation function
    return best_move
```

In order to execute the base call for the minimax function, 5 parameters are required:

1. Board: Equivalent to the `digital_board` attribute belonging to the chessboard object
2. `is_maximising`: Determined by the colour of the A.I (7.1.2)
3. Depth: As this is the base call, this is equivalent to the maximum depth the A.I is willing to search to. The variable `max_depth` is defined in the init method, and is dependent on the difficulty the A.I has been set to by the user (7.1.1)
4. Alpha: During base call, this is initially passed as `MIN_SCORE`, a constant representing the lowest possible value
5. Beta: During base call, this is initially passed as `MAX_SCORE`, a constant representing the highest possible value

After minimax is called, the tuple `best_move` (chosen from the `current_valid_move_list` queue) is finalised and returned (9.1).

Calling `generate_move()` within the main loop

```
if current_turn != gamedata.player_colour: #AI's turn
    move_data = ai.generate_move()
    move_data = move_engine.initiate_move(move_data)
    if gamedata.is_checkmate or gamedata.is_stalemate:
        game_active = False
```

Nice and simple :)

Hint move

Adding a new method and attribute to the chessboard object

```
def display_hint(self):
    if self.hint_move == []:
        return

    if gamedata.player_colour == 'b': #used in order to optimise blacks perspective by vertically inverting the board
        inverse_multiplier = 7
    else:
        inverse_multiplier = 0

    start_square_x = self.hint_move[0][1] * cell_size
    start_square_y = abs(inverse_multiplier-self.hint_move[0][0]) * cell_size
    end_square_x = self.hint_move[1][1] * cell_size
    end_square_y = abs(inverse_multiplier-self.hint_move[1][0]) * cell_size

    #shade the start and end squares of the hint move in orange
    start_square_rect = pygame.Rect(start_square_x, start_square_y, cell_size, cell_size)
    pygame.draw.rect(screen, ORANGE, start_square_rect)

    end_square_rect = pygame.Rect(end_square_x, end_square_y, cell_size, cell_size)
    pygame.draw.rect(screen, ORANGE, end_square_rect)
```

The newly defined `display_hint` method will be continuously running throughout the program (because `chessboard.display` is run continuously within the main infinite loop). However, as the new attribute `hint_move` for the chessboard object is empty the majority of the time, this method will bear no effect (thus represented by the null return statement).

```
if event.type == pygame.KEYDOWN and game_active:
    if event.key == pygame.K_u: # 'U' key is pressed = U = Undo move
        if len(gamedata.move_log_stack) != 0 and len(gamedata.move_log_stack) != 1:
            #cannot undo a move if no moves have taken place
            move_engine.undo_move()
            move_engine.undo_move() #called twice as A.I move also needs to be undone
            move_data = []
            game_active = True
    elif event.key == pygame.K_h and chessboard.hint_move == []: # 'H' key is pressed = H = Show hint
        chessboard.hint_move = ai.generate_move(True)
```

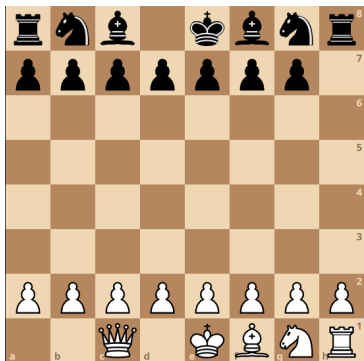
The screenshot above is an extension of the keyboard input portion found in the infinite main loop (first encountered in page 88). In essence, if the user enters the 'H' key during the game, the attribute `hint_move` is modified to a tuple representing the best possible move for the user, allowing the chessboard method `display_hint` to properly be executed (as `best_move` is no longer empty). (4.4.2). In this instance for the `generate_move` call, the optional parameter `generate_hint` is set to 'True', which ensures that the minimax function is called in the perspective of the colour of the user, at the highest depth available in order to optimise accuracy (4.4.1).

After 'H' is pressed, the highlighted squares are shaded in orange until the user initiates its next move (I modified `best_move` back to an empty list in order to remove highlighted squares).

Testing the A.I (5./6./7./8./9.)

Assume that in all cases in this test sequence unless specified, we test the A.I in its most optimal state (expert), and all positions are in the perspective of the white user.

Test Sequence 3

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
21	1. Begin the game under beginner difficulty (as the Beginner A.I is quick and aggressive) 2. Play 10 random moves between yourself and A.I 3. Undo the 10th move 4. Capture enough pieces so that only 3 major pieces are left 5. Undo the final capture move	Normal: This test ensures that the program calculates the phase of the game accurately	• Gamephase: Opening → Middlegame after 10 moves have been made • Gamephase: Middlegame → Endgame when there are less than 3 major pieces on board • Gamephase reverses when appropriate undo actions are made	0:00 - 1:09 Pass NOTE: This Test ID includes a fast-forward
22	1. Call the material comparison function for the following board: 	Normal: Shows that the program calculates the net material score accurately, and that the value attribute for each piece is correct Standard values for material are used (queen = 9, rook = 5, bishop/knight = 3, pawn = 1)	• Program returns a net material score of -1 (-5-3-3+9+1)	1:09 - 1:23 Pass
23	1. Call the pawn structure function for the following board: 	Normal: Shows that the program calculates the net material score accurately (specific individual values not important for now) For now, pawn_island_penalty = -2, and doubled_pawn_penalty = -4	• Program returns a net pawn structure score of -8 (-4 -2 -2)	1:23 - 1:36 Pass

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
24	1. Call the centre control function for the following board: 	Normal: Shows that the program calculates the net centre control score accurately (specific individual values not important for now) For now, piece_in_centre_bonus = 3, pawn_control_bonus = 2, non_pawn_control_bonus = 1	Program returns a net centre control score of +3 (3-3+2+1)	1:36 - 1:51 Pass
25	1. Call the king safety function for the following board: 	Normal: Shows that the program calculates the net king safety score accurately (specific individual values not important for now) For now, pawn_shield_bonus = 1 castling_bonus = 3	Program returns a net king safety score of +5 (1+1+1-1+3)	1:51 - 2:04 Pass

Tuning the numerical evaluation values correctly

Now that we've proved that the program successfully can identify all possible elements that measure the robustness of a chess position for both players, and that it detects the phase of the game accurately, the next step is to calibrate the numerical values that constitute these very elements so that each aspect of an evaluation is weighted correctly.

I decided that a trace table is the more suitable approach to test the weighting of each component instead of the traditional video, as there will be a lot of minor changes made between each row. (say material is constant)

Tuning the individual values

The numerical values for each individual component of the evaluation are constant even throughout different game phases. Therefore, the purpose of the test below is to observe the relationship of values that evaluate the same aspect of the board (e.g. to check that castling bonus and pawn shield bonus relative to each other are weighted finely when evaluating king safety). Note that we are NOT testing the values for each material piece: this is because these are internationally proven to be the most accurate figures.

All values listed in the table below are applied to the position shown to the right (this position is suitable as it represents a disparity in all components in some shape or form).

Key:

- P.I.C.B = Piece In Centre Bonus
- P.C.C.B = Pawn Controlling Centre Bonus
- N.P.C.C.B = Non Pawn Controlling Centre Bonus
- C.C.S = Centre Control Score
- P.I.P = Pawn Island Penalty
- D.P.P = Doubled Pawn Penalty
- P.S.S = Pawn Structure Score
- C.B = Castling Bonus
- P.S.B = Pawn Shield Bonus
- K.S.S = King Safety Score



Row no.	P.I.C.B	P.C.C.B	N.P.C.C.B	C.C.S	P.I.P	D.P.P	P.S.S	C.B	P.S.B	K.S.S	Comments
1	3	2	1	+4	-2	-4	+6	3	2	-7	P.I.P too high relative to D.P.P (i.e. P.I.P/D.P.P ratio too high)
2	3	2	1	+4	-1	-3	+4	3	2	-7	P.C.C.B too low relative to N.P.C.C.B (i.e P.C.C.B/N.P.C.C.B ratio too low)
3	6	5	2	+8	-1	-3	+4	3	2	-7	P.S.B too high relative to C.B. (i.e P.S.B/C.B ratio too high)
4	6	5	2	+8	-1	-3	+4	6	3	-12	Satisfactory values

At this point in time I wouldn't bother inspecting the values of C.C.S, P.S.S and K.S.S in too much detail. For now, it is important to acknowledge that the individual values are well in sync with each other.

Tuning the component multipliers for each gamephase

From the tests we have just carried out, I can confidently say an accurate set of individual values that will be used in my program for all gamephase are as follows:

- piece_in_centre_bonus = 5
- pawn_controlling_centre_bonus = 5
- non_pawn_controlling_centre_bonus = 2
- pawn_island_penalty = -1
- doubled_pawn_penalty = -3
- castling_bonus = 6
- pawn_shield_bonus = 3

Key:

- M.M = Material Multiplier
- F.M.S = Final Material Score
- C.C.M = Centre Control Multiplier
- F.C.C.S = Final Centre Control Score
- P.S.M = Pawn Structure Multiplier
- F.P.S.S = Final Pawn Structure Score
- K.S.M = King Safety Multiplier
- F.K.S.S = Final King Safety Score
- F.T.S = Final Total Score

To reiterate again, the final evaluation score is determined in the `calculate_total_score` method (page [insert page]), which multiplies the initial score of each component by its corresponding multiplier, then computes the sum of the scores for all components. (rounded to 2 decimal places)

Opening Scenario



Reminder: In the opening, in terms of weighting, material > centre control > king safety > pawn structure

Using the individual values above on the board to the left, the initial component scores are:

- Material score = +1
- Centre control score = +7
- Pawn structure score = -2
- King safety score = 0

Row no.	M.M	F.M.S	C.C.M	F.C.C.S	P.S.M	F.P.S.S	K.S.M	F.K.S.S	F.T.S	Comments
1	1	+1	1	+7	1	-2	0.1	0	+6.00	F.C.C.S too high compared to F.M.S
2	3	+3	0.2	+1.4	1	-2	0.1	0	+2.40	F.P.S.S too high compared to F.C.C.S and F.M.S
3	3	+3	0.2	+1.4	0.3	-0.6	0.1	0	+3.80	Satisfactory values

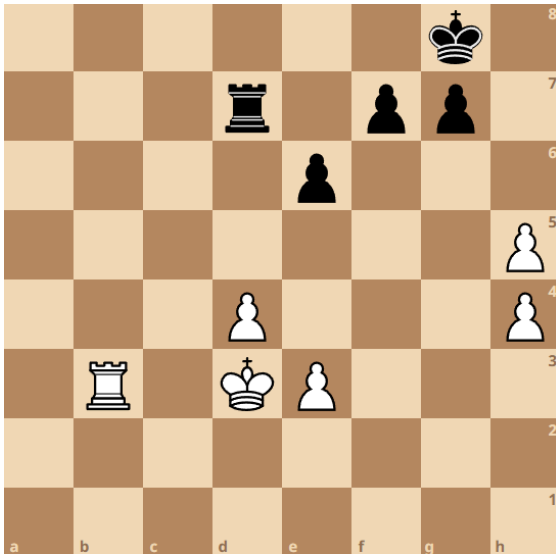
Middlegame Scenario

Reminder: In the middlegame, in terms of weighting,
material > king safety > centre control $\approx$ pawn structure

Using the individual values above on the board to the left, the initial component scores are:

- Material score = -2
- Centre control score = +13
- Pawn structure score = -2
- King safety score = -15

Row no.	M.M	F.M.S	C.C.M	F.C.C.S	P.S.M	F.P.S.S	K.S.M	F.K.S.S	F.T.S	Comments
1	1	-2	1	+13	1	-2	1	-15	-6.00	F.C.C.S too high compared to F.M.S
2	3	-6	0.1	+1.3	1	-2	1	-15	-21.70	F.P.S.S too high compared to F.M.S and F.C.C.S
3	3	-6	0.1	+1.3	0.6	-1.2	1	-15	-20.90	F.K.S.S too high compared to F.M.S
4	3	-6	0.1	+1.3	0.6	-1.2	0.15	-2.25	-8.15	Satisfactory values

Endgame Scenario

Reminder: In the endgame, in terms of weighting, material > pawn structure >> centre control ≈ king safety

Using the individual values above on the board to the left, the initial component scores are:

- Material score = +1
- Centre control score = +6
- Pawn structure score = -3
- King safety score = -9

Row no.	M.M	F.M.S	C.C.M	F.C.C.S	P.S.M	F.P.S.S	K.S.M	F.K.S.S	F.T.S	Comments
1	1	+1	1	+6	1	-3	1	-9	-5.00	F.C.C.S too high compared to F.M.S and F.P.S.S
2	1	+1	0	0	1	-3	1	-9	-11.00	F.K.S.S too high compared to F.M.S and F.P.S.S
3	1	+1	0	0	1	-3	0	0	-2.00	M.M too low compared to F.P.S.S
4	3	+3	0	0	1	-3	0	0	0	Satisfactory values

With all of the component multipliers finalised for all 3 gamephase, here is a final look at the multiplier dictionary held in the `calculate_total_score` method (using the individual set of values defined 2 pages ago).

```
def calculate_total_score(self, material_score, centre_control_score, pawn_structure_score, king_safety_score,
multipliers = {'Opening':[3,0.2,0.3,0.1], 'Midlegame':[3,0.1,0.6,0.15], 'Endgame':[3,0,1,0]}
#multipliers tweaked for optimisation
#multipliers left to right: material score, centre control, pawn_structure, king safety
```

NOTE: The individual values defined for each component are NOT proportional to each other. As a result, the component multipliers are not indicative to how much a component will account to the final score. For instance, if you observe the 'Midlegame' key, the multiplier for pawn structure is greater than the multiplier for centre control ($0.6 > 0.1$), thus it may be misleading one to incorrectly conclude that pawn structure is the bigger factor to consider in the midgame. However, the reason for the higher multiplier is to compensate for the fact that the individual values are much bigger for the `evaluate_centre_control` method (e.g. `piece_in_centre_bonus` = 6 whereas `pawn_shield_bonus` = 1). As a result, the final scores for both components will truly be representative and scaled relative to each other.

Test Sequence 3 Continued

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
26	1. Set up the following position below as beginner difficulty:  2. Display the current valid move list in the console (white's move)	Normal: Shows that the valid move queue is operational, with moves ordered appropriately	• Capture moves by queen shown first, followed by central moves • The king moving to the d1 and e2 squares are at the end of the queue	2:04 - 3:17 Pass
27	1. Set up the following position below (black to move):  2. Observe the move the A.I plays	Normal: Proves that the minimax function works by picking the best available move whilst searching all possible positions in the future at depth = 4. (In this test, the time taken to spot the move is irrelevant)	• The A.I captures the pawn on the d4 square using the rook	3:17 - 4:03 Pass

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
28	1. Set up the following position below, and observe the move the A.I (black) makes:  2. Observe the time taken to make each move for: • Minimax without alpha-beta and move ordering • Minimax with alpha-beta, without move ordering • Minimax with alpha-beta and move ordering	Normal: This test acts as the system testing in the way that all components are being tested: (ALL game functionality and A.I robustness are being analysed) Test proves that alpha-beta pruning is effective by shortening execution time	• Pawn to d5 is played by the A.I • Execution time < 5s after all optimization algorithms are in place	4:03 - 6:27 Fail NOTE: This Test ID includes a fast-forward
29	1. Set up the scholar's mate by using the beginner difficulty (easier)  2. Click the hint button 3. Complete the scholar's mate	Normal: Shows that hint functionality works as required, and that minimax function is called in perspective of user	• The f3 and f7 squares should be highlighted in orange • Once move is completed, the f3 and f7 squares should no longer be highlighted	7:28 - 8:30 Pass

Test ID 28 didn't quite go to plan! However, I do have one last trick up my sleeve...

Redesigning the Move Generation Algorithm

An aspect regarding maximising the time efficiency for the chess A.I that I admittedly I had overlooked was the time taken to generate moves. You see, optimising the time complexity for the minimax algorithm can only take you so far: methods such as alpha-beta pruning and move ordering are redundant if the program takes forever to initiate moves when traversing through each node in the move tree!

Therefore, the goal in this chapter is to accelerate the execution times for the game engine to initiate and undo moves (i.e. make `move_engine.initiate_move()` and `move_engine.undo()` as fast as possible). I traced the problem back to the move generation method held in the `gamedata` class, `obtain_all_valid_moves`, as I concluded that this function eats up the most time when being called. The current state of `obtain_all_valid_moves` is shown below (screenshot copied from page [insert page]).

```
def obtain_all_valid_moves(self, board, colour, class_dictionary):
    self.white_move_list = []
    self.black_move_list = []
    self.current_valid_move_list = [] #all moves of the next player AFTER the current move is made

    for piece_pos in class_dictionary:
        piece_object = class_dictionary[piece_pos]
        if piece_object.colour == 'w':
            king_pos = self.white_king_pos
        else:
            king_pos = self.black_king_pos
        piece_move_list = piece_object.calculate_moves(board, king_pos)
        #method call also modifies the 'valid_move_list' attribute for the piece

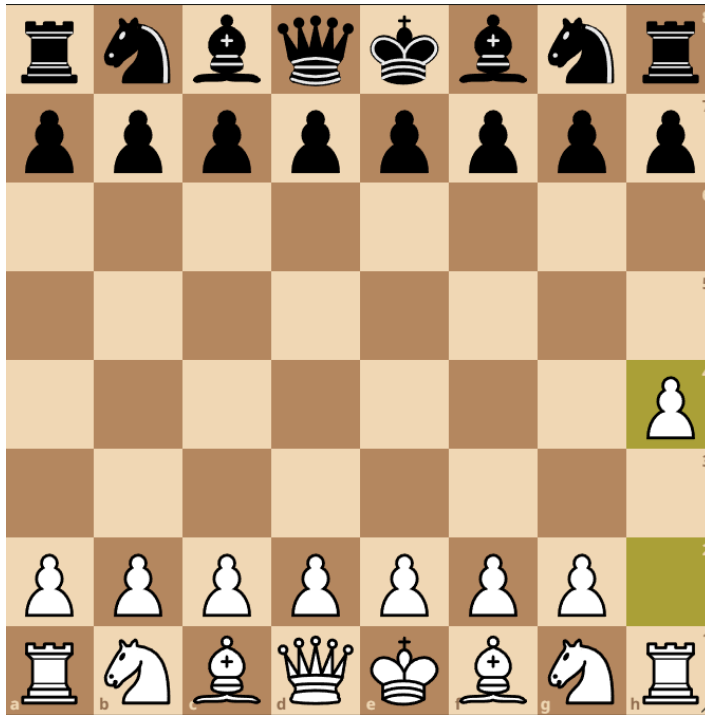
        if len(piece_move_list) != 0:
            if piece_object.colour == 'w':
                for move in piece_move_list:
                    self.white_move_list.append(move)
            else:
                for move in piece_move_list:
                    self.black_move_list.append(move)

    # we want all moves of the next player AFTER the current move is made
    if colour == 'w': #after white's turn, black move now
        self.current_valid_move_list = self.black_move_list
    else: #after black's turn, white move now
        self.current_valid_move_list = self.white_move_list
```

In essence, the method above iterates through ALL pieces on the board and calls their unique `calculate_moves` method for EACH piece before adding the result to the black or white move list (depending on the colour). This is an issue and here's why...

The problem with calling `calculate_moves()` for every piece

The `calculate_moves` method is also one of the more time intensive algorithms in my program! This is because the program has to use iteration in order to loop through every possible direction, and on the occasions it identifies a *potential* move, then must call the `check_for_checks` algorithm which also isn't a quick process (for some queen positions, the checking algorithm may be called up to 27 times!). Whilst I cannot improve the execution speeds of either of these algorithms in any obvious way, I can control how often it is called...

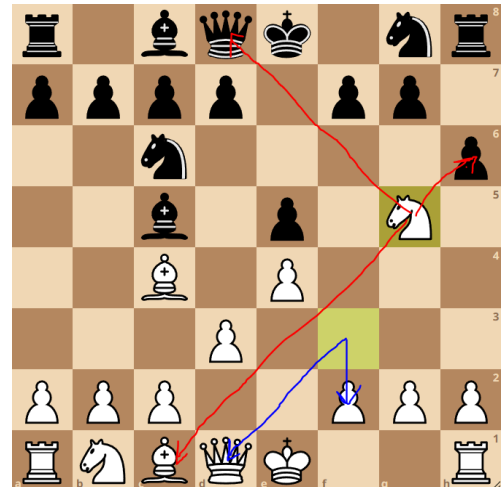


Consider the following move taken place in the picture on the left. Does moving the pawn move affect the `valid_move_list` attribute for the b1 knight object? Or the d8 queen object? In fact, in this scenario, the **ONLY** piece object in which the following move would affect its `valid_move_list` is the rook object on the h1 square (and the moved pawn itself). This means of the 32 instances my current algorithm would call the `calculate_moves` method, 30 of these calls for each piece object would result in no change in their `valid_move_list`.

The solution: Principles of operation for the new 'obtain changed pieces' function

My proposed solution for this problem is to devise another algorithm which would identify all the pieces in which their `valid_move_list` may change after a move has been, and ONLY call the `calculate_moves` method on these specific piece objects.

A great place to start is to consider the starting and ending squares, and determine the pieces that can directly travel to either of those squares. As it turns out, these piece objects will always have their `valid_move_list` attributes change, so these pieces MUST have their moves re-calculated. In the scenario to the right, only 6 pieces need to be re-calculated, much lower than calculating 32 pieces using the naive method beforehand.



Tackling (un)pins

This new algorithm will also need to take into account the position of the king.



Consider the scenario to the right: When the bishop moves to the g5 square, the knight becomes pinned and its list of valid moves changes (actually becomes empty), so the `calculate_moves` method must be called on the knight object to update the list (even though the same knight cannot jump to the g5 square). Likewise when the bishop departs from its previous square, the c3 pawn also becomes pinned so that also needs to be updated.

The same principle applies to when a piece becomes unpinned.

Extra caveats to consider

- If the king moves, or the state of the game is transitioning to/from a check, call `calculate_moves` for ALL pieces
 - The function becomes way too complicated when considering checking and the motion of the king, and given the time I have left on this project, I decided this would be one ambitious task too many
- When necessary evaluate all pawns nearby the start and end squares
 - Pawns are also obnoxiously complex to handle, so to be on the safe-side it would be sensible to re-evaluate their move lists even if they may not need to
- Always call the `calculate_move` method on the enemy king after a move is made
 - The piece that had just moved may block/unblock some of the squares the king could potentially move to beforehand

It is important to acknowledge that it is preferred if this 'obtain_changed_pieces' algorithm was to be overcautious, and took into account more squares than it may need to as opposed to inverse (as a fully functioning move generation function is more ideal than a faster but less reliable one).

Exploring the benefits for the 'obtain_changed_pieces' algorithm

d = depth of minimax algorithm

Time complexity of move generation without optimization algorithm

Best-case scenario: $O(32^d)$

Worst-case scenario: $O(32^d)$

Average-case scenario: $O(32^d)$

Beforehand, the time complexity for generating moves is constant as after each move all 32 pieces were always being inspected and called to recalculate their moves.

Time complexity of move generation with optimization algorithm

Best-case scenario: $O(1^d) = O(1)$ - this occurs if the piece that moved affected no other pieces

Worst-case scenario: $O(32^d)$ - this occurs if the king moves, or the game is moving into/from a check

Average-case scenario: $O(n^d)$ - where n = average number of 'changed pieces' per turn

Let n = 16 (in most cases, n will be lower). When d = 4:

- Move generation without optimization = $O(32^4) = O(1,048,576)$
- Move generation with optimization (average-case scenario) = $O(16^4) = O(65,536)$

Thus, we can conclude (theoretically) that the execution speeds of the `obtain_all_valid_moves` algorithm with implementation of the new 'obtain_changed_pieces' algorithm is $(1,048,576/65,536) = 16$ times faster on average, when depth = 4.

NOTE: In practice, the effect with the new algorithm will not be quite as substantial. This is due to the fact that additional time is spent in order to identify the objects that need to have their moves re-calculated. However, as the time taken to identify changed pieces < time taken to call the `calculate_moves` method for all pieces, the implementation of this new optimisation algorithm should still result in a noticeable effect in execution times!

Implementing the Renovated Move Generation Algorithms

```
def obtain_all_valid_moves(self, board, colour, class_dictionary, move_data):
    self.white_move_list = []
    self.black_move_list = []
    self.current_valid_move_list = [] #all moves of the next player AFTER the current move is made

    changed_pieces = self.obtain_changed_pieces(board, colour, move_data)

    if changed_pieces == 'all':
        changed_pieces = class_dictionary.keys() #all moves

    for piece_pos in changed_pieces: #all pieces which their move list may have changed
        piece_object = class_dictionary[piece_pos]
        if piece_object.colour == 'w':
            king_pos = self.white_king_pos
        else:
            king_pos = self.black_king_pos

        ignore = piece_object.calculate_moves(board, king_pos)
        #method call modifies the 'valid_move_list' attribute for the piece, which is what we want

    for piece_pos in class_dictionary:
        piece_object = class_dictionary[piece_pos]
        piece_move_list = piece_object.valid_move_list
        if len(piece_move_list) != 0:
            if piece_object.colour == 'w':
                for move in piece_move_list:
                    self.white_move_list.append(move)
            else:
                for move in piece_move_list:
                    self.black_move_list.append(move)

    # we want all moves of the next player AFTER the current move is made
    if colour == 'w': #after white's turn, black move now
        self.current_valid_move_list = self.black_move_list
    else: #after black's turn, white move now
        self.current_valid_move_list = self.white_move_list
```

The `obtain_all_valid_move_list` is still belonging to the `gamedata` object, and is still called at the same location in the main `gamedata.update` method. The key difference here is that this time the algorithm only calls `calculate_moves` for the piece objects that are affected after the move had been executed (these pieces objects are fetched using the `obtain_changed_pieces` method)

Implementing the 'obtain changed pieces' method

```
def obtain_changed_pieces(self, board, colour, move_data):
    # this method obtains a record of every single piece which needs to have its moves re-evaluated
    # after a move made
    if move_data == 'first move': #first move
        return 'all'

    start_square = move_data[0]
    end_square = move_data[1]

    if colour == 'w':
        enemy_colour = 'b'
        friendly_king_pos = self.white_king_pos
        enemy_king_pos = self.black_king_pos
    else:
        enemy_colour = 'w'
        friendly_king_pos = self.black_king_pos
        enemy_king_pos = self.white_king_pos

    #if to/from check, or king has moved, search all piece moves
    if board[end_square[0]][end_square[1]][1] == 'K' or self.is_check or self.check_change:
        return 'all' #the signal that all pieces need to be re-evaluated for

    changed_squares = {move_data[1]}
    #destination square obviously needs to have its move re-evaluated

    for square in move_data: #iterate through start and end squares
        changed_squares = changed_squares | set(check_for_checks(board, 'w', square, True))
        #gets the location of all pieces that can jump to 'square' in a single move
        #colour irrelevant in call, '|' means union

        changed_squares = changed_squares | set(get_pawn_change_squares(board, square))
        #all pawns that MAY be changed are added (as a precaution )

        in_between_pieces_attacking = get_in_between_squares_attacking(board, enemy_colour, list(square),
                                list(enemy_king_pos))
        if len(in_between_pieces_attacking) == 1: #only 1 piece between king an enemy piece, may be (un)pinned
            changed_squares = changed_squares | set(in_between_pieces_attacking)

        in_between_pieces_defending = get_in_between_squares_defending(board, colour, list(square),
                                list(friendly_king_pos))
        changed_squares = changed_squares | set(in_between_pieces_defending)

    changed_squares = changed_squares | {enemy_king_pos}

    return list(changed_squares)
```

`check_change` is another attribute I created within the `gamedata` class, and is updated within the `update_gamestate` method. It's defined by looking at the current and previous state of the board: If the current state of the king is not in check after a move was made but was in check before the move (and vice versa), `check_change` is set to `True`, otherwise, it will always be set to `False`.

I decided to initialise `changed_squares` using the `set` data structure in order to eliminate data duplication (i.e. a piece that could jump to the input square could also be pinned). This can be done by exploiting the union (`|`) command, which joins two or more sets and removes any repeating values.

Revisiting the `check_for_checks` algorithm in order to fetch changed squares

```

def check_for_checks(board, colour, king_pos, get_changed_squares = False):
    changed_squares = []
    king_row, king_col = king_pos
    #king_pos: position of the king to be inspected
    #colour: colour of the king to be inspected

    #For each possible vector traverse direction. If the matching opposite piece is within the path return
    #True (king is being checked)

    piece_vectors = [(1,0),(0,1),(-1,0),(0,-1), (1,1),(1,-1),(-1,1),(-1,-1),
                    (1,2),(2,1),(-1,2),(2,-1),(1,-2),(-2,1),(-1,-2),(-2,-1)]
    #1-4: rook, queen, king | 5-8: bishop, queen, king | 9-16: knight

    for vector_no, vector in enumerate(piece_vectors):
        row_vector, col_vector = vector[0], vector[1]
        king_new_row, king_new_col = king_row + row_vector, king_col + col_vector

        if 0 <= king_new_row <= 7 and 0 <= king_new_col <= 7:
            if (0 <= vector_no <= 7) and board[king_new_row][king_new_col][1] == 'K':
                changed_squares.append((king_new_row, king_new_col))
                if board[king_new_row][king_new_col][0] != colour and not get_changed_squares:
                    return True # if an enemy king is adjacent to the king
            elif (8 <= vector_no <= 15) and board[king_new_row][king_new_col][1] == 'N':
                changed_squares.append((king_new_row, king_new_col))
                if board[king_new_row][king_new_col][0] != colour and not get_changed_squares:
                    return True # if an enemy knight is an 'L' shaped distance away from the king
            if 8 <= vector_no <= 15:
                continue # vectors 9-16 are only for knights, so the code below can be skipped

        # while loop to check for ranged pieces only
        while 0 <= king_new_row <= 7 and 0 <= king_new_col <= 7:
            if board[king_new_row][king_new_col] != '00': # if not an empty square
                if (0 <= vector_no <= 3) and (board[king_new_row][king_new_col][1] == 'R' or
                    board[king_new_row][king_new_col][1] == 'Q'):
                    changed_squares.append((king_new_row, king_new_col))
                    if board[king_new_row][king_new_col][0] != colour and not get_changed_squares: #enemy piece
                        return True # if an enemy queen/rook is horizontal or vertical to the king
                elif (4 <= vector_no <= 7) and (board[king_new_row][king_new_col][1] == 'B' or
                    board[king_new_row][king_new_col][1] == 'Q'):
                    changed_squares.append((king_new_row, king_new_col))
                    if board[king_new_row][king_new_col][0] != colour and not get_changed_squares: #enemy piece
                        return True # if an enemy queen/bishop is diagonal to the king
            break

        #for each direction, keep traversing the paths for the king until the square is outside the board,
        #or there is a piece blocking the direction (also examine said piece)
        king_new_row += row_vector
        king_new_col += col_vector

    if get_changed_squares:
        return changed_squares

```

The code above is a portion of the `check_for_checks` algorithm (first seen in page 67). This function is called by `obtain_changed_pieces` with the optional parameter `get_changed_squares` tweaked to `True`. As it turns out I discovered this algorithm (with some tweaks highlighted in red) can fulfil the role of obtaining all of the squares in which the pieces on these squares can jump to a given `king_pos`. (where the start/end square from move data is passed in as the `king_pos` argument).

get_pawn_change_squares

```
def get_pawn_change_squares(board,target_square):
    changed_pawns = [] #stores the squares which contains pawns that can move directly to 'target_square'
    movement_vectors = [(-1,1),(-1,-1),(-1,0),(-2,0), (1,1),(1,-1),(1,0),(2,0)] #movement vectors for pawns
    # 1-4: white pawns|5-8: black pawns

    for vector_no,vector in enumerate(movement_vectors):
        new_row,new_col = target_square[0]+vector[0], target_square[1]+vector[1]
        if 0 <= new_row <= 7 and 0 <= new_col <= 7:
            if board[new_row][new_col][1] == 'p':
                if board[new_row][new_col][0] == 'b' and 0 <= vector_no <= 3:
                    #black pawn (which move DOWN) diagonally or vertically ABOVE the square
                    changed_pawns.append((new_row,new_col))
                elif board[new_row][new_col][0] == 'w' and 4 <= vector_no <= 7:
                    #white pawn (which move UP) diagonally or vertically BELOW the square
                    changed_pawns.append((new_row,new_col))

    return changed_pawns
```

Checking for pins

```
def get_difference_vector(target_square, king_pos):
    row_gain = target_square[0] - king_pos[0]
    col_gain = target_square[1] - king_pos[1]
    if not (abs(col_gain) == abs(row_gain) or col_gain == 0 or row_gain == 0):
        return [] #target square is not diagonal or straight relative to the king, i.e. no in between squares

    if row_gain == 0:
        row_vector = 0
    else:
        row_vector = row_gain // abs(row_gain)
    if col_gain == 0:
        col_vector = 0
    else:
        col_vector = col_gain // abs(col_gain)

    #difference_vector such that king_pos + difference_vector*k = target_square
    #where k = no.squares in between the two squares + 1
    difference_vector = (row_vector, col_vector)

    return difference_vector
```

```
def get_in_between_squares_attacking(board, colour, target_square, king_pos):
    in_between_pieces = []
    difference_vector = get_difference_vector(target_square, king_pos)
    if difference_vector == []:
        return [] #no pins as target square not in straight line relative to king

    while king_pos != target_square:
        king_pos[0] += difference_vector[0]
        king_pos[1] += difference_vector[1]
        if board[king_pos[0]][king_pos[1]][0] == colour:
            in_between_pieces.append((king_pos[0], king_pos[1]))

    return in_between_pieces

def get_in_between_squares_defending(board, colour, target_square, king_pos):
    in_between_pieces = []
    difference_vector = get_difference_vector(target_square, king_pos)
    if difference_vector == []:
        return [] #no pins as target square not in straight line relative to king

    while 0 <= king_pos[0] <= 7 and 0 <= king_pos[1] <= 7:
        if board[king_pos[0]][king_pos[1]][0] == colour:
            in_between_pieces.append((king_pos[0], king_pos[1]))

        elif board[king_pos[0]][king_pos[1]] != '00': #enemy piece between target square/king - potential pin
            in_between_pieces.pop(0) #remove first item (the king), as we do not want to count it
            return in_between_pieces

        king_pos[0] += difference_vector[0]
        king_pos[1] += difference_vector[1]

    return [] #no enemy piece between the target square and king inclusive - no threat
```

The reason why we have 2 functions in order to check for pins is because

`get_in_between_squares_attacking` is used to check for pins between the enemy king and a friendly piece, and `get_in_between_squares_defending` is used to check for unpins between the friendly king and an enemy piece (if there is one).

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
28 Take Two	1. Set up the following position below, and observe the move the A.I (black) makes:  2. Observe the time taken to make each move for: Minimax with alpha-beta and move ordering and the improved move generation	Normal: This test acts as the system test in the way that all components are being tested: (ALL game functionality and A.I robustness are being analysed) Test proves that alpha-beta pruning is effective by shortening execution time	- Pawn to d5 is played by the A.I - Execution time < 5s after all optimization algorithms are in place	6:27 - 7:28 Pass

Access to test sequence 3 can be found by scanning the QR code, or by using the following link:

https://www.youtube.com/watch?v=DBNcbk6zW_A



Implementing the Post Game Report(10.)

Introduction to the post game report class

```
class post_game:
    def __init__(self):
        self.evaluation_queue = [0]
        self.player_colour = ''
        self.standard_moves = 0
        self.strong_moves = 0
        self.blunder_moves = 0

    def push_evaluation_queue(self, is_checkmate_or_stalemate, winner, evaluation):...

    def pop_evaluation_queue(self):
        self.evaluation_queue.pop()

    def evaluate_all_moves(self):...

    def evaluate_player_move(self, move_number):...

    def find_evaluation_difference(self, ai_move_before, ai_move_after):...

    def display(self, move_count, winner):...

    def display_game_over(self):...

    def display_winner(self, winner):...

    def display_game_length(self, move_count):...

    def display_move_analysis(self):...
```

The screenshot above depicts all of the attributes and methods belonging to the post_game class. The first item of the attribute `evaluation_queue` also holds 0, as this is the net score for the beginning of the game when the pieces are in their original position.

The specifics regarding the operation and purpose of each method will be explored shortly.

Implementing the player move evaluation algorithm

Modifying the evaluation queue

```
def push_evaluation_queue(self, is_checkmate_or_stalemate, winner, evaluation):
    if is_checkmate_or_stalemate:
        if winner == 'w':
            self.evaluation_queue.append(MAX_SCORE)
        elif winner == 'b':
            self.evaluation_queue.append(MIN_SCORE)
        else:
            self.evaluation_queue.append(0)
    else:
        self.evaluation_queue.append(evaluation)

def pop_evaluation_queue(self):
    self.evaluation_queue.pop()
```

The following code below is inside the main infinite loop after a player (or A.I) initiates a move, and explains how the method call varies depending on the state of the game

```
if gamedata.is_checkmate or gamedata.is_stalemate:
    post_game_report.push_evaluation_queue(True, gamedata.winner, 'filler')
    game_active = False
else:
    post_game_report.push_evaluation_queue(False, 'filler', ai.evaluation_function(chessboard.digital_board))
```

The `pop_evaluation_queue` method is called after the user decides to undo a move.

```
if event.type == pygame.KEYDOWN and game_active:
    if event.key == pygame.K_u: # 'U' key is pressed = U = Undo move
        if len(gamedata.move_log_stack) != 0 and len(gamedata.move_log_stack) != 1:
            #cannot undo a move if no moves have taken place

            move_engine.undo_move()
            post_game_report.pop_evaluation_queue()
            print(gamedata.move_count, gamedata.game_phase)
            move_engine.undo_move() #called twice as A.I move also needs to be undone
            post_game_report.pop_evaluation_queue()
            print(gamedata.move_count, gamedata.game_phase)
            move_data = []
            game_active = True
```

Calculating the suitability of all player moves

```
def evaluate_all_player_moves(self):
    if self.player_colour == 'w':
        player_move_sync = 1 #every odd index in the queue is an evaluation for white
    else:
        player_move_sync = 0 #every even index (other than index 0) in the queue is an evaluation for black

    for move_number in range(len(self.evaluation_queue)):
        if move_number % 2 == player_move_sync and move_number != 0: #player made move at said 'move number'
            self.evaluate_single_player_move(move_number)

def evaluate_single_player_move(self, move_number):
    if move_number == len(self.evaluation_queue) - 1: #final move
        player_evaluation = self.evaluation_queue[move_number]
        if self.player_colour == 'b': #if player is black, a negative score after is good
            player_evaluation *= -1
    else:
        player_evaluation = self.find_evaluation_difference(move_number-1, move_number+1)

    if player_evaluation > 3:
        self.strong_moves += 1
    elif player_evaluation < -5:
        self.blunder_moves += 1
    else:
        self.standard_moves += 1

def find_evaluation_difference(self, ai_move_before, ai_move_after):
    ai_evaluation_before = self.evaluation_queue[ai_move_before]
    ai_evaluation_after = self.evaluation_queue[ai_move_after]

    evaluation_difference = ai_evaluation_after - ai_evaluation_before
    if self.player_colour == 'b': #if player is black, a negative score after is good
        evaluation_difference *= -1

    return evaluation_difference
```

The code above implements the initial concept I had for measuring the strength of each move in the design phase (page [insert page]). I was happy with this initial idea so I just kept it the way it is! For clarity, if a player made a move that resulted in a deduction of 5 or more points following the move of the A.I, then I considered that as a blunder (gain in 3 points or more = strong move).

```
if gamedata.is_checkmate or gamedata.is_stalemate:
    post_game_report.push_evaluation_queue(True, gamedata.winner, 'filler')
    post_game_report.evaluate_all_player_moves()
    game_active = False
else:
    post_game_report.push_evaluation_queue(False, 'filler', ai.evaluation_function(chessboard.digital_board))
```

I wanted the evaluate_all_player_moves method to be called when and only when the game ended, so all I needed to do was to insert the call to the chunk of code above (first shown in the previous page).

Displaying the report

```
def display(self, move_count, winner):
    screen.fill(GREY)
    self.display_game_over()
    self.display_winner(winner)
    self.display_game_length(move_count)
    self.display_move_analysis()

def display_game_over(self):
    game_over_surface, game_over_rect = appendices.text_schematic('GAME OVER', 360, 50, 80, WHITE, 'centre')
    underline_rect = pygame.Rect(game_over_rect.left, game_over_rect.bottom, game_over_rect.width, 20)
    pygame.draw.rect(screen, WHITE, underline_rect, 2)
    screen.blit(game_over_surface, game_over_rect)

def display_winner(self, winner):
    #if statements ensure that 'winner' is written in full
    if winner == 'w':
        winner = 'White'
    elif winner == 'b':
        winner = 'Black'
    else: #stalemate
        winner = 'Draw'

    winner_surface, winner_rect = appendices.text_schematic('Winner: ' + winner, 100, 150, 40, WHITE, 'topleft')
    screen.blit(winner_surface, winner_rect)

def display_game_length(self, move_count):
    game_length_surface, game_length_rect = appendices.text_schematic('Length of game: ' + str(move_count) + ' moves',
    100, 250, 40, WHITE, 'topleft')

    screen.blit(game_length_surface, game_length_rect)

def display_move_analysis(self):
    #displaying the standard move count
    standard_move_surface, standard_move_rect = appendices.text_schematic('Standard moves: ' + str(self.standard_moves),
    100, 350, 40, WHITE, 'topleft')
    screen.blit(standard_move_surface, standard_move_rect)

    #displaying the strong move count
    strong_move_surface, strong_move_rect = appendices.text_schematic('Strong moves: ' + str(self.strong_moves),
    100, 450, 40, WHITE, 'topleft')
    screen.blit(strong_move_surface, strong_move_rect)

    # displaying the blunder move count
    blunder_move_surface, blunder_move_rect = appendices.text_schematic('blunder moves: ' + str(self.blunder_moves),
    100, 550, 40, WHITE, 'topleft')
    screen.blit(blunder_move_surface, blunder_move_rect)
```

Implementing the GUI for the post-game report was very similar to creating the main menu in the way that most of the process was just re-using the PyGame features as well `text_schematic` function from the `appendices` class.

```
if not game_active: #display main menu/post_game_report if false, display game if true
    if gamedata.is_checkmate or gamedata.is_stalemate: #after game is finished
        post_game_report.display(gamedata.move_count, gamedata.winner)
    else: #before game begins
        main_menu.display()
else: #game is active, display chessboard
```

The code above is within the main infinite loop, and illustrates how/when the `display` method is called within the program (`post_game_report.display()` is ONLY called here and nowhere else).

Testing the Post Game Report(10.)

Test Sequence 4

TestID	Test Data	Type/Purpose of test	Expected outcome	Actual outcome (pass/fail)
30	1. Set up the scholar's mate by using the beginner difficulty (easier)  2. Complete the scholar's mate 3. Observe the post-game report	Normal: Shows that all necessary details are displayed, and that player move evaluation algorithm is accurately	• 'Winner: White' displayed • 'Length of game: [move_count] moves' displayed (appropriate no. moves shown) • 'Standard moves: 3' displayed • 'Strong moves: 1' displayed • 'Blunder moves: 0' displayed	0:00 - 1:15 Pass
31	1.Be on the post-game report 2. Click the 'U' key 3. Click the 'H'	Erroneous: Shows that program can handle invalid inputs by the user when displaying the post-game report	• No observable changes occur for both actions	1:15 - 1:39 Pass

Access to test sequence 4 can be found by scanning the QR code, or by using the following link:

<https://youtu.be/9W2Ym74Tt1U>



Evaluation

Proposed Objectives Checklist

The table below shows each objective that I had initially defined, alongside with the pages where I had completed said objective.

Objective no.	Evidence of completion (page no.)
1. Display Menu	
1.1	20, 56-57
1.2	20, 58-59
1.3	21, 60
2. Create chessboard/pieces	
2.1	21, 63
2.2	22, 64
2.3	24, 62
3. Define piece characteristics	
3.1	25-31, 65-66, 68-71
3.2	72
3.3	29, 70, 82, 92-93
3.4	32-33, 67
4. Updating/Maintaining the chessboard	
4.1	25, 77-78
4.2	46, 79-85
4.3	46, 86-87
4.3.1: Let user undo a move, OPTIONAL	88
4.4	106
5. Game-state inspection	
5.1	34, 84
5.2	34, 84
5.3	34, 84
5.4	37, 94

Objective no.	Evidence of completion (page no.)
6. Creating the Evaluation Function	
6.1	39, 96
6.2	41-42, 99
6.3	40, 97-98
6.4	43, 100
6.5	44, 101
7. Creating the Minimax Algorithm	
7.1	45-46, 102
7.2	45-46, 102
7.3	45-46, 102
7.4	45-46, 102
7.5	45-46, 102
7.6	45-46, 102
8. Implementing alpha-beta pruning	
8.1	49, 103-104
8.2	47-48, 102
9. Generate move using everything	
9.1	105
9.2	105
10. Post-game report	
10.1	51, 126-127
10.2	50, 128
10.3	50, 128

Additional thoughts on my objective completion

Objective 1 (making the main menu GUI) proved to be harder to implement than anticipated! This is due to the fact that I had never any prior experience using the PyGame module, so I had to spend extra time finding articles and watching tutorials in order to fully grasp the concepts of screens and surfaces etc. as well as learning the syntax required in order to display the objects. However, once that obstacle was dealt with, I was satisfied with the way the main menu interface looked and operated at the end: it met the simple yet effective prototype I had initially wanted. Although I could've worked on renovating the main menu even further, I decided that at the end of the day it didn't necessarily need to be a state of art and that allocating more time tackling the more complex parts of the project was the better choice.

Objective 10 (making the post game report GUI) was much easier to implement now that I was familiarised myself with pygame, and similar to what I said previously I didn't want to spend too much time on it so as long as it was functional, and the values for the 'standard/strong/blunder moves' were indicative, that was fine by me.

Other than the higher complexity than first imagined when implementing the subclasses for the pawn objects, (they are definitely the hardest piece to program!), I am very satisfied with the functionality of my game (i.e. objectives 2, 3, 4 and 5). Even outside the testing shown in my videos, during the 'play-tests' I conducted in my spare time when the program was in its final state I couldn't generate any sort of error from the code (which I believe is a respectable accomplishment as even building a chess game without A.I is complicated!). I've attributed this for 2 reasons:

1. Use of the chessboard, gamedata and move engine classes: I think I finally understand why people love OOP! A feature overlooked by people new to OOP is its ability to neatly collate a set of loosely tied methods together at one space. For instance I know that if I ever wanted to change the contents of the 'digital_board' I know exactly where to go (the chessboard object).
2. Input abstraction/sanitisation/validation: Notice how in my entire script (which is over 1500 lines long), I didn't use a single try-except statement - this doesn't show a lack of defensive programming/exception handling, as the reason for not needing to implement these is because of the early decision I made which is to restrict the way the user can input to the program only by mouse clicks (and the 'U' and 'H' key only, both heavily supplemented with input validation). As a result this limited the number of possibilities for error I needed to consider.

And for the icing on the cake, I managed to complete the optional objective 4.3.1 of letting the user undo a move (which was easier than anticipated).

Creating the A.I was unsurprisingly the hardest task for my project, but thanks to my thorough initial designing and decomposing the problem down into several submodules (more detail on this decomposition is shown in the hierarchy chart in page [insert page]), I can confidently say that I have completed the objectives related to the A.I (objectives 5, 6, 7, 8 and 9)

Objective 6 (creating the evaluation function) was an interesting one: creating the algorithms to detect the double-pawns, pawn-shields etc. itself wasn't too challenging (especially as I had essentially created an exact schematic of these algorithms beforehand), but the task of tuning the exact numbers that go with these traits was an arduous task. What I would say on this one is that the testing I had carried out really came to my aid here, and eventually I produced a set of compatible values.

Objectives 7 and 8 (creating the minimax function with alpha-beta pruning) was the inverse in the way that due to the recursive nature of the algorithm, testing it would be a complex procedure (as the console would be flooded with thousands of outputs for each recursive call). Nonetheless, eventually I came to a position where I could say that I had completed objectives 7 and 8.

I would say that 9.1 (<5s per move, depth = 4) was the most problematic objective for me in the entire project, and even became a bit frustrating not seeing the timer go down even after implementing the optimization algorithms of alpha-beta pruning and move ordering. Only after I thought of the idea regarding optimising the move generation speeds did the execution time per move by the A.I declined below the 5s per mark, but even in some rare cases in the middlegame phase (the point at where the number of moves available per turn is at a maximum) does the time hover over the threshold. All inclusive, I would classify objective 9.1 as a 'weak' pass.

Overall, I believe that I completed all of the fundamental objectives I set out to complete at the beginning of this project, including all of the 'optional' objectives. I would say that at its core, my solution is technically complete.

Final discussion with my third-parties

Questions for my Third-Parties

I have emailed my end-users Humza and Seb (who are both beginners wanting to take their chess skills to the next level) a working link to my chess A.I game as well as a set of questions they should answer once they have familiarised themselves with the program

My questions for Humza and Seb are as follows:

1. Can you complete a full game of chess on my program without any system crashes?
 2. Which properties regarding my chess A.I game did you enjoy/find useful the most?
 3. Did you encounter any other issues when running my chess program?
 4. How indicative was the post-game report once a game had ended? (e.g. game is not saying lots of blunder moves were made by yourself even though you felt as if you made very few mistakes)
- Question 1 ensures that a full game can indeed run on my software, solidifying the robustness of my solution as well as further suggesting that all key objectives have been met
 - Questions 2 and 3 allows me to get a taste of some elements of my project that I should keep or renovate if I were to revisit the problem
 - Although the post-game report did meet the objectives 10.1/10.2/10.3, I was still slightly concerned with the accuracy of my calculations for the different types of moves (standard/strong/blunder). Question 4 provides further clarification regarding whether I should consider this possibility of an issue if I were to revisit building the game

The responses

1. **Can you complete a full game of chess on my program without any system crashes?**
 - Both stakeholders reported no system crashes when running and using the program
2. **Which properties regarding my chess A.I game did you enjoy/find useful the most?**
 - Humza: *'I thought the hint functionality was a massive hit! When using it I was shown a series of fascinating moves I would never think of playing. I could tell it improved me as a player due to the fact that overtime I began to think and play the same moves the A.I would suggest to me.'*
 - Seb: *'I really enjoyed the shift in dynamic when transitioning from the Beginner to Intermediate difficulty. It didn't take long for me to overcome the beginner A.I, but once I transitioned to the Intermediate A.I the games I had were a lot more challenging, but not too difficult to the point that I was discouraged from playing. Also, the use of the undo functionality made the learning experience for me a lot more enjoyable.'*
3. **Did you encounter any other issues when running my chess program?**
 - Humza: *'Unfortunately my computer cannot handle the expert difficulty setting! Even though I definitely noticed a huge improvement in the A.I's behaviour, some moves in the middle of the game took as long as 50 seconds! Whilst I acknowledge the disparity in the time taken to make a move is probably due to the very slow performance of my computer, I feel like once I mastered the basics of chess when playing the Beginner and Intermediate A.I, I would probably search for alternative chess engines to take my game to the next level (not your fault at all! The expert difficulty just isn't compatible with my computer).'*
 - Seb: *'One minor suggestion for improvement I have is the performance of the A.I during the endgame. Even when playing against Expert A.I I noticed that sometimes it just tends to make repeating moves when there may be better alternative moves.'*

4. How indicative was the post-game report once a game had ended?

- Both stakeholders agreed that the post-game report was indicative of the game they felt they played (that is a relief!)

If I were to revisit the problem again...

The biggest surprise from the feedback I had received was the amount of time it took for Humza's A.I to make moves: I knew that in some positions the A.I would take longer than usual but even so the time taken would nowhere near be 50 seconds when running the chess game on my machine. I asked Seb if he had noticed speeds as slow as that when choosing the expert difficulty and he replied with a no, which made me conclude that the swiftness of the A.I is dependent on the performance of one's machine. Nevertheless an immediate change I made to the program is to decrease the max_depth when playing as Expert difficulty from 4 to 3. A more long-term solution I could implement to improve speeds if I ever revisited this project again would be to introduce transposition tables; basically, transposition tables are implemented as hash tables in which each current position of the board is encoded with a hash index. As the minimax algorithm likely generates the exact same positions of the board multiple times, the program could just search up the hash index of said position and read off its corresponding evaluation score rather than having to recalculate the exact same position all over again. A technique called zobrist hashing would be used in order to generate such keys for every position of the board, and vice versa.

If I were to research this problem again another feature I would like to implement would be dynamic depth: This involves changing the max_depth variable for the A.I depending on the phase of the game/position of the board (currently my program operates at what is known as a 'fixed depth' - depth is constant for each move. This functionality would likely address the issue seb pointed out, as I believe the reason due to the random nature of the moves the A.I makes towards the endgame is because the depth is lower than it could be: as there are very few moves available at the end, the program can handle the possibility to look 7 or 8 moves into the future over the fixed 4 move depth.

Appendices

Bibliography

- Wikipedia, 2019, *Evaluation Function*, viewed 25 January 2021, https://en.wikipedia.org/wiki/Evaluation_function#:~:text=Evaluation%20functions%20in%20chess%20consist,as%20when%20checkmate%20is%20imminent
- Walker, L, 2020, *The anatomy of a Chess AI*, Medium, viewed 28 January 2021, <https://medium.com/the-innovation/the-anatomy-of-a-chess-ai-2087d0d565>
- Mondev16 and kulseran, 2017, in response to *Game AI and Difficulty Levels* by andi_de_bob, Reddit, viewed 1 February 2021, https://www.reddit.com/r/gamedev/comments/76wpv8/game_ai_and_difficulty_levels/
- Hartikka, L, 2017, *A step-by-step guide to building a simple chess AI*, freeCodeCamp, viewed 1 February 2021, <https://www.freecodecamp.org/news/simple-chess-ai-step-by-step-1d55a9266977/>
- Misherturnball, 2015, in response to *What is an accurate way to evaluate chess positions?* by Menguy, C, StackExchange, viewed 4 February 2021, <https://chess.stackexchange.com/questions/347/what-is-an-accurate-way-to-evaluate-chess-positions>
- UCLA, 2018, <http://web.cs.ucla.edu/~rosen/161/notes/alphabeta.htm>, viewed 31 January 2022
- https://www.kindpng.com/http://web.cs.ucla.edu/~rosen/161/notes/alphabeta.htmlimgv/hxbhmb_chess-pieces-png-chess-pieces-sprite-sheet-transparent/
(link for free png chess pieces I used)

Full Code

NOTE: Some parts of the code shown here may differ from the code shown in the screenshots (any differences will be minimal, and all of the fundamental concepts will still be the same).

Constants.py

```
1  import pygame,sys
2
3  #colour RGB values
4  ARCTIC = (244,243,239)
5  BLACK = (0,0,0)
6  GREEN = (11,102,35)
7  GREY = (58,59,60)
8  MINT = (65,105,88)
9  RED = (138,3,3)
10 ORANGE = (237,135,45)
11 WHITE = (255,255,255)
12
13 cell_size = 90
14 cell_number = 8
15 MAX_SCORE = 99999999
16 MIN_SCORE = -99999999
17 screen = pygame.display.set_mode((720,720)) #720 = cell_size x cell_number = 90 x 8
18 game_font = 'Extras/MochiyPopPOne-Regular.ttf'
```

Main.py

```

1  from constants import *
2  from pieces import pawn,knight,bishop,rook,queen,king
3  from gamedata import gamedata
4  from AI import AI
5
6  pygame.init()
7  pygame.display.set_caption('Chess')
8  clock = pygame.time.Clock()
9
10 game_active = False #initiaize to false
11
12 current_turn = 'w'
13 mouse_pos = (0,0)
14
15 class appendices:
16     #this class neatly stores all additional functions that are frequently used
17     #within the rest of the program, but are not distinctly part of a specific class
18
19     @staticmethod
20     def flip_current_turn(turn):
21         if turn == 'w':
22             turn = 'b'
23         else:
24             turn = 'w'
25         return turn
26
27     @staticmethod
28     def map_piece_to_image():
29         dictionary = {} #use of global dictionary allows easy access when mapping a piece to its image
30         pieces = ['bB','bK','bN','bp','bQ','bR','wB','wK','wN','wp','wQ','wR']
31         for piece in pieces:
32             image_png = pygame.image.load('Extras/pieces/'+piece+'.png').convert_alpha()
33             dictionary[piece] = pygame.transform.scale(image_png,(cell_size,cell_size))
34             #initial size of pieces set to cover a square on the board
35
36         return dictionary
37
38     @staticmethod
39     def map_pos_to_square():
40         dictionary = {} #converts a position in a python array to its corresponding chess square
41         if gamedata.player_colour == 'w':
42             files = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
43             ranks = ['8', '7', '6', '5', '4', '3', '2', '1']
44         else: # if player_colour = black
45             files = ['h', 'g', 'f', 'e', 'd', 'c', 'b', 'a']
46             ranks = ['1', '2', '3', '4', '5', '6', '7', '8']
47         for row in range(cell_number):

```

```

48     for col in range(cell_number):
49         square_pos = files[col]+ranks[row]
50         dictionary[(row,col)] = square_pos
51
52     return dictionary
53
54 @staticmethod
55 def get_col_row_from_pos(x_pos,y_pos):
56     if gamedata.player_colour == 'w':
57         clicked_row = y_pos // cell_size
58     else: #player_colour is black, so vertically invert 'clicked_row'
59         clicked_row = 7 - (y_pos // cell_size)
60
61     clicked_col = x_pos // cell_size
62
63     return clicked_row,clicked_col
64
65 @staticmethod
66 def text_schematic(text,x_pos,y_pos,size,colour,point):
67     #creates text of any colour,font and size in a given position
68
69     font = pygame.font.Font(game_font, size)
70     surface = font.render(str(text),True,colour)
71     if point == 'centre': #if x_pos,y_pos is a central point
72         rect = surface.get_rect(center = (x_pos,y_pos))
73     elif point == 'midleft': #if x_pos,y_pos is the middle-left point
74         rect = surface.get_rect(midleft = (x_pos,y_pos))
75     elif point == 'topleft': #if x_pos,y_pos is the top-left point
76         rect = surface.get_rect(topleft = (x_pos,y_pos))
77     elif point == 'topright': # if x_pos,y_pos is the top-right point
78         rect = surface.get_rect(topright = (x_pos,y_pos))
79     elif point == 'bottomleft': #if x_pos,y_pos is the bottom-left point
80         rect = surface.get_rect(bottomleft = (x_pos,y_pos))
81     else:
82         return 'Invalid information'
83     return surface,rect
84
85 @staticmethod
86 def border_schematic(rect): #if given a rectangle, will create a perimeter rectangle around it
87     border_rect = pygame.Rect(rect.left,rect.top,rect.width,rect.height)
88     return border_rect
89
90 @staticmethod
91 def begin_game(difficulty_chosen,colour_chosen):
92     #instantiates/updates all necessary variables/objects when user clicks start button (successfully)
93     global game_active
94     game_active = True
95     gamedata.difficulty = difficulty_chosen

```

```

96
97     if colour_chosen == 'w': #user chose white, A.I is black
98         gamedata.player_colour = 'w'
99         gamedata.ai_colour = 'b'
100        ai.ai_colour = 'b'
101     else: #user chose black, A.I is white
102         gamedata.player_colour = 'b'
103         gamedata.ai_colour = 'w'
104         ai.ai_colour = 'w'
105
106     chessboard.initialize_pieces()
107     ai.initialize_depth()
108     gamedata.obtain_all_valid_moves(chessboard.digital_board,'b',chessboard.class_dictionary,'first
move')
109     post_game_report.player_colour = colour_chosen
110     # if A.I is white, it already will have some moves to choose from
111
112     class main_menu:
113     def __init__(self):
114         self.is_difficulty = False #true if difficulty has been chosen, false if not
115         self.difficulty_chosen = ""
116         self.is_player_colour = False #true if player colour has been chosen, false if not
117         self.player_colour_chosen = ""
118
119     def display(self):
120         screen.fill(GREY)
121         self.display_start_button()
122         self.display_prompts()
123         self.display_title()
124         self.display_and_update_piece_select()
125         self.display_and_update_difficulty_select()
126         self.display_status_box()
127
128     def check_status(self,start_button_textbox):
129         if start_button_textbox.collidepoint(mouse_pos[0],mouse_pos[1]) == 1:
130             #if the start button has been clicked, check if game can begin
131             if self.is_difficulty == True and self.is_player_colour == True:
132                 #start button is clicked and necessary data has been entered
133                 appendices.begin_game(self.difficulty_chosen,self.player_colour_chosen)
134
135     def display_prompts(self):
136         difficulty_prompt_surface,difficulty_prompt_rect = appendices.text_schematic('• Enter
difficulty',70,620,30,RED,'midleft')
137         colour_prompt_surface,colour_prompt_rect = appendices.text_schematic('• Enter
colour',difficulty_prompt_rect.left,difficulty_prompt_rect.bottom,30,RED,'topleft')
138
139     if not self.is_difficulty:
140         screen.blit(difficulty_prompt_surface,difficulty_prompt_rect)

```

```

141     if not self.is_player_colour:
142         screen.blit(colour_prompt_surface,colour_prompt_rect)
143
144     def display_start_button(self):
145         if self.is_difficulty == True and self.is_player_colour == True:
146             border_colour = GREEN #highlight start button in green if it is allowed to be clicked
147         else:
148             border_colour = RED #highlight start button in red if details are missing
149
150         start_button_surface,start_button_rect =
appendices.text_schematic('START',580,650,50,WHITE,'centre')
151         border_rect = appendices.border_schematic(start_button_rect)
152         pygame.draw.rect(screen,border_colour,border_rect,5)
153         screen.blit(start_button_surface,start_button_rect)
154
155         self.check_status(start_button_rect)
156
157     def display_title(self):
158         title_surface,title_rect = appendices.text_schematic('CHESS',360,50,80,WHITE,'centre')
159         underline_rect = pygame.Rect(title_rect.left,title_rect.bottom,title_rect.width,20)
160         pygame.draw.rect(screen,WHITE,underline_rect,2)
161         screen.blit(title_surface,title_rect)
162
163     def display_and_update_piece_select(self):
164         if self.player_colour_chosen == 'w': #highlight around the white piece only
165             self.is_player_colour = True
166             white_colour = GREEN
167             black_colour = WHITE
168
169         elif self.player_colour_chosen == 'b': #highlight around the black piece only
170             self.is_player_colour = True
171             white_colour = WHITE
172             black_colour = GREEN
173         else: #player piece not yet chosen, don't highlight around either piece
174             white_colour = WHITE
175             black_colour = WHITE
176
177         colour_text_surface,colour_text_rect =
appendices.text_schematic('Colour',450,200,40,WHITE,'centre')
178         screen.blit(colour_text_surface, colour_text_rect)
179
180         colour_text_underline_rect =
pygame.Rect(colour_text_rect.left,colour_text_rect.bottom,colour_text_rect.width,10)
181         pygame.draw.rect(screen, WHITE, colour_text_underline_rect)
182
183         #creating and displaying the image of the white king
184         white_king_display = pygame.transform.rotozoom(piece_images_dictionary['wK'],0,1.5)

```

```

185     white_rect = white_king_display.get_rect(topleft =
(colour_text_underline_rect.left,colour_text_underline_rect.bottom+10))
186     white_border_rect = appendices.border_schematic(white_rect)
187     screen.blit(white_king_display, white_rect)
188     pygame.draw.rect(screen, white_colour, white_border_rect, 5)
189
190     #creating and displaying the image of the black king
191     black_king_display = pygame.transform.rotozoom(piece_images_dictionary['bK'],0,1.5)
192     black_rect = black_king_display.get_rect(topleft = (white_rect.right+30,white_rect.top))
193     black_border_rect = appendices.border_schematic(black_rect)
194     screen.blit(black_king_display,black_rect)
195     pygame.draw.rect(screen,black_colour,black_border_rect,5)
196
197     self.process_colour_input(white_rect,black_rect,mouse_pos)
198
199     def process_colour_input(self,white_piece_image,black_piece_image,clicked_pos):
200         #if the images of the black or white pieces are clicked, update the attributes of the colour chosen
201         if white_piece_image.collidepoint(clicked_pos[0],clicked_pos[1]) == 1:
202             self.player_colour_chosen = 'w'
203         elif black_piece_image.collidepoint(clicked_pos[0],clicked_pos[1]) == 1:
204             self.player_colour_chosen = 'b'
205
206     def display_and_update_difficulty_select(self):
207         if self.difficulty_chosen == 'Beginner': #highlight the text 'Beginner' in green only
208             self.is_difficulty = True
209             beginner_colour = GREEN
210             intermediate_colour = WHITE
211             expert_colour = WHITE
212         elif self.difficulty_chosen == 'Intermediate': #highlight the text 'Intermediate' in green only
213             self.is_difficulty = True
214             beginner_colour = WHITE
215             intermediate_colour = GREEN
216             expert_colour = WHITE
217         elif self.difficulty_chosen == 'Expert': #highlight the text 'Expert' in green only
218             self.is_difficulty = True
219             beginner_colour = WHITE
220             intermediate_colour = WHITE
221             expert_colour = GREEN
222         else: #difficulty not yet chosen, don't highlight any text
223             beginner_colour = WHITE
224             intermediate_colour = WHITE
225             expert_colour = WHITE
226
227         #creating and displaying surfaces for the difficulty UI
228         difficulty_text_surface,difficulty_text_rect =
appendices.text_schematic('Difficulty',50,200,40,WHITE,'midleft')
229         screen.blit(difficulty_text_surface,difficulty_text_rect)
230

```

```

231     difficulty_text_underline_rect =
pygame.Rect(difficulty_text_rect.left,difficulty_text_rect.bottom,difficulty_text_rect.width,10)
232     pygame.draw.rect(screen,WHITE,difficulty_text_underline_rect)
233
234     beginner_surface,beginner_rect =
appendices.text_schematic('Beginner',difficulty_text_underline_rect.left,difficulty_text_underline_rect.bott
om,30,beginner_colour,'topleft')
235     screen.blit(beginner_surface,beginner_rect)
236
237     intermediate_surface,intermediate_rect =
appendices.text_schematic('Intermediate',beginner_rect.left,beginner_rect.bottom+10,30,intermediate_col
our,'topleft')
238     screen.blit(intermediate_surface, intermediate_rect)
239
240     expert_surface,expert_rect =
appendices.text_schematic('Expert',intermediate_rect.left,intermediate_rect.bottom+10,30,expert_colour,'
topleft')
241     screen.blit(expert_surface,expert_rect)
242
243     self.process_difficulty_input(beginner_rect,intermediate_rect,expert_rect,mouse_pos)
244
245     def
process_difficulty_input(self,beginner_textbox,intermediate_textbox,expert_textbox,clicked_pos):
246     # if the 'beginner', 'intermediate' or 'expert' buttons are clicked, update the difficulty
247     if beginner_textbox.collidepoint(clicked_pos[0],clicked_pos[1]) == 1:
248         self.difficulty_chosen = 'Beginner'
249     elif intermediate_textbox.collidepoint(clicked_pos[0],clicked_pos[1]) == 1:
250         self.difficulty_chosen = 'Intermediate'
251     elif expert_textbox.collidepoint(clicked_pos[0],clicked_pos[1]) == 1:
252         self.difficulty_chosen = 'Expert'
253
254     def display_status_box(self):
255     if self.player_colour_chosen == 'w':
256         colour = 'white'
257     elif self.player_colour_chosen == 'b':
258         colour = 'black'
259     else:
260         colour = ''
261
262     difficulty_status_surface,difficulty_status_rect = appendices.text_schematic('Difficulty:
'+self.difficulty_chosen,50,500,30,WHITE,'midleft')
263     colour_status_surface,colour_status_rect = appendices.text_schematic('Colour:
'+colour,difficulty_status_rect.left,difficulty_status_rect.bottom,30,WHITE,'topleft')
264     border_rect = appendices.border_schematic(difficulty_status_rect)
265     border_rect[3] += colour_status_rect.height # updates the height of the border rectangle so that it
outlines both difficulty and colour
266     border_rect[2] = 398 #the maximum width the border rectangle will be is 398 pixels, if the word
intermediate is displayed

```

```

267
268     screen.blit(difficulty_status_surface,difficulty_status_rect)
269     screen.blit(colour_status_surface,colour_status_rect)
270     pygame.draw.rect(screen,WHITE,border_rect,2)
271
272 class chessboard:
273     def __init__(self):
274         self.class_dictionary = {}
275         #enter a co-ordinate in order to return an object (if any) of the piece on that square
276
277         self.digital_board = [
278             ['bR','bN','bB','bQ','bK','bB','bN','bR'],
279             ['bp','bp','bp','bp','bp','bp','bp','bp'],
280             ['00','00','00','00','00','00','00','00'],
281             ['00','00','00','00','00','00','00','00'],
282             ['00','00','00','00','00','00','00','00'],
283             ['00','00','00','00','00','00','00','00'],
284             ['wp','wp','wp','wp','wp','wp','wp','wp'],
285             ['wR','wN','wB','wQ','wK','wB','wN','wR']
286         ]
287
288         self.hint_move = []
289
290     def initialize_pieces(self):
291         #initiating material scores for each piece, king value already defined
292         if gamedata.difficulty == 'Expert':
293             #these are the optimal values
294             pawn.value = 1
295             knight.value = 3
296             bishop.value = 3
297             rook.value = 5
298             queen.value = 9
299         if gamedata.difficulty == 'Intermediate':
300             pawn.value = 1
301             knight.value = 4
302             bishop.value = 4
303             rook.value = 4
304             queen.value = 8
305         if gamedata.difficulty == 'Beginner':
306             pawn.value = 1
307             knight.value = 3
308             bishop.value = 3
309             rook.value = 5
310             queen.value = 5
311
312         #create black major pieces
313         self.class_dictionary[(0,0)] = rook('b',[0,0])
314         self.class_dictionary[(0,1)] = knight('b',[0,1])

```

```

315 self.class_dictionary[(0,2)] = bishop('b',[0,2])
316 self.class_dictionary[(0,3)] = queen('b',[0,3])
317 self.class_dictionary[(0,4)] = king('b',[0,4])
318 self.class_dictionary[(0,5)] = bishop('b',[0,5])
319 self.class_dictionary[(0,6)] = knight('b',[0,6])
320 self.class_dictionary[(0,7)] = rook('b',[0,7])
321
322 #create white major pieces
323 self.class_dictionary[(7,0)] = rook('w',[7,0])
324 self.class_dictionary[(7,1)] = knight('w',[7,1])
325 self.class_dictionary[(7,2)] = bishop('w',[7,2])
326 self.class_dictionary[(7,3)] = queen('w',[7,3])
327 self.class_dictionary[(7,4)] = king('w',[7,4])
328 self.class_dictionary[(7,5)] = bishop('w',[7,5])
329 self.class_dictionary[(7,6)] = knight('w',[7,6])
330 self.class_dictionary[(7,7)] = rook('w',[7,7])
331
332 #create black and white pawns
333 for create_pawn in range(cell_number):
334     self.class_dictionary[(1,create_pawn)] = pawn('b',[1,create_pawn])
335     self.class_dictionary[(6,create_pawn)] = pawn('w',[6,create_pawn])
336
337 def update(self,start_square,end_square,is_undo):
338     #updates all the common necessary objects and attributes, called every time a piece moves
339     self.update_class_dictionary(start_square,end_square,is_undo)
340     self.update_digital_board(start_square,end_square)
341
342 def update_class_dictionary(self,start_square,end_square,is_undo):
343     self.class_dictionary[(end_square[0],end_square[1])] =
self.class_dictionary[(start_square[0],start_square[1])]
344     #transfer the piece object to its new square (overwrites any piece object on that square before)
345     self.class_dictionary.pop((start_square[0], start_square[1])) #delete the piece object from its old
square
346
347     self.class_dictionary[(end_square[0],end_square[1])].position = [end_square[0],end_square[1]]
348     #updates the attribute 'position' of the piece object
349
350     #increments/decrements the attribute 'move_count' by 1
351     if is_undo:
352         self.class_dictionary[(end_square[0], end_square[1])].move_count -= 1
353     else:
354         self.class_dictionary[(end_square[0], end_square[1])].move_count += 1
355
356 def update_digital_board(self,start_square,end_square):
357     self.digital_board[end_square[0]][end_square[1]] =
self.digital_board[start_square[0]][start_square[1]]
358     #new square marked as signature of new piece
359     self.digital_board[start_square[0]][start_square[1]] = '00'

```

```

360     #old square marked as empty
361
362     def add_new_object(self,square,class_info):
363         #creates new object in the class dictionary, and updates digital board
364         self.class_dictionary[(square[0],square[1])] = class_info
365         self.digital_board[square[0]][square[1]] = class_info.initial
366
367     def display_valid_moves(self,clicked_pos):
368         available_moves = self.generate_valid_moves(clicked_pos)
369         #returns a list of moves, containing the start and end squares
370
371         if gamedata.player_colour == 'b':
372             inverse_multiplier = 7 #used in order optimise blacks perspective by vertically inverting the
board
373         else:
374             inverse_multiplier = 0
375         for move in available_moves:
376             end_square_x = move[1][1]
377             end_square_y = move[1][0]
378             x_pos = (end_square_x * cell_size) + (cell_size // 2)
379             y_pos = (abs(inverse_multiplier - end_square_y) * cell_size) + (cell_size // 2)
380             #the 2 lines above provide the co-ordinates of the centre of each square the piece could go to
381             pygame.draw.circle(screen,GREY,(x_pos,y_pos),10)
382             pygame.display.update()
383
384         return available_moves
385
386     def generate_valid_moves(self,clicked_pos):
387         if self.digital_board[clicked_pos[0]][clicked_pos[1]][0] == current_turn:
388             #user has clicked on one of their pieces
389
390             piece_object = self.class_dictionary[(clicked_pos[0],clicked_pos[1])]
391             if piece_object.colour == 'w':
392                 king_pos = gamedata.white_king_pos
393             else:
394                 king_pos = gamedata.black_king_pos
395             available_moves = piece_object.calculate_moves(self.digital_board,king_pos)
396             return available_moves
397         else: #user has clicked on an empty square or on a square of a piece it cannot control
398             return []
399
400     def display_board(self,colour1,colour2):
401         self.display_chessboard_pattern(colour1,colour2)
402         self.display_hint()
403         self.display_labels(colour1,colour2)
404         self.display_pieces()
405

```

```

406 def display_chessboard_pattern(self,colour1,colour2): #input 2 colours, and a chequered pattern is
created

407     screen.fill(colour1)
408     for row in range(cell_number):
409         for col in range(cell_number):
410             if ((row % 2) == 0 and (col % 2) == 0) or ((row % 2) == 1 and (col % 2) == 1):
411                 square_rect = pygame.Rect(row*cell_size,col*cell_size,cell_size,cell_size)
412                 pygame.draw.rect(screen,colour2,square_rect)
413
414 def display_labels(self,colour1,colour2): #creating the file labels 'a-h', and rank labels '1-8'
415     file_order = ['a','b','c','d','e','f','g','h']
416     if gamedata.player_colour == 'w':
417         rank_order = ['8','7','6','5','4','3','2','1']
418     else: #if player_colour = black
419         rank_order = ['1','2','3','4','5','6','7','8']
420
421     for square in range(cell_number):
422         if square % 2 == 1:
423             label_colour = colour1
424         else:
425             label_colour = colour2
426
427     file_surface,file_rect =
appendices.text_schematic(file_order[square],square*cell_size,cell_number*cell_size,20,label_colour,'bott
omleft')
428     rank_surface,rank_rect =
appendices.text_schematic(rank_order[square],cell_number*cell_size,square*cell_size,20,label_colour,'top
right')
429     screen.blit(file_surface,file_rect)
430     screen.blit(rank_surface,rank_rect)
431
432 def display_pieces(self):
433     if gamedata.player_colour == 'b':
434         inverse_multiplier = 7 #used in order optimise blacks perspective by vertically inverting the
board
435     else:
436         inverse_multiplier = 0
437     for row in range(cell_number):
438         for col in range(cell_number):
439             square = self.digital_board[row][col]
440             if square != '00': #if the position is not an empty square
441                 piece_image = piece_images_dictionary[square]
442                 square_rect = piece_image.get_rect(topleft =
(col*cell_size,abs(inverse_multiplier-row)*cell_size))
443                 screen.blit(piece_image,square_rect)
444
445 def display_hint(self):
446     if self.hint_move == []:

```

```

447     return
448
449     if gamedata.player_colour == 'b': #used in order optimise blacks perspective by vertically inverting
the board
450         inverse_multiplier = 7
451     else:
452         inverse_multiplier = 0
453
454     start_square_x = self.hint_move[0][1] * cell_size
455     start_square_y = abs(inverse_multiplier-self.hint_move[0][0]) * cell_size
456     end_square_x = self.hint_move[1][1] * cell_size
457     end_square_y = abs(inverse_multiplier-self.hint_move[1][0]) * cell_size
458
459
460     #shade the start and end squares of the hint move in orange
461     start_square_rect = pygame.Rect(start_square_x,start_square_y,cell_size,cell_size)
462     pygame.draw.rect(screen,ORANGE,start_square_rect)
463
464     end_square_rect = pygame.Rect(end_square_x,end_square_y,cell_size,cell_size)
465     pygame.draw.rect(screen,ORANGE,end_square_rect)
466
467 class move_engine:
468     @staticmethod
469     def initiate_move(move_data):
470         #this is called to initiate a move. Everything in the program is updated here including:
471         #1. Updating the gamedata attribute capture stack
472         #2. updating the chessboard object
473         #3. Initializing special moves such as castling and pawn promotions (if executed by player/A.I)
474         #4. Updating the gamedata object
475         #5. Flipping the current_turn
476         #6. Emptying the contents of move_data
477
478     global current_turn
479     start_square = move_data[0]
480     end_square = move_data[1]
481
482     #1.
483     gamedata.log_captured_piece(end_square,chessboard.class_dictionary)
484
485     #2.
486     chessboard.update(start_square,end_square,False)
487
488     #3.
489     piece_object = chessboard.class_dictionary[(end_square[0],end_square[1])]
490     #gets object of piece that just moved
491     if type(piece_object).__name__ == 'king': # if the piece_object moved is the king
492         move_engine.initiate_move_king(start_square,end_square,piece_object)

```

```

493
494     if type(piece_object).__name__ == 'pawn': # if the piece_object moved is a pawn, may promote
495         move_engine.pawn_promotion(end_square,piece_object)

496
497     #4.
498
499     gamedata.update(chessboard.digital_board,current_turn,chessboard.class_dictionary,move_data,False)
500
501     #5.
502     current_turn = appendices.flip_current_turn(current_turn)
503
504     #6.
505     return []
506
507 @staticmethod
508 def initiate_move_king(start_square,end_square,king_object):
509     # 1.check if castled
510     if abs(end_square[1]-start_square[1]) == 2: # if the king has moved 2 squares to the right/left
511         i.e.castled
512         king_object.castled = True
513         # rook also needs to move, which is what the lines below achieve
514         if end_square[1] == 6: # if player castled king side
515             rook_start_pos = (start_square[0],7)
516             rook_new_pos = (start_square[0],5)
517         else: # if player castled queen side
518             rook_start_pos = (start_square[0],0)
519             rook_new_pos = (start_square[0],3)
520         chessboard.update(rook_start_pos,rook_new_pos,False)
521
522     # 2.update white/black_king_pos attributes
523     if king_object.colour == 'w':
524         gamedata.white_king_pos = (end_square[0],end_square[1])
525     else:
526         gamedata.black_king_pos = (end_square[0],end_square[1])
527
528 @staticmethod
529 def undo_move_king(start_square,end_square,king_object):
530     # 1.check if de-castled
531     if abs(end_square[1]-start_square[1]) == 2: # if the king has moved 2 squares to the right/left
532         i.e.castled
533         king_object.castled = False
534
535     #rook also needs to move, which is what the lines below achieve
536     if start_square[1] == 6: #if player castled king side
537         rook_start_pos = (start_square[0],5)
538         rook_new_pos = (start_square[0],7)
539     else: # if player castled queen side
540         rook_start_pos = (start_square[0],3)

```

```

538     rook_new_pos = (start_square[0],0)
539     chessboard.update(rook_start_pos,rook_new_pos,True)
540
541     # 2.update white/black_king_pos attributes
542     if king_object.colour == 'w':
543         gamedata.white_king_pos = (end_square[0],end_square[1])
544     else:
545         gamedata.black_king_pos = (end_square[0],end_square[1])
546
547     @staticmethod
548     def pawn_promotion(end_square,pawn_object):
549         if pawn_object.check_for_promote(): #pawn can promote. Replace pawn class object to queen
550
551         chessboard.add_new_object(end_square,queen(pawn_object.colour,[end_square[0],end_square[1]],True))
552         #final argument 'True' represents the fact that this object originates from a promoted pawn
553
554     @staticmethod
555     def check_depromote(end_square,piece_object):
556         if piece_object.is_promoted and piece_object.move_count == -1:
557             #if the piece object stems from a pawn and hasn't moved yet
558
559         chessboard.add_new_object(end_square,pawn(piece_object.colour,[end_square[0],end_square[1]]))
560
561     @staticmethod
562     def undo_move():
563         #this is called to undo a move. Everything in the program is updated here including:
564         #1. Access the previous move played from the move log held in the gamedata class
565         #2. Updating the chessboard object (for piece moving back)
566         #3. Recover the captured piece (if there is one for that turn)
567         #4. De-castle (if necessary)
568         #5. De-promote (if necessary)
569         #6. Flipping the current turn
570         #7. Updating the gamedata object
571         global current_turn
572
573         #1.
574         previous_move = gamedata.pop_move_log()
575         start_square = previous_move[1]
576         end_square = previous_move[0]
577
578         #2.
579         chessboard.update(start_square,end_square,True)
580
581         #3.
582         recovered_piece = gamedata.pop_capture_log() #obtains the object of the last captured piece (if
any)
583         if recovered_piece != '.':

```

```

583     chessboard.add_new_object(start_square,recovered_piece)
584
585     #4.

586     piece_object = chessboard.class_dictionary[(end_square[0],end_square[1])] #object of piece
moving back
587     if type(piece_object).__name__ == 'king': #if the piece_object moved is the king
588         move_engine.undo_move_king(start_square,end_square,piece_object)
589
590     #5.
591     move_engine.check_depromote(end_square,piece_object)
592
593     #6.
594     current_turn = appendices.flip_current_turn(current_turn)
595
596     #7.
597
gamedata.update(chessboard.digital_board,current_turn,chessboard.class_dictionary,(start_square,end_sq
uare),True)
598
599 class post_game:
600     def __init__(self):
601         self.evaluation_queue = [0]
602         self.player_colour = ''
603         self.standard_moves = 0
604         self.strong_moves = 0
605         self.blunder_moves = 0
606
607     def push_evaluation_queue(self,is_checkmate_or_stalemate,winner,evaluation):
608         if is_checkmate_or_stalemate:
609             if winner == 'w':
610                 self.evaluation_queue.append(MAX_SCORE)
611             elif winner == 'b':
612                 self.evaluation_queue.append(MIN_SCORE)
613             else:
614                 self.evaluation_queue.append(0)
615         else:
616             self.evaluation_queue.append(evaluation)
617
618     def pop_evaluation_queue(self):
619         self.evaluation_queue.pop()
620
621     def evaluate_all_player_moves(self):
622         if self.player_colour == 'w':
623             player_move_sync = 1 #every odd index in the queue is an evaluation for white
624         else:
625             player_move_sync = 0 #every even index (other than index 0) in the queue is an evaluation for
black
626

```

```

627     for move_number in range(len(self.evaluation_queue)):
628         if move_number % 2 == player_move_sync and move_number != 0: #player made move at said
'move number'

629         self.evaluate_single_player_move(move_number)
630
631     def evaluate_single_player_move(self,move_number):
632         if move_number == len(self.evaluation_queue) -1: #final move
633             player_evaluation = self.evaluation_queue[move_number]
634             if self.player_colour == 'b': #if player is black, a negative score after is good
635                 player_evaluation *= -1
636         else:
637             player_evaluation = self.find_evaluation_difference(move_number-1,move_number+1)
638
639         if player_evaluation > 3:
640             self.strong_moves += 1
641         elif player_evaluation < -5:
642             self.blunder_moves += 1
643         else:
644             self.standard_moves += 1
645
646     def find_evaluation_difference(self,ai_move_before,ai_move_after):
647         ai_evaluation_before = self.evaluation_queue[ai_move_before]
648         ai_evaluation_after = self.evaluation_queue[ai_move_after]
649
650         evaluation_difference = ai_evaluation_after-ai_evaluation_before
651         if self.player_colour == 'b': #if player is black, a negative score after is good
652             evaluation_difference *= -1
653
654         return evaluation_difference
655
656     def display(self,move_count,winner):
657         screen.fill(GREY)
658         self.display_game_over()
659         self.display_winner(winner)
660         self.display_game_length(move_count)
661         self.display_move_analysis()
662
663     def display_game_over(self):
664         game_over_surface,game_over_rect = appendices.text_schematic('GAME
OVER',360,50,80,WHITE,'centre')
665         underline_rect =
pygame.Rect(game_over_rect.left,game_over_rect.bottom,game_over_rect.width,20)
666         pygame.draw.rect(screen,WHITE,underline_rect,2)
667         screen.blit(game_over_surface,game_over_rect)
668
669     def display_winner(self,winner):
670         #if statements ensure that 'winner' is written in full
671         if winner == 'w':

```

```

672     winner = 'White'
673     elif winner == 'b':
674         winner = 'Black'

675     else: #stalemate
676         winner = 'Draw'
677
678     winner_surface, winner_rect = appendices.text_schematic('Winner: ' +
winner,100,150,40,WHITE,'topleft')
679     screen.blit(winner_surface, winner_rect)
680
681     def display_game_length(self, move_count):
682         game_length_surface, game_length_rect = appendices.text_schematic('Length of game:
'+str(move_count)+' moves',100,250,40,WHITE,'topleft')
683
684         screen.blit(game_length_surface, game_length_rect)
685
686     def display_move_analysis(self):
687         #displaying the standard move count
688         standard_move_surface, standard_move_rect = appendices.text_schematic('Standard moves: ' +
str(self.standard_moves),100,350,40,WHITE,'topleft')
689         screen.blit(standard_move_surface, standard_move_rect)
690
691         #displaying the strong move count
692         strong_move_surface, strong_move_rect = appendices.text_schematic('Strong moves: ' +
str(self.strong_moves),100,450,40,WHITE,'topleft')
693         screen.blit(strong_move_surface, strong_move_rect)
694
695         # displaying the blunder move count
696         blunder_move_surface, blunder_move_rect = appendices.text_schematic('blunder moves: ' +
str(self.blunder_moves),100, 550, 40, WHITE, 'topleft')
697         screen.blit(blunder_move_surface, blunder_move_rect)
698
699 #instantiating classes
700 main_menu = main_menu()
701 chessboard = chessboard()
702 gamedata = gamedata()
703 ai = AI(move_engine,gamedata,chessboard)
704 post_game_report = post_game()
705
706 piece_images_dictionary = appendices.map_piece_to_image()
707 pos_to_square_dictionary = appendices.map_pos_to_square()
708
709 def main(): #main game loop
710     global mouse_pos,current_turn,game_active
711     move_data = [] #contains 2 tuples: location of start and end squares clicked by the user
712     while True:
713         mouse_pos = (0,0)
714         for event in pygame.event.get():

```

```

715     if event.type == pygame.QUIT: #close window
716         pygame.quit()
717         sys.exit()

718
719     if event.type == pygame.KEYDOWN and game_active:
720         if event.key == pygame.K_u: # 'U' key is pressed = U = Undo move
721             if len(gamedata.move_log_stack) != 0 and len(gamedata.move_log_stack) != 1:
722                 #cannot undo a move if no moves have taken place
723
724                 move_engine.undo_move()
725                 post_game_report.pop_evaluation_queue()
726                 move_engine.undo_move() #called twice as A.I move also needs to be undone
727                 post_game_report.pop_evaluation_queue()
728                 move_data = []
729                 game_active = True
730             elif event.key == pygame.K_h and chessboard.hint_move == []: # 'H' key is pressed = H = Show
731                 hint
732                 chessboard.hint_move = ai.generate_move(True)
733
734     if event.type == pygame.MOUSEBUTTONDOWN: #user click
735         mouse_pos = pygame.mouse.get_pos()
736         if game_active:
737             clicked_pos = appendices.get_col_row_from_pos(mouse_pos[0],mouse_pos[1])
738             if move_data == []:
739                 move_data.append(clicked_pos)
740             else:
741                 if clicked_pos == move_data[0]: #same square is clicked twice: deselect
742                     move_data = []
743                 else:
744                     move_data.append(clicked_pos)
745
746     if not game_active: #display main menu/post_game_report if false, display game if true
747         if gamedata.is_checkmate or gamedata.is_stalemate: #after game is finished
748             post_game_report.display(gamedata.move_count,gamedata.winner)
749         else: #before game begins
750             main_menu.display()
751     else: #game is active, display chessboard
752         chessboard.display_board(MINT,ARCTIC)
753         if current_turn != gamedata.player_colour: #AI's turn
754             move_data = ai.generate_move()
755             move_data = move_engine.initiate_move(move_data)
756
757         if gamedata.is_checkmate or gamedata.is_stalemate:
758             post_game_report.push_evaluation_queue(True, gamedata.winner, 'filler')
759             post_game_report.evaluate_all_player_moves()
760             game_active = False
761         else:

```

```

761
post_game_report.push_evaluation_queue(False,'filler',ai.evaluation_function(chessboard.digital_board))
762
763     if len(move_data) == 1: #user has clicked on a square once

764         available_moves = chessboard.display_valid_moves(move_data[0])
765     elif len(move_data) == 2:
766         if move_data in available_moves:
767             #if the second square clicked is a possible destination from the first
768
769             move_data = move_engine.initiate_move(move_data)
770             chessboard.hint_move = []
771             chessboard.display_board(MINT,ARCTIC) #update board
772             if gamedata.is_checkmate or gamedata.is_stalemate:
773                 post_game_report.push_evaluation_queue(True,gamedata.winner,'filler')
774                 post_game_report.evaluate_all_player_moves()
775                 game_active = False
776             else:
777
post_game_report.push_evaluation_queue(False,'filler',ai.evaluation_function(chessboard.digital_board))
778         else: #if the second square clicked is independent from the first
779             move_data.pop(0) #have the second square chosen be the first square now
780             #this also handles the situation when the first square by user is 'invalid'. If invalid,
781             #available_moves will empty, so when the second item of move_data is entered, the first
782             #invalid item will ALWAYS be popped
783
784     pygame.display.update()
785     clock.tick(60)
786
787 if __name__ == '__main__':
788     main()

```

Pieces.py

```

1  from copy import deepcopy
2  from constants import MAX_SCORE
3
4  def check_for_checks(board,colour,king_pos,get_changed_squares = False):
5      changed_squares = []
6      king_row,king_col = king_pos
7      #king_pos: position of the king to be inspected
8      #colour: colour of the king to be inspected
9
10     #For each possible vector traverse direction. If the matching opposite piece is within the path return
11     #True (king is being checked)
12
13     piece_vectors = [(1,0),(0,1),(-1,0),(0,-1), (1,1),(1,-1),(-1,1),(-1,-1),
14                     (1,2),(2,1),(-1,2),(2,-1),(1,-2),(-2,1),(-1,-2),(-2,-1)]
15     #1-4: rook,queen,king| 5-8: bishop,queen,king| 9-16: knight
16
17     for vector_no,vector in enumerate(piece_vectors):
18         row_vector,col_vector = vector[0],vector[1]
19         king_new_row,king_new_col = king_row + row_vector, king_col + col_vector
20
21         if 0 <= king_new_row <= 7 and 0 <= king_new_col <= 7:
22             if (0 <= vector_no <= 7) and board[king_new_row][king_new_col][1] == 'K':
23                 changed_squares.append((king_new_row,king_new_col))
24                 if board[king_new_row][king_new_col][0] != colour and not get_changed_squares:
25                     return True # if an enemy king is adjacent to the king
26             elif (8 <= vector_no <= 15) and board[king_new_row][king_new_col][1] == 'N':
27                 changed_squares.append((king_new_row, king_new_col))
28                 if board[king_new_row][king_new_col][0] != colour and not get_changed_squares:
29                     return True # if an enemy knight is an 'L' shaped distance away from the king
30             if 8 <= vector_no <= 15:
31                 continue # vectors 9-16 are only for knights, so the code below can be skipped
32
33     # while loop to check for ranged pieces only
34     while 0 <= king_new_row <= 7 and 0 <= king_new_col <= 7:
35         if board[king_new_row][king_new_col] != '00': # if not an empty square
36             if (0 <= vector_no <= 3) and (board[king_new_row][king_new_col][1] == 'R' or
board[king_new_row][king_new_col][1] == 'Q'):
37                 changed_squares.append((king_new_row,king_new_col))
38                 if board[king_new_row][king_new_col][0] != colour and not get_changed_squares: #enemy
piece
39                     return True # if an enemy queen/rook is horizontal or vertical to the king
40             elif (4 <= vector_no <= 7) and (board[king_new_row][king_new_col][1] == 'B' or
board[king_new_row][king_new_col][1] == 'Q'):
41                 changed_squares.append((king_new_row,king_new_col))
42                 if board[king_new_row][king_new_col][0] != colour and not get_changed_squares: #enemy
piece
43                     return True # if an enemy queen/bishop is diagonal to the king

```

```

44         break
45
46     #for each direction, keep traversing the paths for the king until the square is outside the board,
47     #or there is a piece blocking the direction (also examine said piece)
48     king_new_row += row_vector
49     king_new_col += col_vector
50
51 if get_changed_squares:
52     return changed_squares
53
54 # check if pawns are attacking
55 if colour == 'w':
56     row_vector = -1 #black pawns will be on the top end of the board
57 else:
58     row_vector = 1 #white pawns will be on the bottom end of the board
59
60 if 0 <= king_row + row_vector <= 7:
61     col_vectors = [1,-1]
62     for col in col_vectors:
63         if 1 <= king_col + col <= 7:
64             if board[king_row + row_vector][king_col + col][0] != colour:
65                 if board[king_row + row_vector][king_col + col][1] == 'p':
66                     return True
67
68 return False # no checks identified
69
70 class chesspiece:
71     def __init__(self,colour,position,signature,is_promoted,value):
72         self.initial = colour[0]+signature
73         self.colour = colour[0] #first letter of colour is used ('w' = white, 'b' = black)
74         self.position = position
75         self.move_count = 0
76         self.is_promoted = is_promoted #If 'True', the chess piece is stemmed from a promoted pawn
77         self.valid_move_list = []
78
79         if self.colour == 'w':
80             self.value = value
81         elif self.colour == 'b': #black pieces possess negative value
82             self.value = value * -1
83
84     def calculate_moves(self,vectors,board,is_range,king_pos):
85         #this method will be overridden by most of its sub-classes
86         valid_moves = [] #each piece of data will contain:(start_square,end_square),start_square =
self.position
87         row,col = self.position
88
89         for vector in vectors:
90             row_vector,col_vector = vector[0],vector[1]

```

```

91     new_row,new_col = row+row_vector,col+col_vector
92
93     if not is_range: #used for the king and knights, as they do not possess range
94         if 0 <= new_row <= 7 and 0 <= new_col <= 7:
95             if board[new_row][new_col][0] != self.colour:
96                 #ensures that the new position isn't already occupied by a piece of the same colour
97                 if not self.is_checks(board,(new_row,new_col),king_pos):
98                     start_square = tuple(self.position)
99                     end_square = (new_row,new_col)
100                     valid_moves.append([start_square,end_square])
101
102     else: #used for the queen, rook and bishop, as they do possess range
103         while 0 <= new_row <= 7 and 0 <= new_col <= 7:
104             if board[new_row][new_col] != '00': #if not an empty square
105                 if board[new_row][new_col][0] != self.colour: #if it is an enemy piece
106                     if not self.is_checks(board,(new_row,new_col),king_pos):
107                         start_square = tuple(self.position)
108                         end_square = (new_row,new_col)
109                         valid_moves.append([start_square,end_square])
110                     break
111
112             if not self.is_checks(board,(new_row,new_col),king_pos):
113                 start_square = tuple(self.position)
114                 end_square = (new_row,new_col)
115                 valid_moves.append([start_square,end_square])
116             new_row += row_vector
117             new_col += col_vector
118
119     return valid_moves
120
121 def is_checks(self,board,end_square,king_pos):
122     #ensures that a potential valid move will NOT leave its own king being exposed
123
124     duplicate_board = deepcopy(board)
125     #this variable will be tweaked without changing the contents of the original board
126
127     #Make the move on the duplicate board
128     duplicate_board[end_square[0]][end_square[1]] =
duplicate_board[self.position[0]][self.position[1]]
129     duplicate_board[self.position[0]][self.position[1]] = '00'
130
131     if duplicate_board[end_square[0]][end_square[1]][1] == 'K':
132         king_pos = end_square[0],end_square[1] #position of king needs to be updated if king is moving
133
134     return check_for_checks(duplicate_board,self.colour,king_pos)
135
136 class rook(chesspiece):
137     value = None

```

```

138
139 def __init__(self,colour,position,is_promoted_piece = False):
140     super().__init__(colour,position,'R',is_promoted_piece,rook.value)
141
142 def calculate_moves(self,board,king_pos):
143     rook_vectors = [(1,0),(0,1),(-1,0),(0,-1)] #movement patterns for a rook
144     self.valid_move_list = super().calculate_moves(rook_vectors,board,True,king_pos)
145     return self.valid_move_list
146
147 class bishop(chesspiece):
148     value = None
149
150 def __init__(self,colour,position,is_promoted_piece = False):
151     super().__init__(colour,position,'B',is_promoted_piece,bishop.value)
152
153 def calculate_moves(self,board,king_pos):
154     bishop_vectors = [(1,1),(1,-1),(-1,1),(-1,-1)] #movement patterns for a bishop
155     self.valid_move_list = super().calculate_moves(bishop_vectors,board,True,king_pos)
156     return self.valid_move_list
157
158 class queen(chesspiece):
159     value = None
160
161 def __init__(self,colour,position,is_promoted_piece = False):
162     super().__init__(colour,position,'Q',is_promoted_piece,queen.value)
163
164 def calculate_moves(self,board,king_pos): #queen_moves = rook_moves + bishop_moves
165     queen_vectors = [(1,0),(0,1),(-1,0),(0,-1),(1,1),(1,-1),(-1,1),(-1,-1)]
166     self.valid_move_list = super().calculate_moves(queen_vectors,board,True,king_pos)
167     return self.valid_move_list
168
169 class knight(chesspiece):
170     value = None
171
172 def __init__(self,colour,position,is_promoted_piece = False):
173     super().__init__(colour,position,'N',is_promoted_piece,knight.value)
174
175 def calculate_moves(self,board,king_pos):
176     knight_vectors = [(1,2),(2,1),(-1,2),(2,-1),(1,-2),(-2,1),(-1,-2),(-2,-1)]
177     self.valid_move_list = super().calculate_moves(knight_vectors,board,False,king_pos)
178     return self.valid_move_list
179
180 class king(chesspiece):
181     value = MAX_SCORE #999999999
182
183 def __init__(self,colour,position,is_promoted_piece = False):
184     super().__init__(colour,position,'K',is_promoted_piece,king.value)
185     self.castled = False

```

```

186
187 def calculate_moves(self,board,king_pos):
188     king_vectors = [(1,0),(0,1),(0,-1),(-1,0),(1,-1),(-1,1),(1,1),(-1,-1)]
189     standard_moves = super().calculate_moves(king_vectors,board,False,king_pos)
190     castling_moves = self.castle(board,king_pos)
191     self.valid_move_list = castling_moves + standard_moves
192     return self.valid_move_list
193
194 def castle(self,board,king_pos):
195     castling_moves = []
196     if self.move_count != 0: #illegal to castle if king has already moved
197         return castling_moves
198
199     row = self.position[0] #only row is required
200
201     #castling queen side
202     if (board[row][1] == '00') and (board[row][2] == '00') and (board[row][3] == '00'):
203         #checks if squares between rook (queen side) and king are empty
204
205         if board[row][0] == self.colour + 'R': #if rook on edge is same colour to king
206             end_square = (row,2)
207             if not self.is_checks(board, end_square,king_pos):
208                 start_square = tuple(self.position)
209                 castling_moves.append([start_square,end_square]) #add castle queen side to
castling_moves
210
211     #castling king side
212     if (board[row][5] == '00') and (board[row][6] == '00'):
213         # checks if squares between rook (king side) and king are empty
214
215         if board[row][7] == self.colour + 'R':
216             end_square = (row,6)
217             if not self.is_checks(board,end_square,king_pos):
218                 start_square = tuple(self.position)
219                 castling_moves.append([start_square,end_square]) #add castle king side to castling moves
220
221     return castling_moves
222
223 class pawn(chesspiece):
224     value = None
225
226     def __init__(self,colour,position,is_promoted_piece = False):
227         super().__init__(colour,position,'p',is_promoted_piece,pawn.value)
228         self.capture_move_list = [] #used to evaluate centre control
229
230     def calculate_moves(self,board,king_pos):
231         if self.colour == 'w': #white pawns move up the board
232             row_vector = -1

```

```

233     else: #black pawns move down the board
234         row_vector = 1
235
236         standard_moves = self.move_vertical(board,row_vector,king_pos) #pawn is only moving vertically,
no capturing
237         self.capture_move_list = self.capture(board,row_vector,king_pos) #pawn is moving diagonally,
capturing
238
239         self.valid_move_list = self.capture_move_list + standard_moves
240         return self.valid_move_list
241
242     def move_vertical(self,board,row_vector,king_pos):
243         standard_moves = []
244         row,col = self.position
245         new_row = row+row_vector
246
247         if board[new_row][col] == '00': #checks if square in front is empty
248             if not self.is_checks(board,(new_row,col),king_pos):
249                 start_square = tuple(self.position)
250                 end_square = (new_row,col)
251                 standard_moves.append([start_square,end_square])
252         else:
253             return standard_moves #no need to look further if square in front is blocked
254
255         if self.move_count == 0: #first move
256             new_row = row+(row_vector*2)
257             if board[new_row][col] == '00':
258                 if not self.is_checks(board,(new_row,col),king_pos):
259                     start_square = tuple(self.position)
260                     end_square = (new_row,col)
261                     standard_moves.append([start_square,end_square])
262
263         return standard_moves
264
265     def capture(self,board,row_vector,king_pos,centre_scanning = False):
266         central_moves = []
267         capture_moves = []
268         row,col = self.position
269         new_row = row+row_vector
270         col_vectors = [-1,1]
271
272         for col_vector in col_vectors:
273             new_col = col+col_vector
274             if 0 <= new_col <= 7 and 0 <= new_row <= 7:
275                 start_square = tuple(self.position)
276                 end_square = (new_row,new_col)
277
278                 if centre_scanning: #doesn't matter whether diagonal square is filled or not

```

```
279         central_moves.append([start_square,end_square])
280     else:
281         if board[new_row][new_col] != '00': #if not an empty square
282             if board[new_row][new_col][0] != self.colour: #if it is an enemy piece
283                 if not self.is_checks(board,end_square,king_pos):
284                     capture_moves.append([start_square,end_square])
285
286     if centre_scanning:
287         return central_moves
288
289     return capture_moves
290
291 def check_for_promote(self):
292     #called every time a pawn is moved. If pawn at end row, then it should promote
293
294     current_row = self.position[0]
295     if self.colour == 'w': #the top row is the row where white pawns promote
296         opposite_row = 0
297     else: #the bottom row is the row where black pawns promote
298         opposite_row = 7
299     if current_row == opposite_row: #pawn is at opposite row of board
300         return True
301
302     return False
```

AI.py

```

1  from constants import MAX_SCORE, MIN_SCORE
2
3  class AI:
4      def __init__(self, move_engine_class, gamedata_object, chessboard_object):
5          self.gamedata_object = gamedata_object
6          self.chessboard_object = chessboard_object
7          self.move_engine = move_engine_class
8          self.ai_colour = self.gamedata_object.ai_colour
9          self.difficulty_depth = 0 #not defined yet
10         self.hint_depth = 3 # highest depth available: we want most accurate move as hint for user
11         self.max_depth = 0 #changes per minimax call - depending on whether minimax call is normal or for
hint
12
13         self.best_move = [] #stores only a single tuple - the best move currently found by minimax
14
15     def initialize_depth(self): #called after user picks difficulty from main menu
16         if self.gamedata_object.difficulty == 'Expert':
17             self.difficulty_depth = 3
18         elif self.gamedata_object.difficulty == 'Intermediate':
19             self.difficulty_depth = 2
20         else: #difficulty = beginner
21             self.difficulty_depth = 1
22
23     def generate_move(self, generate_hint = False):
24         self.best_move = []
25
26         if self.gamedata_object.ai_colour == 'w': #white = maximizing
27             is_maximising = True
28         else: #black = minimizing
29             is_maximising = False
30
31         if generate_hint:
32             is_maximising = not is_maximising #e.g. if A.I = white, player = black, so needs to be False
33             self.max_depth = self.hint_depth #highest depth available: we want most accurate move as hint
for user
34         else:
35             self.max_depth = self.difficulty_depth #depends on difficulty of A.I
36
37         #call minimax
38         score =
self.minimax(self.chessboard_object.digital_board, is_maximising, self.max_depth, MIN_SCORE, MAX_SCORE)
39         #we don't actually need the score, but may be handy for testing later
40
41         best_move = self.best_move #the best_move attribute is altered during the evaluation function
42         return best_move
43
44     def evaluation_function(self, board):

```

```

45     material_score = 0
46     centre_control_score = 0
47     pawn_structure_score = 0
48
49     '''
50     • loop through every square on board. For each square:
51     #1. Add/subtract to material score (if there is a piece on the square)
52     #2. Add/subtract to centre control score if piece controls any central squares
53     #3. Add/subtract to pawn structure score if piece is a pawn (evaluate its position)
54
55     • Afterwards, evaluate the safety of the white and black kings, which comprise the king safety score
56
57     • Calculate the weights of each components, depending on the phase of the game
58     '''
59
60     for row_pos,row in enumerate(board): #index of position also obtained using enumerate
61         for col_pos,piece in enumerate(row):
62             if piece != '00':
63                 piece_object = self.chessboard_object.class_dictionary[(row_pos,col_pos)]
64                 material_score += self.evaluate_material_score(piece_object)
65                 centre_control_score += self.evaluate_centre_control(piece_object)
66                 pawn_structure_score += self.evaluate_pawn_structure(board,piece_object)
67
68     king_safety_score = self.evaluate_king_safety(board)
69     total_score =
self.calculate_total_score(material_score,centre_control_score,pawn_structure_score,king_safety_score)
70
71     return total_score
72
73 def evaluate_material_score(self,piece_object):
74     return piece_object.value
75
76 def evaluate_centre_control(self,piece_object):
77     central_squares = [(3,3),(3,4),(4,3),(4,4)]
78     piece_in_centre_bonus = 6 #values tweaked for optimisation
79     pawn_controlling_centre_bonus = 5
80     non_pawn_controlling_centre_bonus = 2
81
82     if piece_object.colour == 'w':
83         row_vector = -1
84         multiplier = 1
85     else: #Black piece
86         row_vector = 1
87         multiplier = -1
88
89     if tuple(piece_object.position) in central_squares: #if the piece is already in the centre
90         return piece_in_centre_bonus * multiplier
91

```

```

92     if type(piece_object).__name__ == 'pawn': #pawns are more valuable to control centre, thus more
points
93         value = pawn_controlling_centre_bonus
94         watched_squares = piece_object.capture('filler',row_vector,'filler',True)
95         #pawns can only control squares diagonal to them, which is why we use the capture method
96     else:
97         value = non_pawn_controlling_centre_bonus
98         #other pieces score less points than pawns if they control the centre
99         watched_squares = piece_object.valid_move_list
100
101     for square in watched_squares:
102         if square[1] in central_squares: #if one of the destination squares is a central square
103             return value * multiplier
104
105     return 0 #pieces is not controlling any central squares
106
107 def evaluate_pawn_structure(self,board,piece_object):
108     if type(piece_object).__name__ != 'pawn': #no need to be evaluated if piece is not a pawn
109         return 0
110
111     pawn_structure_score = 0 #valuestweaked for optimisation
112     pawn_island_penalty = -1
113     double_pawn_penalty = -3 #each pawn on same row e.g. 3 pawns on same row = -6
114     pawn_island = True
115
116     if piece_object.colour == 'w':
117         multiplier = 1
118     else: #Black piece
119         multiplier = -1
120
121     row,col = piece_object.position
122     surrounding_squares = [(0,1),(0,-1),(1,0),(-1,0),(1,1),(1,-1),(-1,1),(-1,-1)]
123
124     for square in surrounding_squares:
125         new_row,new_col = row+square[0],col+square[1]
126         if 0 <= new_row <= 7 and 0 <= new_col <= 7:
127             if board[new_row][new_col] == piece_object.initial:
128                 #this means that there is a friendly pawn next to this one, NOT isolated
129                 pawn_island = False
130                 break
131
132     if pawn_island:
133         pawn_structure_score += pawn_island_penalty
134
135     while row < 7: #only need to look one way, if its below one pawn, then its above another pawn
136         if board[row+1][col] == piece_object.initial: #another pawn on same row, doubled (or worse)
137             pawn_structure_score += double_pawn_penalty
138             break

```

```

139     row += 1
140
141     return pawn_structure_score * multiplier
142
143 def evaluate_king_safety(self, board):
144     king_safety_score = 0
145
146     white_king_pos = self.gamedata_object.white_king_pos
147     white_king_object = self.chessboard_object.class_dictionary[white_king_pos]
148     black_king_pos = self.gamedata_object.black_king_pos
149     black_king_object = self.chessboard_object.class_dictionary[black_king_pos]
150
151     col_vectors = [-1,0,1]
152     castling_bonus = 3 #tweaked for optimisation
153     pawn_shield_bonus = 1 #per pawn
154
155     #deduce if kings have yet castled
156     if white_king_object.castled: #white king has castled
157         king_safety_score += castling_bonus
158     if black_king_object.castled: #black king has castled
159         king_safety_score -= castling_bonus
160
161     #checking the presence of pawn shield
162     for col_vector in col_vectors:
163         #check the white pawn shield first
164         new_row = white_king_pos[0] - 1
165         new_col = white_king_pos[1] + col_vector
166         if 0 <= new_row <= 7 and 0 <= new_col <= 7:
167             if board[new_row][new_col] == 'wp':
168                 king_safety_score += pawn_shield_bonus
169
170         #now check the black pawn shield
171         new_row = black_king_pos[0] + 1
172         new_col = black_king_pos[1] + col_vector
173         if 0 <= new_row <= 7 and 0 <= new_col <= 7:
174             if board[new_row][new_col] == 'bp':
175                 king_safety_score -= pawn_shield_bonus
176
177     return king_safety_score
178
179 def
calculate_total_score(self, material_score, centre_control_score, pawn_structure_score, king_safety_score):
180     multipliers = {'Opening':[3,0.2,0.3,0.1], ' Middlegame':[3,0.1,0.6,0.15], 'Endgame':[3,0,1,0]}
181     #multipliers tweaked for optimisation
182     #multipliers left to right: material score, centre control, pawn_structure, king safety
183
184     material_multiplier, centre_control_multiplier, pawn_structure_multiplier, king_safety_multiplier = \
185     multipliers[self.gamedata_object.game_phase]

```

```

186
187     material_score *= material_multiplier
188     centre_control_score *= centre_control_multiplier
189     pawn_structure_score *= pawn_structure_multiplier
190     king_safety_score *= king_safety_multiplier
191
192     total_score = material_score + centre_control_score + pawn_structure_score + king_safety_score
193     total_score = round(total_score,2)
194
195     return total_score
196
197 def minimax(self,board,is_maximising,depth,alpha,beta): #board = the main board object
198     # A.I is maximising and wants highest score if white,minimizing and wants lowest score if black
199
200     if self.gamedata_object.is_checkmate: #game has ended - return score
201         if self.gamedata_object.winner == 'w': #white wins
202             score = MAX_SCORE
203         elif self.gamedata_object.winner == 'b': # black wins
204             score = MIN_SCORE
205         else: #stalemate
206             score = 0
207
208     return score
209
210     if depth == 0: #maximum depth reached
211         return self.evaluation_function(board) #Evaluate the current board and find its current score
212
213     """
214     Iterate through all available moves. For each move, apply the following steps:
215     step 1. Perform the move on the board
216     step 2. Call the function recursively, decrementing the depth and flipping the isMaximising
variable
217     step 3. Undo the move on the board
218     step 4. If the score is higher/lower than the best score (depending on who is maximising),
219     replace the best score and store that move (if move is at highest depth)
220     step 5. Check if alpha >= beta. If true, prune all necessary moves
221     """
222
223     if is_maximising: #white is maximising, wants highest score
224         best_move_score = MIN_SCORE #we start with the worst possible score for white
225         for available_move in self.gamedata_object.current_valid_move_list:
226             self.move_engine.initiate_move(available_move) #step 1.
227             score = self.minimax(self.chessboard_object.digital_board,False,depth-1,alpha,beta) #step 2.
228             self.move_engine.undo_move() # step 3.
229             if score > best_move_score: #step 4.
230                 best_move_score = score
231             if depth == self.max_depth:
232                 self.best_move = available_move #new optimal move found, and is stored here

```

```
233
234     alpha = max(alpha,best_move_score) #step 5.
235     if beta <= alpha:
236         break
237
238 if not is_maximising: #black is minimising, wants lowest score
239     best_move_score = MAX_SCORE #we start with the worst possible score for white
240     for available_move in self.gamedata_object.current_valid_move_list:
241         self.move_engine.initiate_move(available_move) #step 1.
242         score = self.minimax(self.chessboard_object.digital_board,True,depth-1,alpha,beta) #step 2.
243         self.move_engine.undo_move() #step 3.
244
245         if score < best_move_score: # step 4.
246             best_move_score = score
247             if depth == self.max_depth:
248                 self.best_move = available_move #new optimal move found, and is stored here
249
250     beta = min(beta,best_move_score) #step 5.
251     if beta <= alpha:
252         break
253
254 return best_move_score
```

Gamedata.py

```

1  from pieces import check_for_checks
2
3  class gamedata: #this class stores all important information for the game that will be played
4      def __init__(self):
5          self.difficulty = "" #must be uppercase - first letter
6          self.player_colour = "" # one letter, w or b
7          self.ai_colour = ""
8
9          self.white_king_pos = (7,4)
10         self.black_king_pos = (0,4)
11         self.game_phase = 'Opening' # o = opening, m = middlegame, e = endgame
12         self.move_count = 0
13
14         self.is_check = False
15         self.check_change = False #if there was no check more before but now after/vice versa
16
17         self.is_checkmate = False
18         self.is_stalemate = False
19
20         self.white_move_list = []
21         self.black_move_list = []
22         self.current_valid_move_list = []
23
24         self.move_log_stack = []
25         self.capture_log_stack = []
26
27         self.winner = ""
28
29     def update(self,board,colour,class_dictionary,move_data,is_undo):
30         #this method will be called after every move is completed, and inspects and updates
31         #most of its attributes (if necessary)
32         #colour of player who has made move
33         self.set_is_check(board)
34         self.obtain_all_valid_moves(board,colour,class_dictionary,move_data)
35
36         if not is_undo: #regular move
37             self.move_count += 1
38             self.push_move_log(move_data)
39             self.current_valid_move_list = self.order_moves(board)
40         else: #undo command
41             self.move_count -= 1
42
43         self.update_gamestate(colour)
44         self.game_phase = self.update_gamephase(class_dictionary)
45
46     def obtain_all_valid_moves(self,board,colour,class_dictionary,move_data):
47         self.white_move_list = []

```

```

48     self.black_move_list = []
49     self.current_valid_move_list = [] #all moves of the next player AFTER the current move is made
50
51     changed_pieces = self.obtain_changed_pieces(board,colour,move_data)
52
53     if changed_pieces == 'all':
54         changed_pieces = class_dictionary.keys() #all pieces to be changed
55
56     for piece_pos in changed_pieces: #all pieces which their move list may have changed
57         piece_object = class_dictionary[piece_pos]
58         if piece_object.colour == 'w':
59             king_pos = self.white_king_pos
60         else:
61             king_pos = self.black_king_pos
62
63         ignore = piece_object.calculate_moves(board,king_pos)
64         #method call modifies the 'valid_move_list' attribute for the piece, which is what we want
65
66     for piece_pos in class_dictionary:
67         piece_object = class_dictionary[piece_pos]
68         piece_move_list = piece_object.valid_move_list
69         if len(piece_move_list) != 0:
70             if piece_object.colour == 'w':
71                 for move in piece_move_list:
72                     self.white_move_list.append(move)
73             else:
74                 for move in piece_move_list:
75                     self.black_move_list.append(move)
76
77     # we want all moves of the next player AFTER the current move is made
78     if colour == 'w': #after white's turn, black move now
79         self.current_valid_move_list = self.black_move_list
80     else: #after black's turn, white move now
81         self.current_valid_move_list = self.white_move_list
82
83     def obtain_changed_pieces(self,board,colour,move_data):
84         # this method obtains a record of every single piece which needs to have its moves re-evaluated
85         # after a move made
86         if move_data == 'first move': #first move
87             return 'all'
88
89         start_square = move_data[0]
90         end_square = move_data[1]
91
92         if colour == 'w':
93             enemy_colour = 'b'
94             friendly_king_pos = self.white_king_pos
95             enemy_king_pos = self.black_king_pos

```

```

96     else:
97         enemy_colour = 'w'
98         friendly_king_pos = self.black_king_pos
99         enemy_king_pos = self.white_king_pos
100
101     #if to/from check, or king has moved, search all piece moves
102     if board[end_square[0]][end_square[1]][1] == 'K' or self.is_check or self.check_change:
103         return 'all' #the signal that all pieces need to be re-evaluated for
104
105     changed_squares = {move_data[1]}
106     #destination square obviously needs to have its move re-evaluated
107
108     for square in move_data: #iterate through start and end squares
109         changed_squares = changed_squares | set(check_for_checks(board,'w',square,True))
110         #gets the location of all pieces that can jump to 'square' in a single move
111         #colour irrelevant in call, '|' means union
112
113         changed_squares = changed_squares | set(get_pawn_change_squares(board,square))
114         #all pawns that MAY be changed are added (as a precaution )
115
116         in_between_pieces_attacking =
get_in_between_squares_attacking(board,enemy_colour,list(square),list(enemy_king_pos))
117         if len(in_between_pieces_attacking) == 1: #only 1 piece between king an enemy piece, may be
(un)pinned
118             changed_squares = changed_squares | set(in_between_pieces_attacking)
119
120         in_between_pieces_defending =
get_in_between_squares_defending(board,colour,list(square),list(friendly_king_pos))
121         changed_squares = changed_squares | set(in_between_pieces_defending)
122
123     changed_squares = changed_squares | {enemy_king_pos}
124
125     return list(changed_squares)
126
127 def order_moves(self,board):
128     ordered_move_queue = []
129     capturing_moves = []
130     central_moves = []
131     king_moves = []
132     remaining_moves = []
133
134     central_squares = [(3,3),(3,4),(4,3),(4,4)]
135
136     for move in self.current_valid_move_list:
137         start_square = move[0]
138         piece_in_start_square = board[start_square[0]][start_square[1]]
139         end_square = move[1]
140         piece_in_end_square = board[end_square[0]][end_square[1]]

```

```

141
142     if piece_in_end_square != '00': #if move is a capturing move
143         capturing_moves.append(move)
144     elif end_square in central_squares: #if piece is moving to centre
145         central_moves.append(move)
146     elif piece_in_start_square[1] == 'K': #if the king is moving
147         king_moves.append(move)
148     else: #all other moves that were not any of the above
149         remaining_moves.append(move)
150
151     for capturing_move in capturing_moves:
152         ordered_move_queue.append(capturing_move) #enqueue capturing moves first
153     for central_move in central_moves:
154         ordered_move_queue.append(central_move) #enqueue central moves second
155     for remaining_move in remaining_moves:
156         ordered_move_queue.append(remaining_move) #enqueue remaining moves third
157     for king_move in king_moves:
158         ordered_move_queue.append(king_move) #enqueue king moves last
159
160     return ordered_move_queue
161
162 def update_gamestate(self, colour): #searches if there is a check/checkmate/stalemate
163     self.winner = ""
164     self.is_checkmate = False
165     self.is_stalemate = False
166
167     if len(self.current_valid_move_list) == 0: #game has ended
168         if self.is_check: #discovered earlier in another subroutine
169             self.is_checkmate = True
170
171         if colour == 'w': # white's turn when move was made
172             self.winner = 'w'
173         else: # black's turn when move was made
174             self.winner = 'b'
175             #print('Black wins!')
176
177     else:
178         self.is_stalemate = True
179
180 def update_gamephase(self, class_dictionary):
181     white_king_object = class_dictionary[self.white_king_pos]
182     black_king_object = class_dictionary[self.black_king_pos]
183
184     if self.move_count < 10 and not(white_king_object.castled and black_king_object.castled):
185         return 'Opening'
186
187     major_piece_count = 0
188     for piece_object in class_dictionary:

```

```

189     piece_name = type(class_dictionary[piece_object]).__name__
190     if piece_name != 'king' and piece_name != 'pawn': #if the piece is major
191         major_piece_count += 1
192
193     if major_piece_count > 3:
194         return 'Middlegame'
195
196     return 'Endgame'
197
198     def set_is_check(self,board):
199         if check_for_checks(board,'w',self.white_king_pos) or
check_for_checks(board,'b',self.black_king_pos):
200             #check the safety of both kings
201
202             self.is_check = True
203         else:
204             self.is_check = False
205
206         self.check_change = False
207
208         if check_for_checks(board,'w',self.white_king_pos) or
check_for_checks(board,'b',self.black_king_pos):
209             #check the safety of both kings
210             if not self.is_check:
211                 self.check_change = True
212
213             self.is_check = True
214         else:
215             if self.is_check:
216                 self.check_change = True
217
218             self.is_check = False
219
220     def log_captured_piece(self,capturing_square,class_dictionary): #capturing square = end square
221         if (capturing_square[0],capturing_square[1]) in class_dictionary:
222             #if there is an object at the end square, not empty
223
224             captured_piece = class_dictionary[(capturing_square[0],capturing_square[1])]
225             self.push_capture_log(captured_piece)
226             #object of captured piece is saved into the log (including its unique properties)
227             #can be re-gathered and re-implemented into the game at any time
228         else:
229             self.push_capture_log('.') # '.' means that no piece was captured this turn
230
231     def push_capture_log(self,captured_piece):
232         self.capture_log_stack.append(captured_piece)
233
234     def pop_capture_log(self):

```

```

235     return self.capture_log_stack.pop()
236
237 def push_move_log(self,move_data):
238     self.move_log_stack.append(move_data)
239
240 def pop_move_log(self):
241     return self.move_log_stack.pop()
242
243 def get_difference_vector(target_square,king_pos):
244     row_gain = target_square[0] - king_pos[0]
245     col_gain = target_square[1] - king_pos[1]
246     if not (abs(col_gain) == abs(row_gain) or col_gain == 0 or row_gain == 0):
247         return [] #target square is not diagonal or straight relative to the king, i.e. no in between squares
248
249     if row_gain == 0:
250         row_vector = 0
251     else:
252         row_vector = row_gain // abs(row_gain)
253     if col_gain == 0:
254         col_vector = 0
255     else:
256         col_vector = col_gain // abs(col_gain)
257
258     #difference_vector such that king_pos + difference_vector*k = target_square
259     #where k = no.squares in between the two squares + 1
260     difference_vector = (row_vector,col_vector)
261
262     return difference_vector
263
264 def get_in_between_squares_attacking(board,colour,target_square,king_pos):
265     in_between_pieces = []
266     difference_vector = get_difference_vector(target_square,king_pos)
267     if difference_vector == []:
268         return [] #no pins as target square not in straight line relative to king
269
270     while king_pos != target_square:
271         king_pos[0] += difference_vector[0]
272         king_pos[1] += difference_vector[1]
273         if board[king_pos[0]][king_pos[1]][0] == colour:
274             in_between_pieces.append((king_pos[0],king_pos[1]))
275
276     return in_between_pieces
277
278 def get_in_between_squares_defending(board,colour,target_square,king_pos):
279     in_between_pieces = []
280     difference_vector = get_difference_vector(target_square,king_pos)
281     if difference_vector == []:
282         return [] #no pins as target square not in straight line relative to king

```

```

283
284 while 0 <= king_pos[0] <= 7 and 0 <= king_pos[1] <= 7:
285     if board[king_pos[0]][king_pos[1]][0] == colour:
286         in_between_pieces.append((king_pos[0],king_pos[1]))
287
288     elif board[king_pos[0]][king_pos[1]] != '00': #enemy piece between target square/king - potential
pin
289         in_between_pieces.pop(0) #remove first item (the king), as we do not want to count it
290     return in_between_pieces
291
292     king_pos[0] += difference_vector[0]
293     king_pos[1] += difference_vector[1]
294
295 return [] #no enemy piece between the target square and king inclusive - no threat
296
297 def get_pawn_change_squares(board,target_square):
298     changed_pawns = [] #stores the squares which contains pawns that can move directly to
'target_square'
299     movement_vectors = [(-1,1),(-1,-1),(-1,0),(-2,0), (1,1),(1,-1),(1,0),(2,0)] #movement vectors for pawns
300     # 1-4: white pawns|5-8: black pawns
301
302     for vector_no,vector in enumerate(movement_vectors):
303         new_row,new_col = target_square[0]+vector[0], target_square[1]+vector[1]
304         if 0 <= new_row <= 7 and 0 <= new_col <= 7:
305             if board[new_row][new_col][1] == 'p':
306                 if board[new_row][new_col][0] == 'b' and 0 <= vector_no <= 3:
307                     #black pawn (which move DOWN) diagonally or vertically ABOVE the square
308                     changed_pawns.append((new_row,new_col))
309                 elif board[new_row][new_col][0] == 'w' and 4 <= vector_no <= 7:
310                     #white pawn (which move UP) diagonally or vertically BELOW the square
311                     changed_pawns.append((new_row,new_col))
312
313     return changed_pawns

```